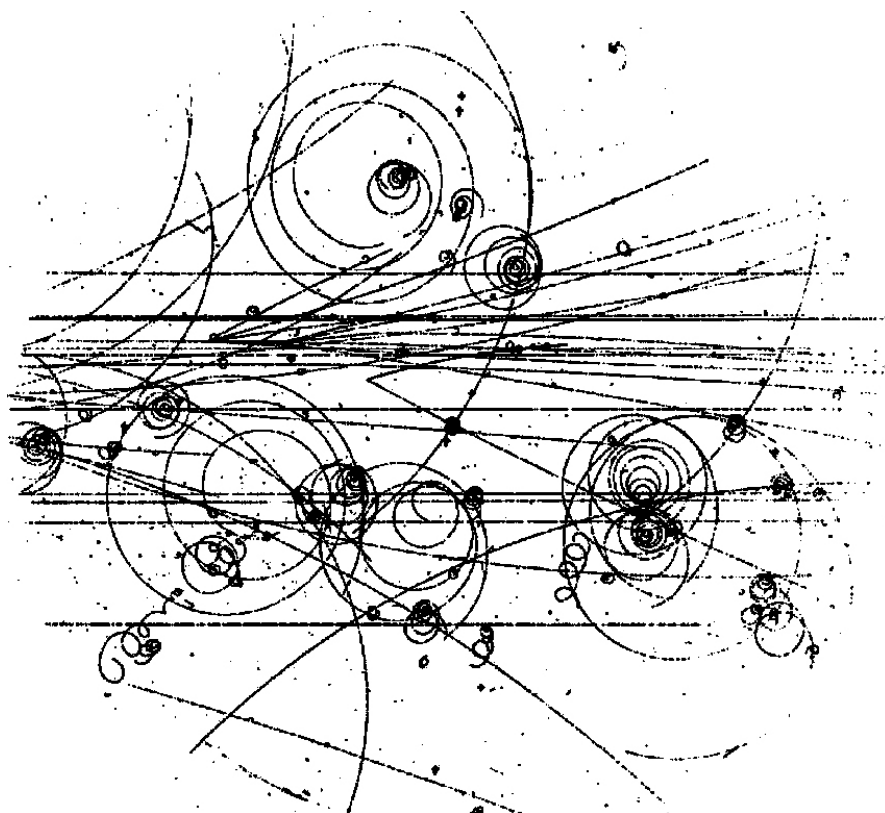


DAVIDE CAMINO

PROGRAMMAZIONE LOGICA PER INFERENZA
SU COMPUTER QUANTISTICI



TITLEBACK

Immagine in copertina di Gordon Fraser/CERN

A tutti quelli a cui voglio bene.

*"If you think you understand quantum mechanics,
you don't understand quantum mechanics."*
— Richard P. Feynman

Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata.

ABSTRACT

In questa tesi rivisitiamo ed estendiamo il lavoro di Scott Pakin presentato in [1]. Questo lavoro presenta una pipeline di riscrittura per passare da un linguaggio di alto livello, Prolog, a una formulazione compatibile con la risoluzione su quantum annealers, un particolare tipo di computazione quantistica adiabatica quantistico adiabatico, ovvero hardware quantistico specializzato nella risoluzione di problemi di ottimizzazione.

In questa tesi partiamo dal lavoro di Pakin rendendolo nuovamente compatibile con i framework utilizzati per interfacciarsi con i quantum annealers. Una volta ripristinata la compatibilità della pipeline, la estendiamo sia a monte che a valle.

A monte: riscriviamo un'ontologia, un particolare tipo di base di conoscenza, in Prolog. Presentiamo quindi alcune soluzioni per rendere questa traduzione compatibile con la pipeline di Pakin, che non implementa l'intero insieme delle funzionalità di Prolog. Infine, studiamo la dimensione del problema ottenuto in funzione dell'ontologia che traduciamo e della query utilizzata per interrogare la base di conoscenza.

A valle: mostriamo che l'output della pipeline non è limitato alla risoluzione tramite annealing, ma che una formulazione come problema di ottimizzazione consente anche l'applicazione di algoritmi sviluppati per l'altro tipo di architettura quantistica (quella basata su porte quantistiche). Mostriamo come passare da una formulazione all'altra e come eseguire alcuni esperimenti per studiare il comportamento di questo approccio.

Vediamo brevemente come è organizzato il lavoro. I primi tre capitoli forniscono un'introduzione teorica. Nel Capitolo 1 forniamo alcune basi di meccanica quantistica; l'obiettivo di questo capitolo è mostrare che le trasformazioni fisiche, e quindi le porte quantistiche, devono essere reversibili. Nel Capitolo 2 introduciamo l'analisi computazionale, definendo le classi di complessità computazionale e mostrando dove i computer quantistici si collocano in questa analisi. Il capitolo si conclude con la presentazione dei due paradigmi del calcolo quantistico: il calcolo basato su porte quantistiche e la computazione quantistica adiabatica. Il Capitolo 3 presenta le ontologie, mostrando le principali caratteristiche di queste basi di conoscenza e discutendo la complessità

di estrarre informazioni da esse.

Passiamo quindi a due capitoli in cui presentiamo gli strumenti utilizzati nel resto del lavoro. Nel Capitolo 4 mostriamo come configurare un ambiente di sviluppo con tutto il software e le librerie necessarie per condurre i nostri esperimenti. Il Capitolo 5 presenta la pipeline di Pakin per la traduzione da Prolog a un problema di ottimizzazione. Esaminiamo le caratteristiche di ciascuna fase della pipeline e presentiamo il nostro contributo per ripristinare la compatibilità tra la pipeline e gli altri software con cui si interfaccia.

Gli ultimi due capitoli riguardano gli esperimenti condotti in questo lavoro. Il Capitolo 6 mostra la traduzione di un'ontologia in Prolog e da Prolog a un problema di ottimizzazione. Il Capitolo 7 tratta la versione discretizzata del quantum annealing implementabile su sistemi basati su porte quantistiche. Mostriamo come partire dal risultato della pipeline di Pakin e come ottenere una formulazione adatta a questo paradigma.

Nelle conclusioni (Capitolo 8) rivediamo i risultati ottenuti e forniamo alcuni spunti per lavori futuri.

ACKNOWLEDGEMENTS

Questa tesi non esisterebbe se non avessi ricevuto il supporto di tantissime persone. Supporto tecnico da parte di Professori, Dottorandi e Compagni di corso che mi hanno aiutato ad appassionarmi alle materie studiate, a capirle meglio, a ripassarle, e a superare gli esami. Supporto morale da parte della mia Famiglia e degli Amici che mi hanno sostenuto, accompagnato, e fatto crescere.

Desidero, in questa pagina, ringraziare il professor Roversi e la professoressa Bono che sono stati, innanzitutto, degli ottimi insegnanti, mi hanno fatto amare le loro materie e i loro corsi. Con la professoressa Bono ho scritto anche la tesi triennale: le sono davvero grato per avermi contattato lei per prima e aver sciolto le mie incertezze. Con entrambi ho invece scritto questa tesi, ho potuto farlo con serenità e con i miei tempi, sentendomi sempre seguito, e sapendo che il mio lavoro era controllato e corretto in tempi brevissimi. Mi sento molto fortunato ad aver trovato un gruppo di lavoro del genere. Gruppo che è costituito anche dai dottorandi, in particolare mi preme ringraziare Mario e Matteo che mi hanno dato preziosissimi consigli durante la scrittura.

Un pensiero non può che andare alla mia famiglia, al Papà e alla Mamma che mi vogliono bene e mi supportano a prescindere da tutto, ai Nonni che mi spronano e credono sempre in me e alla Sorellina che, anche se è in giro per il mondo, so che c'è sempre per me.

Un grosso grosso grazie va poi agli amici, dei quali sono veramente tanto orgoglioso, alcuni veramente di vecchissima data che mi porto nel cuore dalle elementari. Io, Eddi, Emi e Luca siamo cresciuti assieme ormai molto più di quanto non siamo cresciuti da soli. Abbiamo giocato, riso, sbagliato e vissuto assieme esperienze che mi hanno fatto diventare chi sono. Al liceo, ho conosciuto, tra gli altri, Tom e Fra che sono stati miei compagni di classe, Lore, Marco, Alessandro che sono stati compagni di scuola, e Sara che ho conosciuto a Bardonecchia. Con loro, e tanti tanti altri, ho passato cinque anni bellissimi e ho scoperto più in là che le amicizie, quelle vere, non vengono degradate dal tempo e che basta un messaggio per ritrovarsi come se non fosse passato un giorno. Anche l'università è stata occasione per stringere amicizie e ho trovato persone stupende come Amedeo, Matilde, Asia e Margherita a fisica, e Lorenzo, Angelo e Andrea alla triennale di informatica, devo tantissimo del mio percorso a loro. In magistrale ho avuto la fortuna di incontrare Andrea, senza il quale non avrei passato la maggior parte degli esami e con cui ho condiviso molto più che solo lo studio.

Infine un grande grazie e un grosso bacio va ad Alice che mi è stata sempre vicina, mi ha dato supporto nei momenti difficili e ha sopportato le mie crisi e le mie lagne. Mi dà serenità e mi fa sentire amato, e per questo le sono infinitamente grato. Inoltre se la tesi è vagamente piacevole da leggere tanto del merito è suo.

Grazie a tutti

INDICE

ABSTRACT	v
ACKNOWLEDGEMENTS	vii
ACRONIMI	xiii

I THEORETICAL BASES

1	MECCANICA QUANTISTICA	3
1.1	Esperimenti	3
1.1.1	Spin	3
1.1.2	Qubit	5
1.1.3	Logica Booleana	5
1.2	Stati quantistici	7
1.2.1	Spazi Vettoriali	7
1.2.2	Bra e Ket	7
1.2.3	Variabili nascoste	8
1.2.4	Stati di spin	9
1.3	Osservabili	11
1.3.1	Operatore hermitiano	12
1.3.2	Principi della meccanica quantistica	13
1.3.3	Operatore di Spin	14
1.3.4	Teoria ed esperimenti	15
1.3.5	Operatori e Misure	16
1.4	Evoluzione Temporale	17
1.4.1	Unitarietà	17
1.4.2	Operatore di Evoluzione Temporale	18
1.4.3	L'Hamiltoniano	19
1.4.4	Commutatori	20
1.4.5	Conservazione dell'Energia	21
1.5	Conclusioni	21
2	COMPLESSITÀ COMPUTAZIONALE E CALCOLO QUANTISTICO	23
2.1	Introduzione all'analisi computazionale	24
2.1.1	Modelli computazionali	24
2.1.2	Problemi computazionali	27
2.1.3	Complessità computazionale quantistica	31
2.2	Paradigma basato su porte quantistiche	33
2.2.1	Porte reversibili classiche	33
2.2.2	Porta quantistica	34
2.2.3	Algoritmi	39
2.3	Calcolo Adiabatico Quantistico	42
2.3.1	Algoritmo CAQ	42
2.3.2	Risultati teorici	43
2.3.3	Quantum annealing	45
2.4	Conclusione	48
3	ONTOLOGY	51
3.1	Knowledge Base	51

3.1.1	Ontology in philosophy	51
3.1.2	Ontology in computer science	51
3.1.3	OWL Language	52
3.1.4	Importance of ontologies	53
3.2	Example ontologies	53
3.2.1	Simple ontology	53
3.2.2	DOLCE ontology	55
3.3	Reasoning on ontology	55
3.3.1	<i>SROIQ</i> DL	56
3.3.2	Interpretation of a knowledge base	56
3.3.3	Complexity of reasoning	57
3.4	Conclusions	58

II TOOLS

4	CONFIGURAZIONE DELL'AMBIENTE	61
4.1	Ambiente Python	61
4.2	IBM Qiskit	61
4.2.1	Hello World	61
4.2.2	Transpilazione	62
4.2.3	Esecuzione	63
4.3	D-Wave Ocean	64
4.3.1	Hello World	64
4.4	PyQUBO e qubover	65
4.4.1	PyQUBO	66
4.4.2	qubover	67
4.5	Conclusione	68
5	QA-PROLOG	69
5.1	Il progetto	69
5.1.1	Motivazione	69
5.1.2	Prolog	70
5.1.3	Caratteristiche di QA-Prolog	72
5.1.4	Lavori correlati	72
5.2	Pipeline	73
5.2.1	QMASM	74
5.2.2	Yosys e edif2qmasm	77
5.2.3	QA-Prolog	79
5.2.4	Panoramica	80
5.3	Aggiornamento al progetto	81
5.3.1	Ripristino di QMASM	81
5.3.2	Correzione dell'interazione edif2qmasm-QMASM	82
5.3.3	Guida all'installazione	83
5.4	Conclusione	84

III EXPERIMENTS

6	UN'ONTOLOGIA QUANTISTICA	87
6.1	Ontologia di partenza	87
6.1.1	Struttura dell'ontologia	87
6.1.2	Vincoli dell'ontologia	88
6.1.3	Contenuto dell'ontologia	88

6.2	Versione Prolog	90
6.2.1	Classi e sottoclassi	90
6.2.2	Individui e le loro relazioni	91
6.2.3	Altre regole	92
6.2.4	Risultato finale	94
6.3	Versione QA-Prolog	94
6.3.1	Liste e termini complessi	94
6.3.2	Unfolding	95
6.3.3	Criterio di arresto	96
6.3.4	Equivalenza tra le KB	97
6.4	Risultato finale	98
6.4.1	Query	99
6.4.2	Risultati	99
6.4.3	Impatto dell'unfolding sulle prestazioni	100
6.5	Conclusione	103
7	QAOA	105
7.1	QAOA	105
7.1.1	Circuito	106
7.1.2	Ottimizzazione dei parametri	107
7.2	Dal CAQ al QAOA	107
7.2.1	Obiettivo	107
7.2.2	Da QUBO a Ising	108
7.2.3	Hamiltoniano come operatore di Pauli	109
7.2.4	Circuito finale	109
7.2.5	Ottimizzazione dei parametri	110
7.2.6	Recupero dei risultati	112
7.3	Esperimenti	112
7.3.1	Problema	112
7.3.2	Esecuzione	112
7.3.3	Risultati	114
7.4	Conclusione	115
8	CONCLUSION	117
8.1	Results achieved	117
8.1.1	Updating QA-Prolog	117
8.1.2	Ontology in Prolog	117
8.1.3	QAOA Algorithm	118
8.1.4	A rewriting process	118
8.2	Future work	118
8.2.1	Testing on quantum hardware	118
8.2.2	Increasing QMASM compatibility	119
IV APPENDIX		
A	CODE	123
	LIST OF FIGURES	125
	LIST OF LISTINGS	127
	BIBLIOGRAPHY	129

ACRONIMI

AQC

Adiabatic Quantum Computing (*Computazione quantistica adiabatica*).

KB

Knowledge Base (*Base di conoscenza*).

QA

Quantum Annealing.

QAOA

Quantum Approximation Optimization Algorithm (*Algoritmo di ottimizzazione quantistica approssimato*).

Parte I

THEORETICAL BASES

In questo capitolo esploreremo le basi della meccanica quantistica al fine di comprendere cosa possiamo o non possiamo fare con un computer quantistico. L'architettura di riferimento per questo lavoro è il computer quantistico basato su porte quantistiche. Nelle prossime pagine cercheremo di giustificare perché gli algoritmi che vengono eseguiti su questo hardware devono essere reversibili.

Il capitolo inizia con la definizione di sistema quantistico e presenta le basi per comprendere l'evoluzione di un sistema quantistico, cercando di giustificare perché ogni evoluzione deve essere reversibile e cosa significa esattamente la reversibilità. Alla fine del capitolo metteremo insieme tutte le nostre nuove conoscenze per derivare la famosa equazione di Schrödinger.

Grazie ai concetti illustrati saremo in grado di immaginare il funzionamento di una porta quantistica e comprendere i limiti che vengono imposti quando sviluppiamo un algoritmo per un computer quantistico.

Questo capitolo è ispirato a [2], in particolare ai capitoli 1, 2, 3 e 4 del libro originale. Per questo motivo, nel testo non compariranno citazioni in quanto risulterebbero ridondanti.

1.1 ESPERIMENTI

Iniziamo la nostra introduzione alla meccanica quantistica con un esperimento. Gli esperimenti non sono solo un pretesto per introdurre l'argomento, ma la chiave essenziale della fisica, sia classica che moderna.

La teoria e i modelli devono adattarsi agli esperimenti, e quando i risultati sperimentali sono in contraddizione con il modello attuale significa che il modello deve essere modificato per rispettare il comportamento del mondo.

1.1.1 Spin

Analizziamo l'esperimento di un elettrone in un campo magnetico. Un elettrone è una particella elettricamente carica; quando alcuni elettroni vengono sparati in un campo elettrico tutti sono influenzati dalla legge di Coulomb; se tutti gli elettroni hanno la stessa velocità iniziale il fascio di elettroni rimane intatto.

Cosa succede allo stesso fascio in un campo magnetico? Anche in questo caso gli elettroni vengono deflessi da una forza, ma questa volta il fascio si divide. Se la velocità iniziale era parallela all'asse x e il

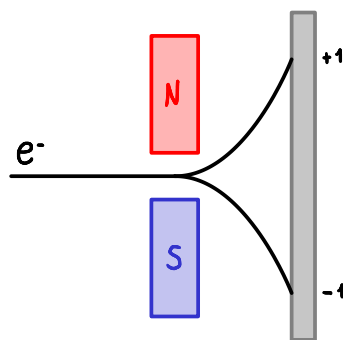


Figura 1.1: Schema dell'esperimento

campo magnetico è orientato lungo l'asse z , alcuni elettroni vengono deflessi verso l'alto, altri verso il basso, ma l'intensità della deflessione è la stessa per tutti gli elettroni. Ciò significa che nessun elettrone viene deflesso meno o più degli altri e il fascio si divide esattamente in due parti.

Partendo da questo esperimento possiamo costruire uno strumento di misura: questo apparato $\mathcal{A}$ può essere orientato lungo un asse arbitrario, e nella configurazione precedente $\mathcal{A}$ visualizza $+1$ se l'elettrone viene deflesso verso l'alto, -1 altrimenti. Chiamiamo questo numero lo spin dell'elettrone.

Ripetibilità della misura

Se misuriamo lo spin di un elettrone e $\mathcal{A}$ visualizza $+1$, possiamo confermare i risultati dell'esperimento misurando nuovamente lo spin e otteniamo spin $+1$ ogni volta. Ciò significa che le misurazioni sono ripetibili (una proprietà essenziale per costruire modelli e fare previsioni). Possiamo pensare, e sarà chiaro più avanti perché è utile, che il primo esperimento prepara lo spin $+1$ e gli altri confermano questo risultato.

Lo spin è una proprietà quantistica e tutte le rappresentazioni visive come la rotazione dell'elettrone attorno al proprio asse porterebbero a una rappresentazione errata. Spin e rotazione, tuttavia, hanno alcune somiglianze. Analizziamo cosa accadrebbe se considerassimo una sfera carica in un campo magnetico con le leggi della fisica classica. Consideriamo una sfera che ruota attorno al proprio asse, e questo asse è parallelo all'asse z . La componente x o y del momento angolare è zero. Misurando la componente lungo un asse generico, orientato come il versore $\hat{n}^1$, otterremmo un risultato proporzionale alla proiezione di $\hat{z}$ su $\hat{n}$. Questa proiezione può essere trovata con il prodotto scalare $\hat{z} \cdot \hat{n} = \cos \theta^2$, dove θ è l'angolo tra gli assi.

Consideriamo ora la versione quantistica di questo fenomeno. Iniziamo misurando la componente z dello spin e assumiamo che il risultato sia $\sigma_z = +1$; se ruotiamo l'apparato $\mathcal{A}$ lungo, ad esempio, all'asse x , possiamo misurare σ_x . In questo caso non otteniamo zero come nel caso classico, ma $\mathcal{A}$ continua a visualizzare solo $+1$ o -1 . Il singolo risultato non ci aiuta a capire il fenomeno, ma possiamo ripetere l'esperimento più volte, ovvero:

1. orientare $\mathcal{A}$ lungo l'asse z e preparare uno spin $\sigma_z = +1$
2. ruotare $\mathcal{A}$ lungo l'asse x
3. misurare la componente x dello spin

statisticamente osserveremmo lo stesso numero di $\sigma_x = +1$ e $\sigma_x = -1$, cioè il valore medio di σ_x è zero.

Se iniziamo gli esperimenti con uno spin preparato come $\sigma_z = +1$ e poi orientiamo $\mathcal{A}$ lungo un asse generico $\hat{n}$, ogni misura sarebbe binaria e imprevedibile, ma la media delle misure tende a $\hat{z} \cdot \hat{n} = \cos \theta$ dove θ è l'angolo tra $\hat{z}$ e $\hat{n}$. Nel caso più generale possiamo iniziare con l'apparato orientato come $\hat{m}$ e preparare lo spin $\sigma_m = +1$, poi ruotiamo $\mathcal{A}$ in modo che sia parallelo a $\hat{n}$ senza interferire con lo spin e misuriamo di nuovo; otterremmo il risultato statistico $\langle \sigma_n \rangle = \hat{m} \cdot \hat{n}^3$.

1 Un versore è un vettore di modulo 1 (vettore unitario), è normalmente usato per specificare una direzione.

2 Possiamo usare direttamente l'angolo perché stiamo considerando versori.

3 La parentesi di Dirac $\langle \rangle$ denota la media statistica di una quantità. La chiamiamo valore di aspettazione.

81 Il risultato di una singola misura è non deterministico, ma possiamo fare
 82 previsioni sui valori medi delle misure: i valori attesi si comportano come i
 83 singoli risultati dell'esperimento classico.

84 Considerando ora una sequenza di tre misure: partendo con $\mathcal{A}$ orientato
 85 lungo z prepariamo lo spin $\sigma_z = +1$, poi ruotiamo $\mathcal{A}$ per misurare σ_x ot-
 86 tenendo, diciamo, $+1$ (il ragionamento è lo stesso se otteniamo -1); infine
 87 riportando $\mathcal{A}$ parallelo a z , non possiamo fare alcuna previsione sul singo-
 88 lo risultato, la configurazione iniziale (con $\sigma_z = +1$) è perduta per sempre,
 89 l'unico risultato che possiamo prevedere è che $\langle \sigma_z \rangle = 0$.

Esperimento invasivo

90 1.1.2 Qubit

91 Abbiamo introdotto lo spin riferendoci agli elettroni in un campo magneti-
 92 co. Tuttavia, possiamo studiare lo spin senza esaminare l'elettrone associato;
 93 abbiamo isolato un semplice sistema fisico, il più semplice che possiamo
 94 studiare.

95 Lo spin appartiene a una classe di semplici sistemi fisici chiamati *qubit*;
 96 in tutti questi sistemi il risultato di una misura è binario. Vedremo che,
 97 anche se il risultato di una misura è uguale al *bit* classico, il sistema qubit è
 98 descritto in modo molto diverso rispetto al suo alter ego classico.

99 1.1.3 Logica Booleana

100 In questo paragrafo cerchiamo di capire perché abbiamo bisogno di due mo-
 101 di diversi per descrivere lo spazio degli stati: uno classico e uno quantistico.
 102 Per farlo analizziamo i risultati di alcune proposizioni logiche, sia elementari
 103 che composte tramite connettivi logici.

104 Partendo dal caso classico consideriamo un sacchetto di palline colora-
 105 te e numerate. Possiamo costruire lo spazio degli stati enumerando tutti
 106 gli stati, ovvero estraendo ogni pallina dal sacchetto e annotando la coppia
 107 numero-colore. Le proposizioni elementari che analizziamo sono:

- 108 • La pallina estratta è rossa.
- 109 • Il numero sulla pallina estratta è pari.

110 Se consideriamo uno stato particolare possiamo dire se una proposizione
 111 è vera o falsa; possiamo anche definire due sottoinsiemi di palline, il primo
 112 con tutte le palline rosse (per questo sottoinsieme la prima proposizione è
 113 vera), il secondo con le palline che mostrano un numero pari (sottoinsieme
 114 che rende vera la seconda proposizione). Considerando ora disgiunzione e
 115 congiunzione:

- 116 • La pallina estratta è rossa OR pari.
- 117 • La pallina estratta è rossa AND pari.

118 Anche in questo caso è semplice associare un valore di verità a queste pro-
 119 posizioni se consideriamo un singolo stato; possiamo anche costruire due
 120 sottoinsiemi che soddisfano le proposizioni a partire dai sottoinsiemi de-
 121 finiti in precedenza: i nuovi sottoinsiemi sono rispettivamente l'unione e
 122 l'intersezione di quelli precedenti.

Nel mondo quantistico la situazione è molto diversa. Partiamo da proposizioni che possono essere verificate con un semplice esperimento:

- La componente z dello spin è $+1$.
- La componente x dello spin è $+1$.

Se vogliamo verificare la prima proposizione possiamo orientare l'apparato $\mathcal{A}$ lungo z ed effettuare una misura; la stessa procedura può essere seguita per la seconda proposizione. La disgiunzione e la congiunzione di queste proposizioni sono:

- La componente z dello spin è $+1$ OR la componente x è $+1$.
- La componente z dello spin è $+1$ AND la componente x è $+1$.

Iniziamo con la disgiunzione. Consideriamo uno stato preparato, a nostra insaputa, con $\sigma_z = +1$. Se la nostra prima misura è lungo l'asse z , $\mathcal{A}$ visualizzerà sempre $+1$ e possiamo concludere immediatamente che la proposizione è vera. Se iniziamo misurando la componente x , abbiamo il 50% di probabilità che $\mathcal{A}$ visualizzi $+1$ o -1 ; inoltre questa misura distrugge lo stato iniziale e la misura di σ_z diventa non prevedibile. In questo scenario abbiamo il 25% di probabilità di dedurre che la proposizione è falsa; la Figura 1.2 mostra tutti i possibili risultati di misura in questo caso. Il valore logico della proposizione dipende dall'ordine in cui eseguiamo le misure.

La disgiunzione non è commutativa

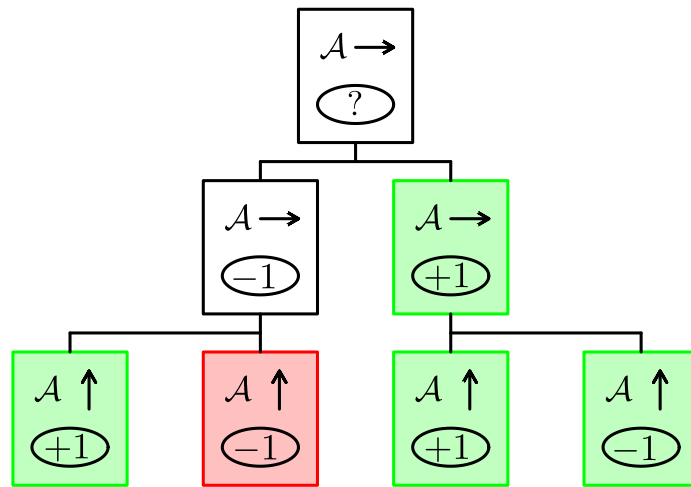


Figura 1.2: L'apparato $\mathcal{A}$ è rappresentato come un riquadro, la freccia rappresenta la direzione lungo cui l'apparato è orientato, il display (ellisse) mostra il risultato della misura. Abbiamo evidenziato in verde i casi in cui possiamo concludere immediatamente che la disgiunzione delle proposizioni è vera

La congiunzione è ancora peggio: indipendentemente dall'ordine delle misure, la seconda distrugge il risultato della prima. La disgiunzione è vera se almeno una delle sotto-proposizioni è vera, e se troviamo una componente dello spin che vale $+1$ possiamo sempre confermare questo risultato con un'altra misura. Nella congiunzione le due sotto-proposizioni devono essere vere *allo stesso tempo*, ma con la seconda misura perdiamo tutta la conoscenza della prima. Non possiamo mai concludere che la congiunzione è vera.

La congiunzione perde il suo significato

149 1.2 STATI QUANTISTICI

150 Nella sezione precedente abbiamo compreso che uno spazio degli stati di un
 151 sistema quantistico non può essere rappresentato nello stesso modo di uno
 152 spazio degli stati classico. Presentiamo ora un modello matematico formale
 153 per descrivere lo spazio degli stati dello spin.

154 **Axiom 1.** *Lo spazio degli stati per un sistema quantistico è uno spazio vettoriale*
 155 *complesso.*

156 Questo è un assioma fisico, il che significa che è vero perché ci sono mol-
 157 ti esperimenti che confermano questo modello e nessuno che mostri una
 158 contraddizione.

159 1.2.1 Spazi Vettoriali

160 Uno spazio vettoriale è una costruzione matematica e astratta che può avere
 161 più dimensioni (anche infinite) e ha, come componenti, numeri interi, reali
 162 o complessi, o altri elementi. possiamo capire quanto possa essere astratto
 163 uno spazio vettoriale considerando le funzione continue a valori complessi
 164 della variabile x ; l'insieme di queste funzioni genera uno spazio vettoriale.

165 In meccanica quantistica lo spazio degli stati è descritto da uno spazio
 166 vettoriale avente come elementi i *ket* indicati con $|A\rangle$. Le proprietà di questo
 167 spazio sono:

Spazio di Hilbert

- 168 • la somma di due ket è un ket;
- 169 • l'addizione è commutativa;
- 170 • l'addizione è associativa;
- 171 • esiste l'elemento neutro per l'addizione;
- 172 • esiste l'elementi inverso per l'addizione;
- 173 • esistenza l'elemento neutro per la moltiplicazione scalare;
- 174 • vale la proprietà di linearità.

175 1.2.2 Bra e Ket

176 Un esempio di ket che incontreremo spesso è il vettore colonna di due
 177 dimensioni:

$$|A\rangle = \begin{pmatrix} \alpha_1 \\ \alpha_2 \end{pmatrix}$$

178 dove α_1 e α_2 sono numeri complessi. Con questo semplice esempio di ket è
 179 facile verificare la validità di tutte le proprietà descritte in precedenza.

180 Se, per i numeri complessi, esiste il complesso coniugato, per ogni ket
 181 esiste un *bra*. L'insieme dei bra genera uno spazio duale coniugato rispetto
 182 allo spazio degli stati dei ket. Denotiamo un bra come $\langle A|$. Se $|A\rangle$ è il ket

183 dell'esempio precedente, il bra corrispondente è un vettore riga avente come
184 elementi i complessi coniugati di $|A\rangle$:

$$\langle A| = (\alpha_1^*, \alpha_2^*).$$

Prodotto interno 185 Il nome e il simbolo associati agli elementi degli spazi di Hilbert diventano
186 chiari quando definiamo il prodotto *bra-ket*, che l'operazione corrispondente
187 al prodotto scalare di un vettore ordinario ed è chiamato *prodotto interno*.
188 Considerando bra e ket di due dimensioni possiamo calcolare il prodotto
189 interno sommando i prodotti delle componenti corrispondenti:

$$\langle A|B\rangle = (\alpha_1^*, \alpha_2^*) \cdot \begin{pmatrix} \beta_1 \\ \beta_2 \end{pmatrix} = \alpha_1^* \beta_1 + \alpha_2^* \beta_2.$$

190 Avendo introdotto il prodotto interno possiamo definire:

191 **VERSORI** vettori normalizzati: $|A\rangle$ in cui $\langle A|A\rangle = 1$;

192 **VETTORI ORTOGONALI** vettori il cui prodotto interno è nullo: $\langle A|B\rangle = 0$.

193 Abbiamo familiarità con questi concetti in due e tre dimensioni, i versori
194 sono vettori di lunghezza 1, vettori ortogonali sono vettori che formano un
195 angolo di 90° . Questa rappresentazione è fuorviante nel nostro caso, non
196 possiamo immaginare un ket come una freccia e lo spazio degli stati è com-
197 pletamente astratto anche se vi sono proprietà e operazioni in comune tra
198 questo spazio e lo spazio 3D a cui siamo abituati.

199 Abbiamo perso l'interpretazione geometrica, e sembra che abbiamo defi-
200 nito due concetti completamente astratti e poco utili; vedremo in seguito che
201 questi sono concetti chiave nella descrizione dei sistemi quantistici e hanno
202 un significato fisico preciso e importante.

Base ortonormale 203 Avendo uno spazio vettoriale è possibile costruire un insieme di versori
204 ortogonali che genera tutti i vettori nello spazio dato. Questo insieme è chia-
205 mato base ortonormale e la cardinalità dell'insieme è uguale alla dimensione
206 dello spazio.

207 Formalmente, avendo una base $\mathcal{B} = \{|i_1\rangle, |i_2\rangle, \dots, |i_N\rangle\}$ di uno spazio
208 con N dimensioni, possiamo scrivere un vettore generico in quello spazio
209 come

$$|A\rangle = \sum_{n=1}^N \alpha_n |i_n\rangle = \sum_{n=1}^N |i_n\rangle \langle i_n|A\rangle \quad (1.1)$$

210 questa è la combinazione lineare dei versori della base; dove i ket $|i_n\rangle$ sono
211 i versori della base e α_n sono le componenti del vettore. Possiamo ottenere
212 queste componenti con il prodotto interno tra il vettore $|A\rangle$ e i versori della
213 base:

$$\alpha_i = \langle i|A\rangle. \quad (1.2)$$

214 1.2.3 Variabili nascoste

215 In fisica classica possiamo misurare tutte le variabili associate a un sistema
216 fisico e poi fare una previsione deterministica dell'evoluzione di quel siste-
217 ma. Dagli esperimenti descritti nella prima sezione abbiamo appreso che un

sistema quantistico non è completamente prevedibile anche se possiamo effettuare tutte le misure che vogliamo⁴. Possiamo perciò chiederci se le nostre misurazioni siano davvero sufficienti, se non esistano invece altre variabili che possano rendere la previsione completamente deterministica. Su questo argomento non abbiamo alcuna prova sperimentale, l'opinione dei fisici è divisa in due visioni principali:

OPINIONE UNO : esistono variabili nascoste che una volta misurate rendono la previsione dei risultati deterministica. Queste variabili possono essere

- molto difficili da misurare;
- inconoscibili per noi perché anche noi siamo costituiti da materia quantistica.

OPINIONE DUE : le variabili nascoste non esistono, conosciamo già tutte le informazioni su un dato sistema e la meccanica quantistica è intrinsecamente non deterministica.

Probabilmente nessun esperimento potrà mai determinare quale visione sia corretta, ma questo dubbio non peggiora la nostra comprensione del mondo fisico. Possiamo semplicemente scegliere una visione e costruire il nostro modello coerentemente. Scegliamo quella più semplice: senza variabili nascoste. Tutto ciò che dobbiamo modellare sono le grandezze che possiamo misurare e le misurazioni ci permettono di conoscere tutte le informazioni su un dato sistema.

Nessuna variabile nascosta

Anche se abbiamo perso il determinismo completo, conoscere lo stato di un sistema ci fornisce alcune informazioni sul sistema e sulle misure successive. Nella prossima sezione vedremo cosa possiamo dedurre sullo spin.

1.2.4 Stati di spin

Iniziamo enumerando tutti i possibili stati di spin lungo gli assi coordinati. Se ruotiamo l'apparato $\mathcal{A}$ attorno a z , possiamo ottenere $\sigma_z = \pm 1$; chiamiamo questi stati *up* e *down* e li etichettiamo con i ket $|u\rangle$ e $|d\rangle$. Orientando $\mathcal{A}$ lungo x , otteniamo *left* $|l\rangle$ e *right* $|r\rangle$. Infine, lungo l'asse y , misuriamo gli stati *in* $|i\rangle$ e *out* $|o\rangle$.

L'ipotesi che non esistano variabili nascoste ci permette di rappresentare lo spazio degli stati in modo semplice: ogni stato di spin può essere rappresentato come un ket in uno spazio vettoriale complesso bidimensionale.

Lo spazio degli stati di spin ha due dimensioni

Per esprimere un vettore abbiamo bisogno di una base; scegliamo $\mathcal{B} = \{|u\rangle, |d\rangle\}$ ⁵ e cerchiamo di ottenere tutti gli stati come combinazione lineare (*sovrapposizione*) dei vettori della base. Un generico stato $|A\rangle$ può essere espresso come:

$$|A\rangle = \alpha_u |u\rangle + \alpha_d |d\rangle$$

⁴ Ricordiamo che una misura lungo un asse distrugge la nostra conoscenza del risultato lungo un altro asse.

⁵ Mostreremo che questi vettori sono effettivamente ortogonali e perché devono esserlo.

dove α_u e α_d sono le componenti di $|A\rangle$ lungo $|u\rangle$ e $|d\rangle$, e possono essere ottenute per proiezione: $\alpha_u = \langle u | A \rangle$ e $\alpha_d = \langle d | A \rangle$ (come nell'Equazione 1.2).

Le componenti di $|A\rangle$ sono numeri complessi e il loro significato fisico è: avendo uno spin preparato nello stato $|A\rangle = \alpha_u |u\rangle + \alpha_d |d\rangle$ ⁶; $\alpha_u^* \alpha_u$ è la probabilità di misurare $\sigma_z = +1$, mentre $\alpha_d^* \alpha_d$ è la probabilità che una misura di σ_z dia -1 . Formalmente possiamo denotare la probabilità di misurare $+1$ e -1 rispettivamente come P_u e P_d e scrivere:

$$\begin{aligned} P_u &= \langle A | u \rangle \langle u | A \rangle \\ P_d &= \langle A | d \rangle \langle d | A \rangle. \end{aligned} \quad (1.3)$$

Le componenti α_u e α_d sono chiamate ampiezze di probabilità, e il loro significato fisico è dato dal quadrato del modulo. Questa è la probabilità effettiva, e vogliamo che la somma di tutte le probabilità sia uno. Ciò equivale a richiedere che $|A\rangle$ sia normalizzato: $\langle A | A \rangle = 1$.

Mostriamo ora perché $|u\rangle$ e $|d\rangle$ devono essere ortogonali:

$$\begin{aligned} \langle u | d \rangle &= 0 \\ \langle d | u \rangle &= 0. \end{aligned}$$

Cerchiamo di dare un'idea con una *reductio ad absurdum*: se $|u\rangle$ e $|d\rangle$ non fossero ortogonali, la proiezione dell'uno sull'altro non sarebbe nulla. Ciò significa che se orientiamo A lungo z e misuriamo $\sigma_z = +1 = |u\rangle$, avremmo $\alpha_d = \langle d | u \rangle \neq 0$, il che è in contraddizione con i risultati sperimentali. Se $\alpha_d \neq 0$, allora $\alpha_d^* \alpha_d > 0$; abbiamo iniziato con uno stato preparato come $\sigma_z = +1$ e siamo finiti con una probabilità non nulla di misurare $\sigma_z = -1$: questo è assurdo.

Gli stati ortogonali sono mutualmente esclusivi. Possiamo estendere il ragionamento a un concetto generale e fondamentale della meccanica quantistica: due stati ortogonali sono distinti e mutualmente esclusivi. Se il sistema si trova nel primo stato, la probabilità di trovarlo nel secondo è zero.

Siamo ora pronti ad esprimere gli stati di spin come combinazioni lineari dei vettori della base $\mathcal{B} = \{|u\rangle, |d\rangle\}$. La rappresentazione degli stessi vettori della base è naturalmente semplice:

$$|u\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad (1.4)$$

$$|d\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \quad (1.5)$$

Per costruire il vettore *right*, consideriamo uno spin preparato nello stato $|r\rangle$. Se misuriamo σ_z , abbiamo il 50% di probabilità di ottenere $+1$ (e il 50% per -1); ciò significa che per $|r\rangle$ abbiamo $\alpha_u^* \alpha_u = \alpha_d^* \alpha_d = 1/2$. Un vettore che soddisfa questo vincolo è:

$$|r\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}. \quad (1.6)$$

⁶ D'ora in poi usiamo "preparato" o "misurato" come sinonimi: ogni misura è invasiva e può cambiare lo stato di spin, quindi indipendentemente da quale fosse lo stato precedente, dopo una misura lo stato è quello che abbiamo misurato.

Il ragionamento è lo stesso per lo stato *left*; aggiungiamo anche il vincolo che uno stato *left* non può essere *right* e viceversa: $\langle r | l \rangle = \langle l | r \rangle = 0$. Possiamo esprimere *left* come:

$$|l\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{i}{\sqrt{2}} \end{pmatrix}. \quad (1.7)$$

Infine, i vincoli per trovare le forme esplicite di *in* e *out* sono:

- gli stati devono essere ortogonali: $\langle i | o \rangle = \langle o | i \rangle = 0$;
- se abbiamo uno spin preparato come *in* o *out*:
 - equiprobabilità di misurare $\sigma_z = +1$ e $\sigma_z = -1$;
 - equiprobabilità di misurare $\sigma_x = +1$ e $\sigma_x = -1$.

Due vettori che soddisfano questi vincoli sono:

$$|i\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{i}{\sqrt{2}} \end{pmatrix} \quad (1.8)$$

$$|o\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{i}{\sqrt{2}} \end{pmatrix}. \quad (1.9)$$

Quest'ultima derivazione mostra perché è importante che lo spazio degli stati sia complesso: se accettassimo solo componenti reali per i nostri vettori, il sistema di equazioni che abbiamo implicitamente definito non avrebbe alcuna soluzione⁷.

1.3 OSSERVABILI

Nella nostra esperienza quotidiana abbiamo imparato che in meccanica classica possiamo fidarci della nostra intuizione, e possiamo effettuare una o più misure per conoscere esattamente lo stato di un sistema: una misura non perturba lo stato, che è lo stesso prima, durante e dopo la misura.

In meccanica quantistica la situazione è più complessa; la nostra intuizione è fuorviante, e abbiamo bisogno di strumenti matematici per descrivere ciò che possiamo misurare: le *osservabili*. Questi strumenti sono operatori matematici chiamati *macchine* (**M**) e hanno come input e output vettori di stato.

Axiom 2. *Le macchine associate alle osservabili sono descritte da operatori lineari.*

Mostreremo che le macchine sono operatori hermitiani, quindi iniziamo definendo questi operatori e descrivendo le loro proprietà⁸.

⁷ Per evitare confusione, precisiamo che $|i\rangle$ è il ket dello stato *dentro*. i , invece, è l'unità immaginaria.

⁸ La ragione per cui abbiamo bisogno di questo tipo di operatore sarà chiara nella Sezione 1.3.2.

1.3.1 Operatore hermitiano

Formalmente, le macchine modificano un vettore di stato in questo modo:

$$\mathbf{M}|A\rangle = |B\rangle$$

La linearità delle macchine implica che:

$$\mathbf{M}|A\rangle = |B\rangle \Rightarrow \mathbf{M}z|A\rangle = z|B\rangle$$

e:

$$\mathbf{M}(|A\rangle + |B\rangle) = \mathbf{M}|A\rangle + \mathbf{M}|B\rangle.$$

Se scegliamo una base per rappresentare le macchine e i vettori di stato, possiamo scrivere esplicitamente l'operatore lineare come una matrice $N \times N$, dove N è la dimensione dello spazio vettoriale dei vettori di stato. Una macchina generica che trasforma gli spin può essere espressa come:

$$\mathbf{M} = \begin{pmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{pmatrix}.$$

Quando fissiamo una base, siamo costretti ad esprimere tutti i vettori di stato e gli operatori in quella base, ma ora abbiamo un insieme di regole per definire l'applicazione dell'operatore a un vettore di stato, ovvero la moltiplicazione matriciale:

$$\mathbf{M}|A\rangle = \begin{pmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{pmatrix} \times \begin{pmatrix} \alpha_1 \\ \alpha_2 \end{pmatrix} = \begin{pmatrix} \beta_1 \\ \beta_2 \end{pmatrix} = |B\rangle$$

Quando consideriamo un operatore lineare, possiamo cercare autovalori e autovettori (se esistono). Gli autovettori sono vettori che non cambiano direzione quando vengono moltiplicati per l'operatore; la loro ampiezza viene scalata da un fattore costante chiamato autovalore. Formalmente:

$$\mathbf{M}|\lambda\rangle = \lambda|\lambda\rangle$$

dove $|\lambda\rangle$ è l'autovettore e λ l'autovalore corrispondente.

Considerando la trasformazione dal ket $|A\rangle$ a $|B\rangle$: $\mathbf{M}|A\rangle = |B\rangle$, tenendo conto dello spazio duale dei bra e cercando una macchina che trasformi il bra $\langle A|$ in $\langle B|$, non possiamo semplicemente usare la matrice avente come elementi il complesso coniugato di $\mathbf{M}$; l'operatore corretto è il *coniugato hermitiano* di $\mathbf{M}$, che è la trasposta della matrice avente come elementi i complessi coniugati di $\mathbf{M}$. Denotiamo il coniugato hermitiano con la daga $\dagger$:

$$\mathbf{M}^\dagger = [\mathbf{M}^*]^\mathrm{T} = [\mathbf{M}^\mathrm{T}]^*.$$

Possiamo ora scrivere:

$$\mathbf{M}|A\rangle = |B\rangle \Rightarrow \langle A|\mathbf{M}^\dagger = \langle B|.$$

Un operatore uguale al proprio coniugato hermitiano è chiamato *operatore hermitiano*. Formalmente, $\mathbf{M}$ è hermitiano se e solo se

$$\mathbf{M} = \mathbf{M}^\dagger.$$

339 Gli operatori hermitiani hanno alcune proprietà importanti:

- 340 • tutti gli autovalori sono reali;
- 341 • gli autovettori formano un *insieme completo*: tutti i vettori ottenuti con
- 342 l'applicazione dell'operatore possono essere espressi come combinazio-
- 343 ne lineare degli autovettori;
- 344 • se λ_1 e λ_2 sono autovalori distinti, gli autovettori associati sono ortogo-
- 345 nali;
- 346 • se due autovalori sono uguali (*degenerazione*), è sempre possibile trova-
- 347 re due autovettori associati che siano ortogonali.

348 Le ultime tre proprietà possono essere riassunte nel modo seguente:

Teorema fondamentale

349 **Theorem 1.** *Gli autovettori di un operatore hermitiano formano una base ortonor-*
 350 *male.*

351 1.3.2 Principi della meccanica quantistica

352 Introduciamo i primi quattro principi della meccanica quantistica, quelli
 353 riguardanti le osservabili⁹.

354 **Principles 1.** *Le osservabili in meccanica quantistica sono descritte da operatori*
 355 *lineari L.*

356 L deve essere anche un operatore hermitiano: possiamo considerare que-
 357 sta proposizione un assioma a sé stante oppure dedurla dagli altri principi.

358 **Principles 2.** *I risultati di una misura possono essere solo gli autovalori associati*
 359 *all'operatore osservabile.*

360 Chiamando λ_i un autovalore generico e $|\lambda_i\rangle$ l'autovettore associato, se il si-
 361 stema si trova nell'*autostato* $|\lambda_i\rangle$, la misura restituisce sempre λ_i . Poiché tutti i
 362 λ_i devono essere grandezze fisiche devono essere reali, proprietà soddisfatta
 363 degli operatori hermitiani.

364 **Principles 3.** *Stati inequivocabilmente distinguibili sono rappresentati da vettori*
 365 *ortogonali.*

366 Gli stati distinguibili possono essere separati senza ambiguità da una mi-
 367 sura. Ad esempio, se vogliamo distinguere tra $|u\rangle$ e $|d\rangle$, misuriamo σ_z : *up*
 368 e *down* sono distinti. Non possiamo invece dire se un certo sistema si trova
 369 nello stato *up* o *right*, perché anche se il sistema è nello stato $|u\rangle$ possiamo
 370 comunque misurare σ_x e trovare (con il 50% di probabilità) che il sistema si
 371 trova nello stato $|r\rangle$.

372 Il prodotto interno è una misura di quanto due stati siano indistingui-
 373 bili; per questo motivo è anche chiamato sovrapposizione. Due stati sono
 374 fisicamente distinti se la sovrapposizione è zero.

Sovrapposizione

$$\langle u | d \rangle = 0$$

$$\langle u | r \rangle \neq 0$$

⁹ Il quinto, e ultimo principio, riguarda l'evoluzione temporale. Verrà discusso più avanti (Sezione 1.4).

Principles 4. Se il sistema si trova nello stato $|A\rangle$ e misuriamo l'osservabile $\mathbf{L}$, la probabilità di ottenere λ_i è:

$$P(\lambda_i) = \langle A | \lambda_i \rangle \langle \lambda_i | A \rangle.$$

dove λ_i è un autovalore generico di $\mathbf{L}$ e $\langle \lambda_i |$, $|\lambda_i\rangle$ sono il bra e il ket associati a quell'autovalore (autovettore di λ_i).

1.3.3 Operatore di Spin

I principi ci dicono quali proprietà deve avere una macchina per rappresentare un'osservabile. Costruiamo l'operatore di spin σ .

Finora abbiamo misurato gli spin con l'apparato $\mathcal{A}$, orientando $\mathcal{A}$ lungo la componente di nostro interesse. σ è uno strumento matematico che ci permette di fare previsioni sul risultato di una misura con $\mathcal{A}$ (quarto principio). Poiché possiamo ruotare $\mathcal{A}$, dobbiamo anche ruotare (matematicamente) σ . Per questa proprietà spaziale, σ è chiamato *operatore trivettoriale*.

OPERATORE σ_z : Iniziamo con l'operatore più semplice¹⁰. Il secondo principio dice che tutti gli autovettori di σ_z sono $|u\rangle$ e $|d\rangle$, con autovalori associati $+1$ e -1 . Possiamo scrivere questa affermazione come equazioni:

$$\begin{aligned}\sigma_z |u\rangle &= |u\rangle \\ \sigma_z |d\rangle &= -|d\rangle.\end{aligned}$$

In forma matriciale:

$$\begin{aligned}\begin{pmatrix} (\sigma_z)_{11} & (\sigma_z)_{12} \\ (\sigma_z)_{21} & (\sigma_z)_{22} \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ \begin{pmatrix} (\sigma_z)_{11} & (\sigma_z)_{12} \\ (\sigma_z)_{21} & (\sigma_z)_{22} \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} &= - \begin{pmatrix} 0 \\ 1 \end{pmatrix}.\end{aligned}$$

La soluzione di questo sistema è¹¹:

$$\sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

OPERATORE σ_x : Con lo stesso ragionamento, possiamo costruire l'operatore lungo l'asse x . Abbiamo già dedotto le rappresentazioni di *right* e *left* nelle Equazioni 1.6 e 1.7. Le equazioni che ci permettono di costruire σ_x sono:

$$\begin{aligned}\begin{pmatrix} (\sigma_x)_{11} & (\sigma_x)_{12} \\ (\sigma_x)_{21} & (\sigma_x)_{22} \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} &= \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} \\ \begin{pmatrix} (\sigma_x)_{11} & (\sigma_x)_{12} \\ (\sigma_x)_{21} & (\sigma_x)_{22} \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix} &= - \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix}.\end{aligned}$$

La soluzione di questo sistema è:

$$\sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

¹⁰ Questo perché abbiamo scelto $\mathcal{B} = \{|u\rangle, |d\rangle\}$ come base.

¹¹ È facile verificare che anche questo operatore è lineare.

397 **OPERATORE σ_y :** L'ultima direzione è lungo l'asse y . Considerando le
 398 espressioni per *in* e *out* fornite nelle Equazioni 1.8 e 1.9, e seguendo il
 399 secondo principio, possiamo scrivere:

$$\begin{aligned}\sigma_y |i\rangle &= |i\rangle \\ \sigma_y |o\rangle &= -|o\rangle.\end{aligned}$$

400 Possiamo riscrivere questo sistema di equazioni in forma matriciale, e la
 401 soluzione che otterremmo è:

$$\sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}.$$

402 Abbiamo ottenuto una rappresentazione matriciale dei tre operatori di *Matrici di Pauli*
 403 spin σ_z , σ_x e σ_y :

$$\sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad \sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad \sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}. \quad (1.10)$$

404 Queste famose e importanti matrici (unite alla matrice identità) prendono il
 405 nome dal loro inventore, Wolfgang Ernst Pauli.

406 1.3.4 Teoria ed esperimenti

407 Grazie agli operatori σ_z , σ_x e σ_y , se conosciamo il vettore di stato, possiamo
 408 prevedere statisticamente il risultato di una misura dello spin lungo uno dei
 409 tre assi coordinati. Cosa possiamo dire di una misura effettuata orientando
 410 l'apparato $\mathcal{A}$ lungo una direzione generica?

411 Considerando $\mathcal{A}$ orientato lungo il versore $\hat{n}$, se σ si comporta come un
 412 trivettore, per ottenere σ_n è sufficiente il prodotto interno:

$$\sigma_n = \vec{\sigma} \cdot \hat{n}$$

413 Espandendo le componenti:

$$\sigma_n = \sigma_x n_x + \sigma_y n_y + \sigma_z n_z.$$

414 Se scegliamo la base $\mathcal{B} = \{|u\rangle, |d\rangle\}$, possiamo usare le matrici di Pauli per
 415 esprimere in forma matriciale l'espressione per σ_n :

$$\sigma_n = n_x \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} + n_y \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} + n_z \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} n_z & n_x - in_y \\ n_x + in_y & -n_z \end{pmatrix}.$$

416 Data una direzione (espressa dal versore $\hat{n}$), possiamo costruire la matrice
 417 che abbiamo ora reso esplicita, e poi, dopo aver trovato autovalori e auto-
 418 vettori, possiamo conoscere tutti i possibili risultati di una misura e ottenere
 419 la probabilità associata a ciascun risultato. Ad esempio, considerando una
 420 direzione nel piano x - z , l'operatore σ_n diventa:

$$\sigma_n = \begin{pmatrix} \cos \theta & \sin \theta \\ \sin \theta & -\cos \theta \end{pmatrix}$$

dove θ è l'angolo tra $\hat{n}$ e z . Per questa matrice, gli autovalori e gli autovettori sono:

$$\lambda_1 = 1 \quad |\lambda_1\rangle = \begin{pmatrix} \cos \frac{\theta}{2} \\ \sin \frac{\theta}{2} \end{pmatrix}$$

e

$$\lambda_2 = -1 \quad |\lambda_2\rangle = \begin{pmatrix} -\sin \frac{\theta}{2} \\ \cos \frac{\theta}{2} \end{pmatrix}.$$

È opportuno sottolineare che la teoria è in accordo con i risultati sperimentali¹². Gli autovalori sono $+1$ e -1 , esattamente i soli risultati che l'apparato $\mathcal{A}$ può restituire. La probabilità di ottenere un determinato risultato può essere valutata come:

$$P(+1) = |\langle u | \lambda_1 \rangle|^2 = \cos^2 \frac{\theta}{2}$$

$$P(-1) = |\langle u | \lambda_2 \rangle|^2 = \sin^2 \frac{\theta}{2}$$

Infine, calcoliamo il valore medio per la misura σ_n . Dal primo esperimento che abbiamo visto nella Sezione 1.1.1, sappiamo già che il risultato di misure ripetute con $\mathcal{A}$ è $\cos \theta$. Verifichiamo se il nostro modello è coerente con il mondo fisico.

I valori di aspettazione si ottengono come:

$$\langle L \rangle = \sum_i \lambda_i P(\lambda_i)$$

Nello specifico:

$$\langle \sigma_n \rangle = (+1) \cos^2 \frac{\theta}{2} + (-1) \sin^2 \frac{\theta}{2} = \cos \theta.$$

Questo è in completo accordo con i risultati sperimentali.

Prima di proseguire, presentiamo, senza dimostrazione, un teorema utile sui valori di aspettazione:

Theorem 2. *Per conoscere il valore di aspettazione di un'osservabile, possiamo semplicemente collocare l'operatore associato all'osservabile tra il bra e il ket del vettore di stato:*

$$\langle L \rangle = \langle A | L | A \rangle \quad (1.11)$$

dove L è un'osservabile, $|A\rangle$ è un vettore di stato e $\langle A|$ è il bra corrispondente.

1.3.5 Operatori e Misure

Gli operatori ci permettono di conoscere la probabilità di misurare un certo spin data la direzione della misura e il vettore di stato. Questa probabilità è espressa dal vettore di stato che otteniamo quando applichiamo l'operatore σ allo stato iniziale.

È importante non confondere l'atto di misura con l'applicazione di una macchina che rappresenta le osservabili. Lo stato di spin dopo la misura non è lo stesso che otteniamo dopo l'applicazione dell'operatore. L'operatore

¹² In caso contrario, dovremmo abbandonare questo modello e costruirne un altro.

449 è solo un costrutto matematico astratto che ci permette di fare previsioni
450 statistiche sui risultati, ma non ha implicazioni fisiche.

451 Consideriamo un esempio per chiarire l'affermazione precedente. Avendo
452 uno spin preparato nello stato up , il suo vettore di stato è $|u\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$. Se
453 applichiamo l'operatore σ_z , otterremo di nuovo $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$, e se misuriamo lo
454 spin con $\mathcal{A}$ orientato lungo z , visualizzerà sempre $+1$, e concludiamo che lo
455 stato dopo la misura è $|u\rangle$.

456 Consideriamo ora uno spin preparato *right*, ovvero $|r\rangle = 1/\sqrt{2}|u\rangle + 1/\sqrt{2}|d\rangle$.
457 Applicando nuovamente l'operatore σ_z , il nuovo vettore di stato è $1/\sqrt{2}|u\rangle -$
458 $1/\sqrt{2}|d\rangle$. Questo vettore ci dice la probabilità di misurare $\sigma_z = +1$ (50%),
459 ma non è lo stato di spin dopo la misura. Usando l'apparato $\mathcal{A}$, potremmo
460 misurare:

- 461 • $+1$: lo stato finale sarà *up*;
- 462 • -1 : lo stato finale sarà *down*.

463 Indipendentemente dal risultato della misura, lo stato finale sarà diverso da
464 quello ottenuto applicando l'operatore.

465 1.4 EVOLUZIONE TEMPORALE

466 Esploriamo le leggi che descrivono l'evoluzione temporale di un sistema
467 quantistico. In particolare, vedremo come il vettore di stato può evolvere nel
468 tempo.

469 1.4.1 Unitarietà

470 In meccanica classica siamo abituati ad avere una legge del moto che colle-
471 ga deterministicamente i diversi stati del nostro sistema; ciò significa poter
472 conoscere con precisione lo stato successivo dato quello precedente. Una
473 buona legge, tuttavia, non ci permette solo di conoscere il futuro, ma anche
474 gli stati passati che hanno portato il sistema allo stato attuale¹³.

475 In altre parole, vogliamo che le trasformazioni fisiche siano reversibili. Reversibilità
476 Questo requisito è così importante che chiamiamo questa proprietà la *leg-*
477 *ge meno uno*, perché è alla base di tutto il resto. Se pensiamo agli stati del
478 sistema come nodi in un grafo orientato, la reversibilità impone che ogni
479 nodo abbia esattamente un arco entrante e uno uscente. Questa legge fonda-
480 mentale vale anche in meccanica quantistica ed è chiamata *unitarietà*¹⁴, e ci
481 assicura che nessuna informazione venga persa. La legge di unitarietà può
482 essere espressa come:

483 **Axiom 3.** *Se due sistemi isolati identici si trovano in stati diversi, rimangono in*
484 *stati diversi, ed erano in stati diversi nel passato.*

¹³ Ad esempio, se osserviamo una palla in caduta libera che tocca il suolo con una certa velocità e in un certo momento, possiamo sapere esattamente quando e da quale altezza la palla ha iniziato la sua caduta.

¹⁴ Vedremo nel prossimo paragrafo la ragione di questo nome

1.4.2 Operatore di Evoluzione Temporale

Considerando un sistema nello stato $|\Psi(t)\rangle$, dove la t indica che il vettore di stato evolve nel tempo, le equazioni del moto quantistiche ci permettono di ottenere lo stato al tempo t dato lo stato iniziale:

$$|\Psi(t)\rangle = U(t) |\Psi(0)\rangle. \quad (1.12)$$

Determinismo Grazie all'operatore $U(t)$ possiamo conoscere esattamente il vettore di stato $|\Psi(t)\rangle$ al tempo t , dato $|\Psi(0)\rangle$. Questa affermazione può essere riformulata come:

Axiom 4. *L'evoluzione temporale del vettore di stato è deterministica.*

La meccanica quantistica è ancora non deterministica, perché conoscere il vettore di stato non significa conoscere il risultato di una misura.

Affinché $U(t)$ si comporti come vogliamo, deve:

- essere un operatore lineare;
- rispettare la reversibilità.

Il secondo vincolo ci permette di definire le proprietà matematiche di $U(t)$. Considerando due stati inizialmente diversi $|\Psi(0)\rangle$ e $|\Phi(0)\rangle$, poiché esiste un esperimento capace di distinguere con certezza gli stati, $|\Psi(0)\rangle$ e $|\Phi(0)\rangle$ devono essere ortogonali:

$$\langle \Psi(0) | \Phi(0) \rangle = 0.$$

Conservazione delle distinzioni La legge meno uno assicura che durante l'intera evoluzione temporale dei due sistemi, i vettori di stato $|\Psi(t)\rangle$ e $|\Phi(t)\rangle$ continueranno ad essere distinguibili (ortogonali):

$$\langle \Psi(t) | \Phi(t) \rangle = 0 \quad \forall t \geq 0.$$

Se riscriviamo questa equazione usando la Formula 1.12, otteniamo:

$$\langle \Psi(0) | U^\dagger(t) U(t) | \Phi(0) \rangle = 0.$$

Da questo possiamo vedere che $U^\dagger(t)U(t)$ deve comportarsi come l'operatore identità, ovvero:

$$U^\dagger(t)U(t) = I. \quad (1.13)$$

Un operatore che si comporta come U è detto *unitario*.

Principles 5. *L'evoluzione temporale dei vettori di stato è unitaria.*

Dall'unitarietà di U discende la *conservazione delle sovrapposizioni*: la sovrapposizione tra due stati (il loro prodotto interno), soggetti allo stesso operatore di evoluzione temporale, è preservata nel tempo.

1.4.3 L'Hamiltoniano

Spesso, in fisica classica, una legge del moto è il risultato di un'equazione differenziale in cui abbiamo sostituito un intervallo di tempo finito con un numero infinito di intervalli infinitesimi.

In meccanica quantistica possiamo seguire lo stesso percorso e considerare intervalli di tempo ϵ prossimi a zero. In questo scenario, dopo un intervallo di tempo ϵ , il vettore di stato cambierà leggermente e in modo "continuo", e l'operatore $\mathbf{U}(\epsilon)$ sarà molto simile all'identità. Possiamo riscrivere $\mathbf{U}(\epsilon)$ in modo da evidenziare la differenza con l'identità $\mathbf{I}$ come:

$$\mathbf{U}(\epsilon) = \mathbf{I} - i\epsilon\mathbf{H}. \quad (1.14)$$

Per ora, i è un semplice fattore di scala che in seguito ci aiuterà a riconoscere in $\mathbf{H}$ la versione quantistica dell'Hamiltoniano classico.

Possiamo ora esprimere l'evoluzione infinitesima di un sistema quantistico combinando le Equazioni 1.12 e 1.14:

$$|\Psi(\epsilon)\rangle = |\Psi(0)\rangle - i\epsilon\mathbf{H}|\Psi(0)\rangle.$$

Portando a sinistra l'intervallo di tempo:

$$\frac{|\Psi(\epsilon)\rangle - |\Psi(0)\rangle}{\epsilon} = -i\mathbf{H}|\Psi(0)\rangle.$$

Considerando ora il limite per $\epsilon \rightarrow 0$, possiamo riconoscere nel membro sinistro la derivata temporale del vettore di stato:

$$\frac{\partial |\Psi(t)\rangle}{\partial t} = -i\mathbf{H}|\Psi(0)\rangle.$$

Prima di usare $\mathbf{H}$ come Hamiltoniano quantistico, dobbiamo verificare la correttezza dimensionale. Come in meccanica classica, l'Hamiltoniano è il costrutto matematico che rappresenta l'energia. Nella nostra formula, tuttavia, trascurando il vettore di stato, abbiamo l'inverso del tempo a sinistra e l'energia a destra. Per risolvere questo problema, introduciamo un'importante costante fisica: la costante di Planck ridotta, $\hbar$. L'equazione diventa:

$$\hbar \frac{\partial |\Psi\rangle}{\partial t} = -i\mathbf{H}|\Psi\rangle \quad \text{oppure} \quad \frac{\partial |\Psi\rangle}{\partial t} = \frac{-i\mathbf{H}|\Psi\rangle}{\hbar}. \quad (1.15)$$

Equazione di Schrödinger dipendente dal tempo

La costante $\hbar$ ha unità di misura $\text{kg} \cdot \text{m}^2/\text{s}$ e risolve l'incompatibilità tra i due membri. Questa equazione è fondamentale ed è chiamata *equazione di Schrödinger generalizzata*, o equazione di Schrödinger dipendente dal tempo. Se conosciamo l'Hamiltoniano di un sistema indisturbato, possiamo conoscere l'evoluzione del vettore di stato.

Se $\mathbf{H}$ rappresenta l'energia del sistema, dovremmo essere in grado di misurarla, quindi $\mathbf{H}$ deve essere un'osservabile. Se $\mathbf{H}$ è un'osservabile, deve essere un operatore hermitiano; verifichiamolo. Partendo dall'Equazione 1.13 e sostituendo $\mathbf{U}$ con l'Espressione 1.14, otteniamo:

$$(\mathbf{I} + i\epsilon\mathbf{H}^\dagger)(\mathbf{I} - i\epsilon\mathbf{H}) = \mathbf{I}.$$

Espandendo al primo ordine in ϵ , troviamo:

$$\mathbf{H}^\dagger - \mathbf{H} = 0 \Rightarrow \mathbf{H}^\dagger = \mathbf{H}.$$

Hamiltoniano quantistico

Abbiamo concluso che $\mathbf{H}$ è un operatore hermitiano che rappresenta un'osservabile: l'energia del sistema. Gli autovalori di $\mathbf{H}$ sono i risultati di tutte le possibili misure dirette dell'energia del sistema.

1.4.4 Commutatori

In un sistema che evolve nel tempo, ci aspettiamo che anche i valori di aspettazione per una certa osservabile $\mathbf{L}$ cambino. Grazie all'equazione 1.11, possiamo scrivere esplicitamente la dipendenza temporale dei valori di aspettazione:

$$\langle \mathbf{L} \rangle = \langle \Psi(t) | \mathbf{L} | \Psi(t) \rangle.$$

La derivata temporale¹⁵ è:

$$\frac{d}{dt} \langle \Psi(t) | \mathbf{L} | \Psi(t) \rangle = \langle \dot{\Psi}(t) | \mathbf{L} | \Psi(t) \rangle + \langle \Psi(t) | \mathbf{L} | \dot{\Psi}(t) \rangle.$$

Sostituendo bra e ket con l'equazione di Schrödinger dipendente dal tempo 1.15 (ovvero $|\dot{\Psi}(t)\rangle = \frac{-i}{\hbar} \mathbf{H} |\Psi(t)\rangle$), otteniamo:

$$\frac{d}{dt} \langle \Psi(t) | \mathbf{L} | \Psi(t) \rangle = \frac{i}{\hbar} \langle \Psi(t) | \mathbf{H} \mathbf{L} | \Psi(t) \rangle - \frac{i}{\hbar} \langle \Psi(t) | \mathbf{L} \mathbf{H} | \Psi(t) \rangle.$$

Che può essere riscritta come:

$$\frac{d}{dt} \langle \Psi(t) | \mathbf{L} | \Psi(t) \rangle = \frac{i}{\hbar} \langle \Psi(t) | [\mathbf{H}, \mathbf{L}] | \Psi(t) \rangle.$$

La quantità $\mathbf{H} \mathbf{L} - \mathbf{L} \mathbf{H}$ è chiamata *commutatore*, e poiché, in generale, il prodotto tra operatori (matrici) non è commutativo, il commutatore non è zero (quando è zero, diciamo che $\mathbf{H}$ e $\mathbf{L}$ commutano). I commutatori sono importanti in fisica, e il commutatore tra due operatori, in questo caso $\mathbf{H}$ e $\mathbf{L}$, è denotato da:

$$\mathbf{H} \mathbf{L} - \mathbf{L} \mathbf{H} = [\mathbf{H}, \mathbf{L}].$$

Con il commutatore possiamo esprimere in modo conciso la derivata del valore di aspettazione per l'osservabile $\mathbf{L}$:

$$\frac{d}{dt} \langle \mathbf{L} \rangle = \frac{i}{\hbar} \langle [\mathbf{H}, \mathbf{L}] \rangle \quad (1.16)$$

o equivalentemente:

$$\frac{d}{dt} \langle \mathbf{L} \rangle = -\frac{i}{\hbar} \langle [\mathbf{L}, \mathbf{H}] \rangle. \quad (1.17)$$

Questa equazione collega le variazioni dei valori di aspettazione di un'osservabile ($\mathbf{L}$) ai valori di aspettazione di un'altra osservabile fisica $(-\frac{i}{\hbar} [\mathbf{L}, \mathbf{H}])$ ¹⁶.

¹⁵ Derivata di un prodotto: $\mathbf{L}$ non dipende dal tempo e il punto denota la derivata temporale (notazione di Newton).

¹⁶ È possibile dimostrare che se $\mathbf{L}$ e $\mathbf{H}$ sono hermitiani, allora $[\mathbf{L}, \mathbf{H}]$ è anch'esso hermitiano.

1.4.5 Conservazione dell'Energia

In meccanica quantistica, quando diciamo che una grandezza è conservata, intendiamo che il valore di aspettazione di quella grandezza non cambia. Se guardiamo all'Equazione 1.17, la condizione affinché il valore di aspettazione non cambi è che il commutatore tra questa grandezza e l'Hamiltoniano sia zero. È possibile dimostrare che:

Theorem 3. *Data un'osservabile Q , se $[Q, H] = 0$, allora ogni potenza soddisfa $[Q^n, H] = 0$. Ciò significa che il valore di aspettazione $\langle Q \rangle$ è conservato, e qualsiasi potenza del valore di aspettazione $\langle Q^n \rangle$ non cambia nel tempo.*

La grandezza più ovvia che è conservata è l'Hamiltoniano H e, poiché ogni operatore commuta con sé stesso, abbiamo sempre:

$$[H, H] = 0.$$

Possiamo concludere che, in condizioni molto generali, l'energia si conserva in meccanica quantistica.

1.5 CONCLUSIONI

Concludiamo questo capitolo con un riepilogo di ciò che abbiamo scoperto in queste pagine, cercando di mettere tutto insieme per rispondere alla domanda che ha aperto questo capitolo: quali sono i limiti fisici del calcolo quantistico, e perché i nostri algoritmi devono essere reversibili?

Abbiamo iniziato il capitolo con un esperimento che mostra che la meccanica quantistica non è deterministica. Possiamo tuttavia fare alcune previsioni se consideriamo il valore di aspettazione di una misura invece del singolo risultato.

Abbiamo costruito i vettori di stato e compreso il loro significato matematico, concentrandoci sul fatto che conoscere il vettore di stato non ci permette di conoscere il risultato di una misura. Abbiamo definito il prodotto interno tra vettori di stato, osservato che è una misura della sovrapposizione tra stati, e concluso che due stati distinguibili devono essere ortogonali.

Abbiamo collegato un vettore di stato al risultato di una misura –per essere precisi, alla media dei risultati di misure multiple– tramite le macchine, operatori hermitiani che rappresentano le osservabili. Abbiamo costruito l'operatore di spin e lo abbiamo usato per prevedere il risultato di un semplice esperimento, mostrando come la teoria che abbiamo costruito finora sia in accordo con i risultati sperimentali.

La nostra introduzione prosegue con l'analisi dell'evoluzione temporale di un sistema quantistico. Abbiamo descritto l'evoluzione di un vettore di stato con un operatore unitario; l'applicazione di questo operatore a un vettore di stato produce il nuovo stato in cui si troverà il sistema. Abbiamo compreso che l'evoluzione temporale del vettore di stato è deterministica e che l'indeterminazione è causata solo dall'atto di misura.

Considerare intervalli di tempo infinitesimi ci ha permesso di dedurre l'equazione di Schrödinger dipendente dal tempo e, grazie a questa equazione, abbiamo mostrato come descrivere l'evoluzione temporale dei valori

di aspettazione per una certa osservabile. Durante questa analisi, abbiamo anche introdotto l'Hamiltoniano del sistema, un operatore hermitiano che descrive l'energia del sistema.

La discussione si conclude con un risultato confortante: come in fisica classica, l'energia di un sistema chiuso è conservata. Abbiamo ottenuto questo risultato presentando il commutatore e collegando l'evoluzione temporale di un'osservabile con il commutatore tra l'osservabile e l'Hamiltoniano (energia) del sistema. Il commutatore dell'Hamiltoniano con sé stesso è banalmente zero, quindi il valore di aspettazione dell'energia non cambia.

Tutte le informazioni che abbiamo appreso ci permettono di comprendere il vincolo di scrivere solo algoritmi reversibili per i computer quantistici basati su porte quantistiche. Le porte quantistiche operano sui qubit tramite trasformazioni fisiche¹⁷. Queste trasformazioni, come tutte le trasformazioni in meccanica quantistica, sono descritte da operatori unitari che sono intrinsecamente reversibili. Ciò significa che tutte le porte quantistiche sono reversibili.

In altre parole, possiamo costruire solo porte quantistiche che, avendo come input stati diversi (distinguibili), restituiscono stati ortogonali; inoltre, grazie alla conservazione delle sovrapposizioni, il prodotto interno tra gli stati di input è conservato durante la trasformazione della porta quantistica.

La reversibilità non significa che possiamo andare avanti e indietro nel tempo a nostro piacimento, ma che tutte le porte quantistiche esprimono funzioni iniettive: se conosciamo l'output, possiamo conoscere l'input, o in termini più fisici, se conosciamo lo stato finale dei qubit¹⁸ e le trasformazioni applicate a questo sistema (ovvero quelle implementate dalle porte quantistiche), possiamo determinare lo stato iniziale.

Poiché ogni algoritmo quantistico deve essere implementato come un percorso attraverso porte quantistiche, e ogni porta quantistica è reversibile, gli algoritmi nel loro insieme devono essere anch'essi reversibili.

¹⁷ Come dipende fortemente dalla particolare implementazione fisica.

¹⁸ Questo è un sistema complesso (composto da più di un qubit); per comprendere appieno questi sistemi, dovremmo tener conto dell'entanglement. Poiché la nostra discussione è già abbastanza lunga, e l'evoluzione temporale di un sistema entangled è ancora unitaria (reversibile), escludiamo l'entanglement dalla nostra introduzione.

COMPLESSITÀ COMPUTAZIONALE E CALCOLO QUANTISTICO

640 In questo capitolo introduciamo i concetti di base del calcolo quantistico.
641 Esaminiamo le ragioni che ci spingono a cercare nuovi paradigmi computa-
642 zionali e perché il calcolo quantistico appare promettente.

643 Per comprendere appieno le aspettative riposte nel calcolo quantistico, è
644 necessario innanzitutto introdurre alcuni elementi di calcolabilità e comples-
645 sità. Esamineremo quindi i fondamenti dell'informatica a partire dai model-
646 li computazionali. Daremo una definizione di problema computazionale e
647 vedremo quali problemi possiamo risolvere con un computer. Infine, studie-
648 remo la complessità computazionale dei problemi classificandoli in base alle
649 risorse necessarie per trovare una soluzione.

650 Completata questa introduzione allo studio della complessità computazio-
651 nale, vedremo come i computer quantistici si inseriscono in questa analisi.
652 Inizieremo presentando un modello computazionale che ci permette di defi-
653 nire quali problemi sono risolvibili su un computer quantistico, e osservere-
654 mo come questo ci porti naturalmente verso i computer quantistici basati su
655 porte quantistiche. Vedremo quali classi di complessità possono essere defi-
656 nite considerando questo nuovo paradigma, e cercheremo di comprendere
657 quali relazioni esistono tra le classi di problemi identificate con il modello
658 classico di computazione e quelle derivate dal modello quantistico.

659 Nella seconda sezione di questo capitolo presenteremo in modo più ri-
660 goroso i computer basati su porte quantistiche. Vedremo cosa si intende
661 per porta quantistica, presenteremo le porte più comuni e costruiremo un
662 insieme universale di porte. Vedremo poi cosa significa programmare un
663 computer basato su porte quantistiche presentando l'algoritmo di Deutsch.

664 Il capitolo si conclude presentando l'*Adiabatic Quantum Computing* (AQC)
665 (Computazione Quantistica Adiabatica): un paradigma alternativo alla com-
666 putazione basata su porte quantistiche. Descriveremo i fondamenti teorici
667 di questo paradigma e mostreremo che l'AQC e il calcolo basato su por-
668 te quantistiche sono equivalenti. Vedremo che programmare una macchina
669 che sfrutta il paradigma AQC significa tradurre il problema che vogliamo ri-
670olvere in un problema di minimizzazione. Presentiamo un altro approccio
671 nativamente implementabile con l'AQC: il *Quantum Annealing* QA. Descri-
672 veremo il QA tracciando un parallelo tra il simulated annealing classico e il
673 quantum annealing. Infine, ci concentreremo sulla formulazione di problemi
674 di minimizzazione naturalmente risolvibili con il quantum annealing.

675 Questo capitolo è ispirato ai seguenti libri e articoli: la Sezione 2.1.1 è
676 ispirata a [3, capitoli 3 e 4], la Sezione 2.2 è tratta da [3, capitoli 2, 3, 4 e 7],
677 infine la Sezione 2.3 è ispirata a [4, capitoli 2 e 3] e [5]. Nelle pagine seguenti
678 questi riferimenti non vengono più considerati e, quando necessario, citiamo
679 altri lavori.

2.1 INTRODUZIONE ALL'ANALISI COMPUTAZIONALE

L'analisi della complessità computazionale si occupa di studiare quali problemi possono essere risolti tramite un computer e quali risorse sono necessarie per trovare la soluzione. Possiamo dividere questi due obiettivi in due linee di ricerca parallele e complementari. Da un lato, per dimostrare che un problema è risolvibile, è sufficiente mostrare un algoritmo capace di risolverlo, senza preoccuparsi troppo della sua complessità. Dall'altro, per capire quali risorse sono necessarie, vorremmo trovare un limite inferiore alle operazioni che devono essere eseguite per ottenere la soluzione.

Queste linee di ricerca sono complementari nel senso che, idealmente, una volta identificato un limite inferiore, vorremmo essere in grado di costruire un algoritmo che lo raggiunga. In pratica, questo non è sempre possibile, perché costruire algoritmi che si comportino bene su tutte le istanze di un problema è difficile, e perché il limite inferiore delle operazioni richieste non è sempre noto, quindi ci si deve accontentare del miglior algoritmo sviluppato finora.

Abbiamo fatto riferimento a un algoritmo sapendo intuitivamente che è una sequenza di operazioni che ci permette di trovare la risposta a un problema. Per studiare formalmente le caratteristiche di un algoritmo, questa intuizione non è sufficiente; abbiamo bisogno di qualcosa che possiamo caratterizzare in modo più preciso. Introduciamo quindi i modelli computazionali.

2.1.1 Modelli computazionali

Un modello computazionale ci permette di definire matematicamente il concetto di algoritmo. Queste astrazioni furono introdotte per la prima volta da Turing e Church per trovare una soluzione all'*entscheidungsproblem* di Hilbert¹. Consideriamo ora due modelli computazionali equivalenti: la *macchina di Turing* e il *modello a circuiti*.

Macchina di Turing

Questo è il modello computazionale più noto. Introdotto da Turing per descrivere formalmente la nozione di algoritmo che esegue un compito computazionale.

Definizione di macchina di Turing Una macchina di Turing può essere vista come la concettualizzazione di un computer. È composta da quattro elementi principali:

PROGRAMMA: che consente la codifica di un algoritmo;

CONTROLLO A STATI FINITI: unità che funge da processore;

NASTRO: memoria del computer;

TESTINA DI LETTURA-SCRITTURA: interagisce con la memoria.

È possibile definire varie versioni della macchina di Turing. Per la nostra discussione possiamo considerare la versione più semplice, in cui il nastro

¹ Per ogni problema matematico, esiste un algoritmo capace di risolverlo?

di memoria è illimitato a destra e limitato a sinistra. I simboli che possono essere scritti sul nastro sono 0, 1, b (il simbolo vuoto) e il simbolo speciale $\triangleright$ che segna l'estremità sinistra del nastro. Questi simboli formano il nostro alfabeto Γ : tutto ciò che può essere rappresentato sul nastro consiste in stringhe di lunghezza finita Γ^* composte da simboli dell'alfabeto (tranne $\triangleright$, che appare solo all'inizio del nastro). Il controllo a stati finiti consiste in un insieme finito di stati $q_1, \dots, q_m$ più due stati speciali q_s e q_h che rappresentano, rispettivamente, lo stato iniziale della macchina e lo stato finale (di arresto); quest'ultimo viene raggiunto quando la macchina ha terminato il calcolo.

Un programma per la macchina di Turing consiste in righe di programma; ogni riga è una quintupla $\langle q, x, q', x', s \rangle$ che viene interpretata come segue: se la macchina si trova nello stato q e legge il simbolo $x \in \Gamma$ sul nastro, allora transita nello stato q' , scrive il simbolo $x' \in \Gamma$ sul nastro e si muove come indicato da s (a destra $+1$, a sinistra -1 , oppure rimane ferma 0).

Il modello della macchina di Turing può sembrare semplice, ma è il formalismo di riferimento per stabilire cosa consideriamo calcolabile o meno. L'importanza della macchina di Turing è supportata dalla *tesi di Church–Turing* proposta indipendentemente da Church e Turing:

Thesis 1. *La classe delle funzioni calcolabili da una macchina di Turing corrisponde esattamente alla classe delle funzioni che naturalmente considereremmo calcolabili da un algoritmo.*

Tesi di Church–Turing

La tesi di Church–Turing afferma che la macchina di Turing è un modello computazionale capace di fornire una definizione rigorosa del concetto intuitivo di algoritmo, ed è quindi adatta come fondamento dell'informatica.

La macchina di Turing che abbiamo descritto può essere modificata, ad esempio immaginando di avere più nastri di memoria; questa nuova macchina potrebbe essere in grado di risolvere i problemi più rapidamente dell'originale, ma saremo sempre in grado di simulare la macchina multinastro con una macchina a nastro singolo. Ciò significa che considerare nastri multipli non aumenta la potenza della macchina di Turing.

Un'altra modifica, che sarà utile in seguito, che possiamo apportare a questo modello computazionale è considerare una versione probabilistica della macchina di Turing: se la macchina si trova nello stato q e il simbolo x viene letto, si lancia una moneta; se il risultato è testa, la macchina transita nello stato q_H , altrimenti nello stato q_T . Anche questa modifica può essere simulata con una macchina di Turing che esplora tutti i possibili percorsi di calcolo uno alla volta (usando un numero esponenzialmente maggiore di passi), quindi ancora una volta l'insieme delle funzioni calcolabili non viene esteso.

INDECIDIBILITÀ: I modelli computazionali, e con essi l'informatica, furono sviluppati per rispondere all'*entscheidungsproblem* di Hilbert. Church, con il λ -calcolo², e Turing, con la macchina di Turing, dimostrarono che la risposta è no: non può esistere un algoritmo per decidere tutti i problemi della matematica.

Questo risultato divide chiaramente i problemi matematici in due classi: quelli che possono essere risolti con un algoritmo e quelli così difficili da es-

² Un altro modello computazionale equivalente alla macchina di Turing.

sere computazionalmente irrisolvibili. Chiamiamo i problemi appartenenti a questa classe *indecidibili*. Turing dimostra che il *problema della fermata* (il problema di determinare se una macchina di Turing si ferma dato un input) è, in generale, indecidibile³. Oltre ai problemi usati nelle dimostrazioni di Church e Turing, esistono molti altri problemi indecidibili, ad esempio trovare un omeomorfismo tra due spazi topologici o alcuni problemi relativi al comportamento dei sistemi dinamici.

Modello a circuiti

Il modello a circuiti è un modello alternativo di computazione in cui descriviamo un algoritmo implementandolo tramite un circuito composto da porte logiche. Questo approccio è interessante perché ci avvicina al primo modello quantistico di computazione: il modello basato su porte.

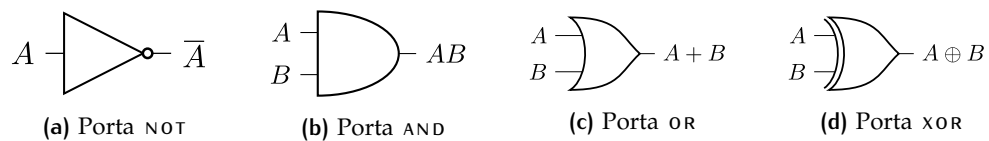


Figura 2.1: Porte logiche

Nella Figura 2.1 sono mostrate le porte logiche più note, con le quali possiamo costruire un circuito digitale. A queste aggiungiamo normalmente la possibilità di ottenere una copia di un bit (un filo che si divide), un'operazione che chiamiamo *FANOUT*, e la possibilità di scambiare due bit, chiamata *CROSSOVER*.

È possibile dimostrare che con questo insieme limitato di porte si può calcolare qualsiasi funzione $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$. Questo non è sufficiente per descrivere un modello computazionale utile. Prima di vedere quali caratteristiche richiediamo, definiamo il concetto di famiglia di circuiti.

Una *famiglia di circuiti* è un insieme di circuiti $\{C_n\}$ indicizzati da $n \in \mathbb{N}$. Il circuito C_n ha n bit di input e un numero finito di bit di lavoro extra e bit di output. Dato un circuito C_n e una stringa di bit x di lunghezza massima n come input del circuito, denotiamo con $C_n(x)$ l'output del circuito.

Diciamo che una famiglia di circuiti è *consistente* se tutti i circuiti nella famiglia calcolano la stessa funzione $C(\cdot)$. Formalmente, la definizione di consistenza è: la famiglia di circuiti $\{C_n\}$ è consistente se per $m < n$ e x di lunghezza al più m bit, allora $C_m(x) = C_n(x)$.

Infine, definiamo una famiglia di circuiti *uniforme* se esiste un algoritmo per una macchina di Turing che, dato n come input, è in grado di generare una descrizione di C_n . Cioè, questo algoritmo deve fornire l'elenco delle porte che costituiscono C_n , e la descrizione di come queste porte sono collegate tra loro.

Definizione del modello a circuiti Siamo finalmente pronti a descrivere le caratteristiche formali del modello computazionale a circuiti. In questo modello consideriamo le funzioni calcolabili da famiglie di circuiti $\{C_n\}$ tali che:

³ Church dimostra che non esiste una funzione calcolabile che permetta di determinare se due espressioni del λ -calcolo sono equivalenti.

- 803 • i circuiti nella famiglia sono *aciclici*: nel circuito non devono essere
804 presenti cicli, questo previene il verificarsi di possibili instabilità;
- 805 • $\{C_n\}$ è una famiglia di circuiti *consistente*: tutti i circuiti calcolano la
806 stessa funzione (su input di dimensione crescente);
- 807 • $\{C_n\}$ è una famiglia di circuiti *uniforme*: i circuiti possono essere gene-
808 rati a partire da un algoritmo ragionevole.

809 È possibile dimostrare che la classe di funzioni calcolabili da una famiglia
810 di circuiti consistente, uniforme e aciclica è esattamente la stessa calcolabile
811 su una macchina di Turing.

812 2.1.2 Problemi computazionali

813 I modelli teorici che abbiamo definito nella sezione precedente (e altri equi-
814 valenti che sono stati proposti nel corso dello studio dell'informatica) ci per-
815 mettono di studiare formalmente i problemi computazionali. Questa analisi
816 dipende dalla risposta a tre domande fondamentali:

817 COS'È UN PROBLEMA COMPUTAZIONALE: i problemi computazionali sono
818 una classe specifica di problemi chiamati *problemi decisionali*. Questa
819 scelta ci permette di isolare una classe ristretta per semplificare l'analisi
820 e fare progressi nello sviluppo della teoria. Vedremo che i risultati
821 ottenuti si estendono ben oltre i problemi decisionali;

822 COME SVILUPPARE UN ALGORITMO PER RISOLVERLO: dopo l'identificazio-
823 ne del problema, è necessario sviluppare un algoritmo capace di tro-
824 vare una soluzione. Siamo poi interessati a determinare se esistono
825 algoritmi generali capaci di risolvere ampie classi di problemi. Infine,
826 vorremmo trovare strumenti per garantire che un algoritmo funzioni
827 effettivamente come previsto;

828 QUALI RISORSE SONO NECESSARIE PER LA SOLUZIONE: un algoritmo con-
829 suma risorse computazionali come tempo, spazio ed energia; vogliamo
830 sapere se l'algoritmo sviluppato consuma più risorse di quelle effeti-
831 vamente necessarie. Inoltre, vorremmo classificare i problemi in base
832 alle risorse richieste per risolverli.

833 *Complessità computazionale*

834 Lo studio della complessità computazionale mira a quantificare le risorse
835 computazionali necessarie per risolvere un dato problema. Per studiare
836 il consumo di risorse indipendentemente dalle caratteristiche dell'architettura di riferimento⁴, viene introdotta la *notazione asintotica*, che cattura il
837 comportamento essenziale di una funzione:
838

Notazione asintotica

- 839 • O ("O grande"): usato per definire il *limite superiore* di una funzione;

⁴ Intendiamo l'indipendenza da specifiche scelte architetturali che, ad esempio, consentono a un processore di risparmiare qualche ciclo di clock ogni volta che viene eseguita una certa operazione. Vedremo presto che diversi modelli computazionali possono, invece, avere un consumo di risorse asintoticamente diverso.

- 840 • Ω ("Omega grande"): usato per definire il *limite inferiore* di una funzio-
841 ne;
- 842 • Θ ("Theta grande"): è la combinazione di O e Ω , descrive il comporta-
843 mento asintotico di una funzione.

844 La complessità computazionale considera le risorse spaziali e temporali
845 necessarie per risolvere un problema e cerca di dimostrare l'esistenza di un
846 limite inferiore per queste risorse indipendentemente dall'esistenza o meno
847 di un algoritmo ottimale che rispetti questo limite inferiore. Chiaramente, lo
848 sviluppo di algoritmi dovrebbe mirare a trovare un algoritmo che non uti-
849 lizzi più risorse del necessario, ovvero il limite inferiore teorico identificato
850 dallo studio della complessità computazionale.

851 Nello sviluppo della teoria della complessità computazionale, si è osser-
852 vato che il consumo di risorse dipende dal modello computazionale di ri-
853 ferimento; ad esempio, le tre diverse macchine di Turing presentate nella
854 sezione precedente possono risolvere lo stesso problema con un numero di
855 operazioni molto diverso.

856 *Problemi facili e difficili* Questo problema è stato chiaramente affrontato dividendo i problemi in
857 due classi:

- 858 • problemi *facili*, *trattabili* o *gestibili*: sono problemi per i quali esiste un
859 algoritmo capace di trovare una soluzione in tempo *polinomiale* rispetto
860 alla dimensione del problema. Un algoritmo il cui consumo di risorse
861 cresce come $O(p(n))$, dove n è la dimensione del problema e $p(\cdot)$ è un
862 polinomio, è chiamato *efficiente*;
- 863 • problemi *difficili*, *intrattabili* o *ingestibili*: se il miglior algoritmo richiede
864 risorse esponenziali⁵. In questo caso si parla di algoritmi *inefficienti*.

865 Ci sono diverse ragioni per cui la scelta di dividere i problemi in due
866 classi di complessità computazionale (polinomiale ed esponenziale) è appro-
867 priata. La prima è storica: i polinomi che descrivono la maggior parte degli
868 algoritmi proposti non hanno grado elevato, il che significa che gli algoritmi
869 polinomiali quasi sempre superano quelli esponenziali. La seconda ragione
870 è più fondamentale ed è dovuta alla *tesi forte di Church–Turing*:

871 *Tesi forte di Church–Turing* **Thesis 2.** *Qualsiasi modello di computazione può essere simulato su una macchina*
872 *di Turing probabilistica con al più un aumento polinomiale nel numero di operazioni*
873 *elementari richieste.*

874 Ciò significa che le macchine di Turing probabilistiche sembrano essere il
875 modello di computazione "ragionevole" più potente. Formalmente, possia-
876 mo affermare che se, considerando un certo modello computazionale (diver-
877 so dalla macchina di Turing probabilistica), è possibile risolvere un problema
878 con k operazioni elementari, allora è possibile risolvere lo stesso problema su
879 una macchina di Turing probabilistica con al più $p(k)$ operazioni elementari
880 (dove $p(\cdot)$ è un polinomio).

881 Invertendo questa descrizione, la tesi forte di Church–Turing afferma che,
882 dato un certo problema da risolvere, se non esiste un algoritmo efficiente per

⁵ Consideriamo esponenziale tutto ciò che cresce più velocemente di un polinomio; $n^{\log n}$, con un certo abuso di notazione, è considerata una funzione esponenziale.

la macchina di Turing probabilistica, allora non esiste un algoritmo efficiente per nessun dispositivo computazionale.

Classi di problemi

Finora abbiamo diviso i problemi in polinomiali ed esponenziali; consideriamo ora suddivisioni più fini dei problemi computazionali, tenendo conto non solo del tempo richiesto ma anche dello spazio necessario per trovare la soluzione. Prima di dividere i problemi in classi, definiamo formalmente i problemi decisionali introdotti all'inizio di questa sezione.

Un problema decisionale è un problema la cui risposta è sì o no. Possiamo definirli formalmente usando la terminologia dei *linguaggi formali*. Un linguaggio L sull'alfabeto Γ è un sottoinsieme dell'insieme Γ^* di tutte le stringhe (finite) di simboli di Γ . Una stringa rappresenta l'input del nostro problema e può essere scritta sul nastro di una macchina di Turing. La macchina esegue un calcolo che può terminare scrivendo sul nastro una stringa equivalente a sì o no a seconda che la stringa di input appartenga al linguaggio L o meno.

In generale, un linguaggio L è *deciso* da una macchina di Turing se la macchina è in grado di decidere se un input x sul suo nastro è un elemento del linguaggio L oppure no.

CLASSE P: Diciamo che un problema può essere risolto in tempo polinomiale se esiste una macchina di Turing capace di decidere se l'input x di dimensione n appartiene al linguaggio in tempo $O(n^k)$ per qualche k finito; diciamo che questo problema appartiene a **TIME**(n^k). L'insieme di tutti i linguaggi in **TIME**(n^k) è denotato con **P**. **P** è il nostro primo esempio di *classe di complessità*.

Per dimostrare che un linguaggio L appartiene a **P**, è sufficiente mostrare un algoritmo che decide L in tempo polinomiale. Purtroppo, esistono problemi per i quali tale algoritmo non è stato ancora trovato. Ci si può quindi chiedere se esistano problemi al di fuori di **P**.

Viceversa, sebbene esistano dimostrazioni non costruttive che mostrano che devono esistere problemi che richiedono risorse esponenziali, sembra molto difficile dimostrare che un problema specifico non appartiene a **P**.

CLASSE NP: Esistono problemi per i quali non è stato trovato un algoritmo polinomiale, ma se la soluzione al problema viene in qualche modo fornita, è possibile applicare un algoritmo efficiente per verificare tale soluzione. I problemi con questa proprietà appartengono alla classe **NP**. Formalmente, l'insieme **NP** contiene i linguaggi L tali che esiste una macchina di Turing M tale che:

- se $x \in L$, esiste una stringa testimone w tale che M si ferma rispondendo "sì" dopo un tempo polinomiale nella dimensione di x quando la macchina parte dallo stato xbw (b è lo spazio vuoto);
- se $x \notin L$ per qualsiasi stringa testimone w , M si ferma rispondendo "no" dopo un tempo polinomiale nella dimensione di x quando la macchina parte dallo stato xbw .

Questione aperta $P \neq NP$ Il problema aperto più famoso dell'informatica è se esistano o meno problemi in **NP** che non sono in **P**, spesso abbreviato come problema $P \neq NP$.

All'interno della classe di problemi **NP** troviamo i problemi **NP-completi**. Un linguaggio L è **NP-completo** se qualsiasi altro linguaggio in **NP** può essere ridotto a L . Si dice che un linguaggio B è riducibile a un linguaggio A se esiste una macchina di Turing che, in tempo polinomiale, partendo da x produce $R(x)$ e $x \in B$ se e solo se $R(x) \in A$. Possiamo vedere la riduzione come una traduzione di un problema in un altro equivalente: trovare la soluzione all'uno significa risolvere anche l'altro. I problemi **NP-completi** rappresentano i problemi più difficili nella classe **NP**, nel senso che trovare un limite inferiore per un problema **NP-completo** significa trovare un limite inferiore per tutti i problemi in **NP**.

Un problema **NP-completo** è il seguente: dato un circuito con n bit di input e un bit di output, determinare se esiste un'assegnazione dei bit di input tale che l'output del circuito sia 1. Questo problema è chiamato problema della soddisfacibilità del circuito o **CSAT**. La dimostrazione della **NP-completezza** di **CSAT** è dovuta al *teorema di Cook-Levin*.

Theorem 4. **CSAT** è **NP-completo**.

Una volta trovato un linguaggio L che è **NP-completo**, per dimostrare che un altro linguaggio L' è **NP-completo** è sufficiente trovare una riduzione da L a L' .

CLASSE PSPACE: Se smettiamo di considerare il tempo necessario per risolvere un problema e ci concentriamo invece sul consumo di spazio, possiamo definire la classe di problemi **PSPACE**. Questa classe contiene i linguaggi L che possono essere riconosciuti da una macchina di Turing usando un numero polinomiale di bit di lavoro senza limitazioni di tempo. Le classi di problemi **P** e **NP** sono sottoinsiemi di **PSPACE**, ma non esistono dimostrazioni che questi sottoinsiemi siano stretti o meno.

UNA CATENA DI INCLUSIONI: Possiamo organizzare le principali classi di problemi computazionali nella seguente catena di inclusioni:

$$L \subseteq P \subseteq NP \subseteq PSPACE \subseteq EXP. \quad (2.1)$$

Dove **L** sono i problemi risolvibili in spazio logaritmico e **EXP** i problemi risolvibili in tempo esponenziale.

Sebbene la maggior parte degli informatici ritenga che ciascuna di queste inclusioni sia stretta, non esistono dimostrazioni in tal senso. Esistono tuttavia risultati che ci permettono di affermare con certezza che non esiste una singola classe di problemi. Il *teorema della gerarchia temporale* e il *teorema della gerarchia spaziale* assicurano rispettivamente che $L \subset PSPACE$ e $P \subset EXP$. Ciò significa che nell'Equazione 2.1 esiste almeno un'inclusione stretta, anche se non sappiamo quale.

Soluzioni approssimate

Se abbiamo bisogno di risolvere un problema **NP-completo**, oggi possiamo contare su molti algoritmi che si comportano bene sulla maggior parte delle

istanze di questi problemi difficili. Se capita di essere particolarmente sfortunati e l'istanza che stiamo cercando di risolvere non sembra avere proprietà favorevoli per i nostri algoritmi, possiamo cambiare approccio e cercare una *soluzione approssimata*.

Esistono problemi **NP-completi** per i quali sono noti algoritmi approssimati che garantiscono un certo errore massimo rispetto alla soluzione corretta. Esiste un'intera branca dell'informatica dedicata agli algoritmi di approssimazione. È possibile definire la classe di linguaggi **MAXSNP**, analoga alla classe **NP**, per la quale è possibile verificare efficientemente una soluzione approssimata del problema.

CLASSE BPP: Questa è l'ultima classe di problemi che introduciamo considerando i modelli computazionali classici, e sarà utile nella prossima sezione dove introdurremo i modelli quantistici di computazione. Per definire questa classe di linguaggi cambiamo il modello computazionale; abbiamo già introdotto la macchina di Turing probabilistica e abbiamo visto che, seguendo la tesi forte di Church-Turing, si assume che sia il modello classico di computazione più potente.

Con questo modello possiamo definire la classe **BPP**, ovvero *bounded-error probabilistic polynomial time*, che contiene i linguaggi L per i quali esiste una macchina di Turing probabilistica M tale che se $x \in L$ allora M accetta x con probabilità $\geq \frac{3}{4}$, e se $x \notin L$ allora M rifiuta x con probabilità $\geq \frac{3}{4}$ ⁶.

Il *limite di Chernoff* implica che con un numero limitato di ripetizioni dell'algoritmo, la probabilità di successo viene amplificata fino a essere essenzialmente uno. Per questa ragione, i linguaggi in **BPP**, ancor più di quelli in **P**, sono considerati quelli risolvibili efficientemente su un computer classico.

2.1.3 Complessità computazionale quantistica

Estendiamo ora la classificazione dei problemi computazionali considerando un nuovo modello computazionale: i *circuiti quantistici*. Nella Sezione 2.2.2 definiremo con precisione cos'è una *porta quantistica* e come possono essere assemblate per creare circuiti quantistici. Per ora, è sufficiente sapere che una porta quantistica applica una trasformazione unitaria (introdotta nella Sezione 1.4.2) a uno o più bit quantistici (qubit). La dimensione di un circuito quantistico è definita dal numero di porte presenti nel circuito e dalla profondità del circuito⁷. Quest'ultima è il numero di passi temporali distinti in cui le porte vengono applicate.

Classe BQP

Grazie al concetto di circuiti quantistici possiamo definire l'alternativa quantistica alla classe **BPP**: la classe di problemi **BQP**. Questo insieme contiene i problemi decisionali che possono essere risolti con probabilità di errore limitata usando un circuito quantistico di dimensione polinomiale (sia il numero di porte che la profondità). Per essere precisi, un circuito è un'entità fissa

⁶ Il valore $\frac{3}{4}$ è arbitrario; è sufficiente che la probabilità sia maggiore di $\frac{1}{2}$.

⁷ Nella Sezione 2.2.3 utilizzeremo un altro modo di misurare la complessità: il numero di interrogazioni a un oracolo.

Definizione del
modello a circuiti
quantistici

che può trovare la soluzione di un problema fino a una certa dimensione dell'input. Per formalizzare meglio la classe **BQP** richiamiamo il concetto di famiglia di circuiti. Considerando un linguaggio L e una stringa di input x di dimensione n , diciamo che $L \in \mathbf{BQP}$ se:

- il numero di porte dei circuiti nella famiglia che risolvono il problema cresce, al più, polinomialmente rispetto a n ;
- la famiglia di circuiti è *uniforme*, ovvero esiste un insieme di istruzioni chiare, il nostro algoritmo, che genera il circuito quantistico.

Relazione con le classi classiche

Uno dei risultati più significativi nella complessità computazionale quantistica è che $\mathbf{BQP} \in \mathbf{PSPACE}$. È chiaro che $\mathbf{BPP} \in \mathbf{BQP}$. La questione se $\mathbf{BPP} \neq \mathbf{BQP}$ rimane aperta; in altre parole, non è ancora stata trovata una dimostrazione che un computer quantistico sia più potente di uno classico.

La disuguaglianza $\mathbf{BPP} \neq \mathbf{BQP}$ violerebbe la tesi forte di Church–Turing e non riguarda solo il confronto tra modelli classici e quantistici di computazione, ma avrebbe anche ripercussioni sulla gerarchia delle inclusioni classiche. Se riscriviamo l'Equazione 2.1 alla luce di $\mathbf{BQP} \in \mathbf{PSPACE}$ otteniamo:

$$\mathbf{L} \subseteq \mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{BQP} \subseteq \mathbf{PSPACE} \subseteq \mathbf{EXP}. \quad (2.2)$$

Assumendo che $\mathbf{BPP} \neq \mathbf{BQP}$, potremmo dedurre che $\mathbf{BPP} \neq \mathbf{PSPACE}$.

Esistono alcune indicazioni che i computer quantistici potrebbero essere più potenti di quelli classici. I due esempi più famosi di superiorità quantistica sono l'algoritmo di Grover e l'algoritmo di Shor.

L'algoritmo di Grover mostra un'accelerazione quadratica nella ricerca di un elemento in una lista non ordinata rispetto al limite inferiore di un algoritmo classico. L'algoritmo di ricerca di Grover è ottimale: la sua complessità è uguale al limite inferiore teorico nel modello computazionale quantistico.

L'algoritmo di Shor mostra un'accelerazione esponenziale nella fattorizzazione di un numero rispetto al miglior algoritmo classico noto. La chiave per la soluzione quantistica efficiente del problema della fattorizzazione è stata lo sfruttamento di una struttura nascosta all'interno del problema.

Limiti del calcolo
quantistico

Molti informatici ritengono che la ragione essenziale della difficoltà dei problemi **NP**-completi sia che il loro spazio di ricerca non ha struttura, e che il miglior metodo possibile per risolvere tali problemi (nel caso più generale) sia adottare un metodo di ricerca.

Adottare questo punto di vista significa che il computer quantistico non può risolvere efficientemente un problema **NP**-completo perché la massima accelerazione ottenibile è quadratica. I ricercatori nel campo del calcolo quantistico e dell'informazione quantistica si concentrano quindi su un'altra classe di problemi per dimostrare la superiorità del calcolo quantistico su quello classico. Questa classe di problemi è chiamata **NPI** (*NP-intermedio*) e contiene i problemi in **NP** che sono sia al di fuori di **P** (o **BPP**) che dei problemi **NP**-completi.

Si ritiene che il problema della fattorizzazione sia in **NPI** e l'algoritmo di Shor sembrerebbe dimostrare la superiorità computazionale del modello

quantistico; tuttavia, purtroppo, non esiste una dimostrazione che $\mathbf{P} \neq \mathbf{NPI}$.
 Queste considerazioni sono approfondite in [3, pagine 150, 151, 270 e 271]

2.2 PARADIGMA BASATO SU PORTE QUANTISTICHE

Il paradigma di computazione basato su porte quantistiche è l'implementazione diretta del modello computazionale che abbiamo appena descritto. Questo approccio al calcolo quantistico è fondato su un esteso lavoro teorico, parte del quale abbiamo menzionato nella sezione precedente, e per questo motivo la maggior parte dei risultati teorici sul calcolo quantistico deriva da questo paradigma.

Come suggerisce il nome, questo modello è basato su porte quantistiche; le porte quantistiche manipolano i qubit applicando loro trasformazioni fisiche. Lo scopo del Capitolo 1 era convincerci che queste porte devono avere determinate caratteristiche per essere conformi alle leggi della fisica.

Le porte quantistiche modificano i qubit applicando operatori unitari. Ci siamo concentrati sul significato fisico di un operatore unitario nella Sezione 1.4.2, dove abbiamo visto come, conoscendo lo stato del sistema dopo l'applicazione dell'operatore, potessimo risalire allo stato iniziale.

Ci concentriamo ora sulle implicazioni che un operatore unitario ha su un modello computazionale. La prima conseguenza, la più ovvia, è che siamo autorizzati a costruire solo porte reversibili, ovvero porte che ci permettono di determinare lo stato dei qubit di input se conosciamo lo stato dei qubit di output.

2.2.1 Porte reversibili classiche

Le porte reversibili non esistono solo nel contesto dei computer quantistici. Esistono numerosi esempi di porte reversibili classiche e vi sono studi che partono da una computazione classica completamente reversibile per ridurre drasticamente il consumo energetico dei computer. È utile per noi esaminare alcune porte reversibili classiche perché alcune considerazioni che possiamo fare su di esse saranno utili nello studio delle porte quantistiche.

La porta reversibile più semplice è la porta NOT (Figura 2.1a). Un'altra importante porta reversibile classica è la porta TOFFOLI mostrata nella Figura 2.2⁸.

La porta TOFFOLI è anche chiamata *controlled controlled NOT*. Dalla Figura 2.2 vediamo che il valore del bit C viene negato se sia il bit di controllo A che il bit di controllo B sono impostati.

Se osserviamo la tavola di verità della porta TOFFOLI (Figura 2.3) possiamo notare come gli input siano univocamente identificati dagli output (definizione di porta reversibile). La porta TOFFOLI ha tre bit di input, quindi la tavola di ve-

A	B	C	A	B	$AB \oplus C$
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	1
1	1	1	1	1	0

⁸ La Figura 2.2b mostra l'implementazione della porta TOFFOLI usando le porte mostrate nella Figura 2.1.

Figura 2.3: Tavola di verità della porta TOFFOLI

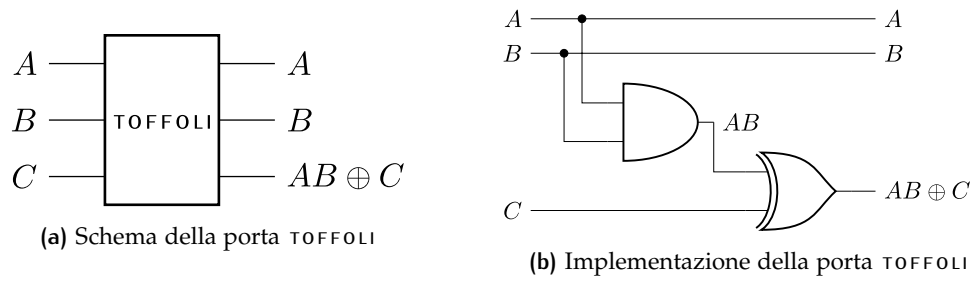


Figura 2.2: Esempio di circuito

1096 rità ha $2^3 = 8$ righe; poiché gli output
 1097 devono essere chiavi univoche non ci so-
 1098 no ripetizioni negli output. Ciò significa
 1099 che tutte le possibili combinazioni di tre
 1100 bit compaiono anche negli output. Pos-
 1101 siamo quindi dedurre che la porta TOF-
 1102 FOLI agisce *permutando* le righe di input
 1103 della tavola di verità. Questo è vero in
 1104 generale:

1105 **Theorem 5.** *La multi-colonna di output della tavola di verità di una porta reversi-*
 1106 *bile classica è una permutazione della multi-colonna di input.*

1107 Questo concetto di permutazione dell'input riapparirà anche quando di-
 1108 scuteremo le porte quantistiche.

1109 2.2.2 Porta quantistica

1110 Nel Capitolo 1 abbiamo visto che lo spin è un esempio di proprietà binaria,
 1111 e abbiamo classificato come qubit tutte quelle proprietà fisiche che, quando
 1112 misurate, restituiscono solo due valori. Abbiamo misurato lo spin rispetto
 1113 agli assi coordinati e identificato diversi stati di spin (*su* e *giù*, *sinistra* e *destra*,
 1114 *dentro* e *fuori*).

1115 Questi stati cambiano leggermente notazione quando passiamo dallo stu-
 1116 dio della fisica all'informatica. Prima di descrivere le porte quantistiche
 1117 introduciamo questa notazione alternativa per gli stati di un qubit e mostria-
 1118 mo una rappresentazione grafica che sarà utile per comprendere l'effetto
 1119 delle porte quantistiche.

1120 Qubit

1121 *Spin e qubit* In informatica gli stati *su* e *giù* (Equazioni 1.4 e 1.5) rappresentano rispetti-
 1122 vamente un qubit spento $|0\rangle$ e acceso $|1\rangle$. Denotiamo con $|+\rangle$ e $|-\rangle$ i qubit
 1123 negli stati *destra* e *sinistra* (Equazioni 1.6 e 1.7); infine gli stati *dentro* e *fuori*
 1124 (Equazioni 1.8 e 1.9) sono rappresentati da $|i\rangle$ e $|-i\rangle$. Possiamo riscrivere le
 1125 equazioni citate considerando la nuova notazione:

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \quad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \quad (2.3)$$

1126 e:

$$|i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle) \quad |-i\rangle = \frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle) \quad (2.4)$$

Per rappresentare i qubit in modo grafico che rispetti le proprietà matematiche degli stati quantistici, possiamo usare una sfera chiamata *sfera di Bloch*. La sfera ha raggio 1 e ogni punto sulla sfera rappresenta un qubit⁹. La Figura 2.4 mostra, sulla sfera di Bloch, i qubit descritti sopra.

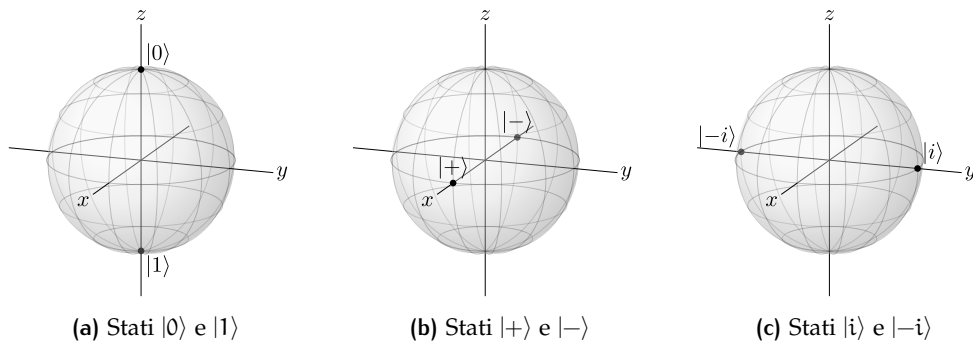


Figura 2.4: Sfera di Bloch [6]

Uno stato rappresentato come un punto sulla superficie della sfera di Bloch è chiamato *stato puro*. È anche possibile considerare i punti all'interno della sfera, nel qual caso si parla di *stati misti*. Per continuare la nostra discussione, possiamo considerare solo gli stati puri.

Porta quantistica

Richiamando i risultati ottenuti nel Capitolo 1 e considerando anche quanto detto sulle porte reversibili classiche nella Sezione 2.2.1, possiamo fare le seguenti affermazioni riguardo alle porte quantistiche:

- le porte quantistiche sono mappe lineari che preservano la probabilità totale uguale a 1^{10} ;
- possiamo rappresentare una porta quantistica come una matrice che preserva la probabilità totale uguale a 1;
- le porte quantistiche sono matrici unitarie e le matrici unitarie sono porte quantistiche;
- una porta quantistica U è reversibile e $U^\dagger$ è la porta inversa;
- le porte reversibili classiche sono valide porte quantistiche¹¹.

Ora che abbiamo descritto le principali proprietà delle porte quantistiche possiamo esaminare l'effetto che queste porte hanno sui qubit; iniziamo la nostra analisi con le porte che operano su un singolo qubit.

PORTE QUANTISTICHE A QUBIT SINGOLO: Le porte quantistiche che operano su un singolo qubit possono essere visualizzate come *rotazioni* sulla sfera di Bloch. Qualsiasi rotazione può essere percorsa sia in avanti che indietro,

⁹ A meno di una fase globale, che è tuttavia irrilevante.

¹⁰ Ricordiamo che la probabilità totale dello stato $|s\rangle = c_1|0\rangle + c_2|1\rangle$ formato da un qubit è $\alpha^2 + \beta^2$.

¹¹ In particolare, è possibile dimostrare che una porta reversibile classica permuta le ampiezze di probabilità, preservando così la probabilità totale uguale a 1.

il che rende le rotazioni reversibili e, di conseguenza, valide porte quantistiche. Esaminiamo ora alcune rotazioni che definiscono porte particolarmente interessanti.

Porte di Pauli Le rotazioni di 180° rispetto agli assi definiscono le *porte di Pauli*:

- Porta di Pauli X: rotazione rispetto all'asse x . Questa porta è la versione quantistica della porta classica NOT:

$$X|0\rangle = |1\rangle \quad X|1\rangle = |0\rangle ;$$

- Porta di Pauli Y: rotazione rispetto all'asse y . Questa porta non ha un corrispondente classico e, applicata ai qubit $|0\rangle$ e $|1\rangle$, li modifica come segue:

$$Y|0\rangle = i|1\rangle \quad Y|1\rangle = -i|0\rangle ;$$

- Porta di Pauli Z: rotazione rispetto all'asse z . Anche questa è una porta puramente quantistica, che agisce come segue:

$$Z|0\rangle = |0\rangle \quad Z|1\rangle = -|1\rangle ;$$

- Porta di Pauli I: rotazione zero. Questa porta non modifica lo stato dei qubit:

$$I|0\rangle = |0\rangle \quad I|1\rangle = |1\rangle .$$

È anche possibile definire queste porte come matrici; la definizione è la stessa degli operatori che rappresentavano le osservabili σ_x , σ_y e σ_z (rispettivamente Equazioni 1.3.3, 1.3.3 e 1.3.3) più la matrice identità per la porta I. Infatti, se osserviamo le matrici che descrivono le osservabili notiamo che, oltre ad essere operatori hermitiani (che descrivono osservabili), sono anche unitarie, quindi valide porte quantistiche.

Possiamo quindi considerare rotazioni inferiori a 180° ; definiamo la porta S o *porta di fase* come una rotazione di 90° rispetto all'asse z e dimezzando nuovamente l'angolo di rotazione otteniamo la porta T (rotazione di 45° rispetto a z). L'azione di T su $|0\rangle$ e $|1\rangle$ è:

$$T|0\rangle = |0\rangle \quad T|1\rangle = e^{i\frac{\pi}{4}}|1\rangle .$$

L'ultima porta che agisce su un singolo qubit che consideriamo è la *porta di Hadamard* (H). Questa porta può essere interpretata come una rotazione di 180° rispetto all'asse $x + z$. H consente la creazione di sovrapposizioni di stati¹², in particolare:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle \quad H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle .$$

Porte importanti sulla sfera di Bloch

La Figura 2.5 mostra le porte descritte come rotazioni sulla sfera di Bloch.

¹² Torneremo su questo nella Sezione 2.2.2.

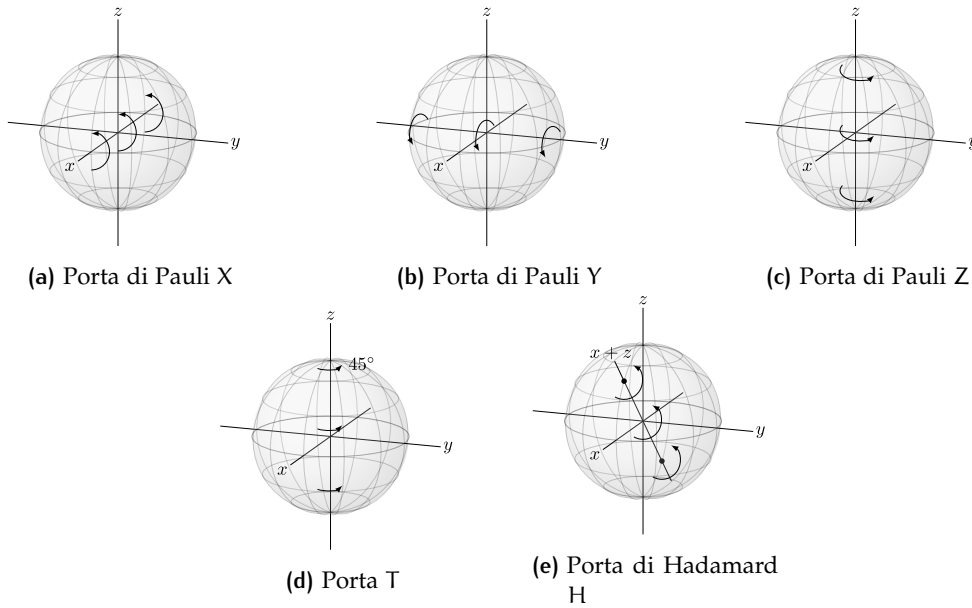


Figura 2.5: Porte come rotazioni [6]

1182 **PORTE QUANTISTICHE A QUBIT MULTIPLI:** Quando descriviamo stati com-
 1183 posti da più qubit dobbiamo rappresentarli come un *prodotto tensoriale* $\otimes^{13}$.
 1184 Ad esempio, uno stato in cui due qubit si trovano entrambi nello stato $|0\rangle$ è
 1185 scritto:

$$|0\rangle \otimes |0\rangle \text{ abbreviato come } |0\rangle |0\rangle \text{ abbreviato come } |00\rangle.$$

1186 Con due qubit la base z è $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$. Un generico stato $|s\rangle$ sarà
 1187 la sovrapposizione degli stati della base

$$|s\rangle = c_1 |00\rangle + c_2 |01\rangle + c_3 |10\rangle + c_4 |11\rangle,$$

1188 e la probabilità totale sarà $|c_1|^2 + |c_2|^2 + |c_3|^2 + |c_4|^2$ che, se lo stato è norma-
 1189 lizzato, sarà uguale a 1.

1190 Alcuni stati composti possono essere scritti come prodotto tensoriale di
 1191 stati a qubit singolo, ad esempio:

$$\frac{1}{2}(|00\rangle - |01\rangle + |10\rangle - |11\rangle) = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |+\rangle |-\rangle.$$

1192 Questi stati sono chiamati *stati prodotto* o *stati semplicemente separabili*. Esisto-
 1193 no stati che non possono essere decomposti in stati semplici. In questo caso
 1194 si parla di *stati entangled*, stati in cui la conoscenza completa del vettore di
 1195 stato dell'intero sistema non fornisce alcuna informazione sugli stati degli
 1196 elementi che compongono il sistema¹⁴.

*Stati
entangled*

1197 Studiamo le porte quantistiche a qubit multipli precisamente per la loro ca-
 1198 pacità di creare stati entangled. La prima porta che consente l'entanglement
 1199 tra qubit è la $CNOT$ o CX .

¹³ La descrizione matematica del prodotto tensoriale va oltre gli obiettivi di questo lavoro.

¹⁴ L'entanglement è uno degli aspetti più interessanti della meccanica quantistica. Una discussione dettagliata delle sue implicazioni fisiche va oltre lo scopo di questo lavoro. Per materiale introduttivo sull'entanglement si rimanda il lettore a [2].

1200 La porta CNOT (*controlled NOT*) è una
 1201 porta a due qubit; il primo è chiama-
 1202 to qubit di *controllo*, il secondo è il *qu-*
 1203 *bit target*. Se il qubit di controllo è
 1204 impostato, il qubit target viene inverti-
 1205 to (da cui il nome *controlled NOT*). Il
 1206 comportamento della porta CNOT può essere descritto come:

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Figura 2.6: Porta CNOT

1207 La Figura 2.6 riporta la definizione matriciale della CNOT. Questa porta è
 1208 in grado di creare stati entangled. In particolare:

$$\begin{aligned} \text{CNOT} |+\rangle |0\rangle &= \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = |\Psi^+\rangle \\ \text{CNOT} |-\rangle |0\rangle &= \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) = |\Psi^-\rangle \\ \text{CNOT} |+\rangle |1\rangle &= \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) = |\Phi^+\rangle \\ \text{CNOT} |+\rangle |1\rangle &= \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) = |\Phi^-\rangle. \end{aligned} \quad (2.5)$$

1209 *Stati di Bell* Gli stati $|\Psi^+\rangle, |\Psi^-\rangle, |\Phi^+\rangle$ e $|\Phi^-\rangle$ non possono essere ottenuti come prodotti
 1210 tensoriali di due stati a qubit singolo e sono quindi stati entangled. Questi
 1211 quattro stati sono chiamati *stati di Bell*; sono ortogonali e formano una base
 1212 chiamata *base di Bell*.

1213 Altre porte quantistiche importanti sono quelle che abbiamo già incontrato
 1214 quando abbiamo discusso le porte classiche; abbiamo già visto che una porta
 1215 reversibile classica è una valida porta quantistica. Le porte di maggiore
 1216 interesse per noi sono la porta TOFFOLI e la porta CROSSOVER.

1217 Non è possibile, tuttavia, costruire un analogo quantistico della porta FA-
 1218 NOUT; quindi non possiamo ottenere una copia di un qubit senza conoscerne
 1219 lo stato. Misurare lo stato, tuttavia, lo modificherebbe inevitabilmente, quin-
 1220 di in generale non possiamo duplicare un qubit. Questo è un importante
 1221 risultato teorico ed è noto come *teorema di non-clonazione*:

1222 *Teorema di non-clonazione* **Theorem 6.** È impossibile fare una copia di uno stato quantistico sconosciuto. I
 1223 qubit non possono essere copiati.

1224 Porte quantistiche universali

1225 Così come esistono insiemi di porte classiche che consentono la costruzione
 1226 di qualsiasi altra porta¹⁵, possiamo identificare insiemi di porte quantistiche
 1227 universali. Un insieme di porte quantistiche universali deve essere in grado
 1228 di:

- 1229 • creare sovrapposizioni di stati come la porta di Hadamard H;
- 1230 • creare stati entangled come la porta CNOT;

¹⁵ Esempi di questi insiemi sono: {AND, NOT}, {OR, NOT}, {NAND} e {TOFFOLI}.

- creare ampiezze di probabilità complesse; H e $CNOT$ contengono solo valori reali, mentre la porta di fase S consente di ottenere ampiezze di probabilità complesse.

Sebbene l'insieme di porte che abbiamo implicitamente definito $\{CNOT, H, S\}$ abbia tutte le proprietà richieste, si può dimostrare che non è un insieme universale. In particolare, il *teorema di Gottesman-Knill* afferma che:

Theorem 7. *Un circuito quantistico contenente solo porte $CNOT$, H e S è simulato efficientemente da un computer classico.*

Chiamiamo i circuiti generati da questo insieme di porte il *gruppo di Clifford*, e l'insieme $\{CNOT, H, S\}$ prende di conseguenza il nome di *generatore del gruppo di Clifford*. Un insieme di porte universali deve quindi generare più del gruppo di Clifford, anche se non è stato dimostrato che questa sia una condizione sufficiente per essere un insieme universale.

Esempi di insiemi di porte universali sono:

*Insiemi di porte
quantistiche universali*

- $\{CNOT, H, T\}$: sostituendo la porta di fase con la porta T nel generatore del gruppo di Clifford si ottiene un insieme universale di porte;
- $\{TOFFOLI, H, S\}$: partendo nuovamente dal generatore del gruppo di Clifford, si ottiene un insieme universale sostituendo la $CNOT$ con la porta $TOFFOLI$.

Il concetto di insieme universale di porte nel calcolo quantistico è diverso da quello a cui siamo abituati con le porte classiche. Nel caso classico possiamo costruire qualsiasi tavola di verità partendo da un insieme di porte classiche universali. Il significato di un insieme universale nel calcolo quantistico è invece descritto dal *teorema di Solovay-Kitaev*:

Theorem 8. *Qualsiasi insieme universale di porte può approssimare una porta quantistica su n qubit con precisione ϵ usando $\Theta(2^n \log^c(\frac{1}{\epsilon}))$ porte per qualche costante c .*

Il termine 2^n non dovrebbe sorprendere perché un operatore unitario che agisce su n qubit (ovvero la porta che stiamo cercando di approssimare) è una matrice di dimensione $2^n \times 2^n$. Ciò che è davvero rilevante è il termine $\log^c(\frac{1}{\epsilon})$, che ci dice che possiamo ridurre l'errore di approssimazione a un costo che cresce come un polinomio di un logaritmo —questa funzione è spesso chiamata *polylog*— il che è efficiente.

2.2.3 Algoritmi

Abbiamo discusso cosa sono le porte quantistiche e come possono essere assemblate per costruire un circuito quantistico. Vediamo ora come questo circuito deve essere costruito per essere utile nella risoluzione di un problema computazionale. In questa sezione introduciamo i circuiti quantistici basati su oracolo e presentiamo il più semplice algoritmo basato su oracolo.

1270 *Oracoli quantistici*

1271 Un oracolo è una funzione booleana, ovvero una funzione che agisce sui
1272 qubit e che può essere composta con porte. Per poter essere tradotta in porte
1273 quantistiche dobbiamo assicurarci che la porta risultante sia reversibile. In
1274 generale, è possibile rendere reversibile un circuito usando un *qubit extra* e
1275 applicando la porta `xor` tra l'output del circuito e il qubit extra.

1276 Ad esempio, se la funzione $f(x)$ agi-
1277 sce su un singolo qubit possiamo co-
1278 struire una porta reversibile U_f come

$$\begin{array}{ccc} |y\rangle & \text{---} \boxed{U_f} & |y \oplus f(x)\rangle \\ |x\rangle & \text{---} & |x\rangle \end{array}$$

1279 mostrato nella Figura 2.7. È facile ve-
1280 dere che indipendentemente dall'azio-
1281 ne di $f(x)$, che può essere reversibile o
1282 meno, conoscendo gli output della porta è possibile ricostruire gli input.
1283 Possiamo riscrivere l'applicazione di U_f ai qubit come:

Figura 2.7: Porta oracolo quantistico

$$|x\rangle |y\rangle \xrightarrow{U_f} |x\rangle |y \oplus f(x)\rangle$$

1284 Il qubit extra —quello che abbiamo usato per rendere U_f reversibile— è
1285 chiamato *qubit risposta* o *qubit target*, mentre $|x\rangle$ è chiamato *qubit di input*.
1286 Quando impostiamo $|y\rangle = |0\rangle$ possiamo leggere il risultato di $f(x)$ applicato
1287 a $|x\rangle$ nel qubit target.

1288 In realtà, questo non è il modo standard di utilizzare un oracolo quanti-
1289 stico. Ciò che si fa invece è preparare il qubit target nello stato $|-\rangle$ e poi
1290 applicare la porta U_f . Le porte quantistiche possono essere applicate anche
1291 a stati in sovrapposizione, e in questo caso il risultato è un cambiamento di
1292 fase del qubit $|x\rangle$ mentre il qubit target rimane invariato. Possiamo scrivere
1293 l'applicazione di questa porta come:

$$|x\rangle |-\rangle \xrightarrow{U_f} \begin{cases} |x\rangle |-\rangle & \text{se } f(x) = 0 \\ -|x\rangle |-\rangle & \text{se } f(x) = 1 \end{cases} = (-1)^{f(x)} |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle,$$

1294 dove, nell'ultimo passaggio, omettiamo il valore del qubit target perché non
1295 viene modificato dalla porta.

Oracolo di fase

$$\begin{array}{ccc} |0\rangle & \text{---} \boxed{H} \text{---} \boxed{Z} \text{---} \boxed{U_f} & |-\rangle \\ |x\rangle & \text{---} & (-1)^{f(x)} |x\rangle \end{array}$$

1300 **Figura 2.8:** Oracolo di fase
1301 può vedere come sia possibile preparare $|0\rangle$ nello stato $|-\rangle$ per poi usare U_f
1302 come oracolo di fase¹⁶.

Poiché, usato in questo modo, l'o-
racolo applica un cambiamento di
fase a $|x\rangle$, è chiamato *oracolo di fase*.
La Figura 2.8 mostra l'applicazione
dell'oracolo di fase; in particolare, si

1303 *Algoritmo di Deutsch*

1304 L'algoritmo di Deutsch mostra come usare l'oracolo di fase per risolvere un
1305 problema computazionale. Vedremo che già con questo primo algoritmo ot-
1306 teniamo prestazioni migliori rispetto a quelle ottenibili con un'architettura
1307 classica. In questo caso, tuttavia, il guadagno in prestazioni è polinomiale;

¹⁶ È possibile ottenere lo stesso risultato applicando prima la porta di Pauli X e poi la porta di Hadamard H.

1308 pertanto questo non è uno di quegli algoritmi che suggeriscono che i compu-
 1309 ter quantistici possano risolvere efficientemente una classe di problemi più
 1310 ampia rispetto ai computer classici¹⁷.

1311 Il problema computazionale che vogliamo risolvere è determinare la *parità*
 1312 di due bit sconosciuti b_1 e b_2 . Determinare la parità significa calcolare $b_1 \oplus$
 1313 b_2 . Per ottenere il risultato ci viene fornito l'accesso a un oracolo $f(x)$ con
 1314 $x \in \{0, 1\}$ che, quando interrogato con l'indice del bit sconosciuto, restituisce
 1315 il valore del bit, ovvero:

$$f(0) = b_0 \quad f(1) = b_1.$$

1316 In un modello computazionale classico dobbiamo interrogare l'oracolo
 1317 due volte, ottenere i valori dei bit e calcolare il risultato dello xor tra i
 1318 due valori.

1319 L'algoritmo di Deutsch ci permette di ottenere il valore di parità interro-
 1320 gando l'oracolo una sola volta. La Figura 2.9 mostra come viene implemen-
 tato il circuito quantistico.

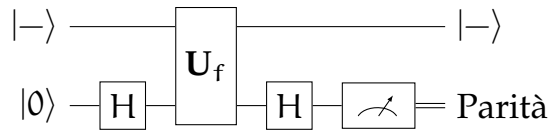


Figura 2.9: Algoritmo di Deutsch

1321 Senza sviluppare tutti i passaggi algebrici, vediamo cosa accade al qubit di
 1322 input $|0\rangle$ mentre attraversa le porte quantistiche. La prima porta quantistica
 1323 è la porta H; la sua applicazione trasforma $|0\rangle$ in
 1324

$$|0\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle.$$

1325 Interroghiamo ora l'oracolo con un input in una sovrapposizione di stati. Il
 1326 risultato dell'interrogazione è¹⁸

*Oracolo di fase
e sovrapposizione*

$$\left[\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \right] |-\rangle \xrightarrow{U_f} \frac{1}{\sqrt{2}} \left[(-1)^{f_0} |0\rangle + (-1)^{f_1} |1\rangle \right].$$

1327 Tralasciando alcuni passaggi possiamo semplificare questa espressione co-
 1328 me:

$$\begin{cases} (-1)^{b_0} |+\rangle & \text{se } b_0 = b_1 \\ (-1)^{b_0} |-\rangle & \text{se } b_0 \neq b_1 \end{cases}.$$

1329 Dopo l'oracolo, se $b_0 = b_1$, otteniamo lo stato $|+\rangle$ a meno di una fase glo-
 1330 bale; se $b_0 \neq b_1$ otteniamo lo stato $|-\rangle$ a meno di una fase globale. Questa
 1331 fase non modifica la posizione del qubit sulla sfera di Bloch, e possiamo già
 1332 distinguere i due stati con una misura parallela all'asse x . Poiché, conven-
 1333 zionalmente, le misure vengono effettuate rispetto all'asse z , applichiamo
 1334 nuovamente la porta H ottenendo

$$\begin{cases} (-1)^{b_0} |0\rangle & \text{se } b_0 = b_1 \\ (-1)^{b_0} |1\rangle & \text{se } b_0 \neq b_1 \end{cases}.$$

Le fasi globali sono
fisicamente irrilevanti.

L'atto di misura scarta la fase globale e ci permette di distinguere i due casi con certezza: otteniamo $|0\rangle$ se i bit sconosciuti erano uguali (parità pari) e $|1\rangle$ se i bit sconosciuti erano diversi (parità dispari). È interessante notare che durante l'intero algoritmo non veniamo mai a conoscenza del valore dei singoli bit, ma solo se sono uguali o meno.

L'algoritmo di Deutsch ci permette di dimezzare il numero di interrogazioni all'oracolo, e può essere esteso a un numero arbitrario di bit di cui vogliamo determinare la parità. Questo esempio conclude la nostra discussione del paradigma basato su porte quantistiche. Incontreremo nuovamente questo paradigma nel Capitolo 7, dove lo combineremo con i concetti di quantum annealing che illustriamo ora.

2.3 CALCOLO ADIABATICO QUANTISTICO

Il Calcolo Adiabatico Quantistico (CAQ) è un paradigma di calcolo quantistico alternativo al modello basato su porte. I due modelli sono computazionalmente equivalenti, ma profondamente diversi nella natura della computazione. Il CAQ ha una natura analogica che contrasta con quella digitale del paradigma basato su porte. Quando adottiamo il CAQ, studiamo l'evoluzione di un sistema fisico nel tempo.

Prima di descrivere più dettagliatamente questo approccio al calcolo quantistico, è necessario fornire alcune definizioni che saranno utili nella discussione seguente. Chiamiamo *sistema dinamico quantistico* un sistema formato da n qubit che evolve nel tempo secondo determinate forze. L'evoluzione temporale è determinata dall'Hamiltoniano $\mathbf{H}(t)$ ¹⁹ del sistema ed è descritta dall'equazione di Schrödinger (Equazione 1.15). Come osservato nella Sezione 1.4.3, $\mathbf{H}(t)$ è un operatore hermitiano, e l'osservabile ad esso associata è l'energia del sistema. Gli autovalori di $\mathbf{H}(t)$ in un dato istante rappresentano tutti i possibili livelli energetici del sistema che possiamo misurare. Chiamiamo questo insieme di valori lo *spettro energetico* del sistema. Siamo interessati a due particolari livelli energetici: quello di energia più bassa, chiamato *stato fondamentale*, e il *primo stato eccitato*, il livello energetico immediatamente superiore allo stato fondamentale.

2.3.1 Algoritmo CAQ

Gli algoritmi CAQ sono progettati per risolvere problemi di ottimizzazione formulati come segue: data una funzione obiettivo $f: \mathbb{D}^n \rightarrow \mathbb{R}$ definita su n variabili $x = x_1, \dots, x_n$ appartenenti a qualche dominio discreto $\mathbb{D}$, trovare un'assegnazione di valori a x che minimizzi $f(x)$.

Un algoritmo CAQ per risolvere il problema di ottimizzazione P con funzione obiettivo $f(x)$ è descritto da un Hamiltoniano dipendente dal tempo $\mathbf{H}(t)$ che definisce l'evoluzione del sistema Q composto da n qubit. $\mathbf{H}(t)$ è specificato da tre componenti:

¹⁷ La presentazione di questi algoritmi, incluso l'algoritmo di Shor, va oltre lo scopo di questo lavoro.

¹⁸ Dove, nel risultato, abbiamo ommesso il qubit risposta $|-\rangle$.

¹⁹ Lo scriviamo in questo modo per sottolineare che, in generale, $\mathbf{H}$ dipende anche dal tempo.

- 1375 • un *Hamiltoniano iniziale* $\mathbf{H}_i$ scelto in modo che il suo stato fondamentale
1376 sia facile da trovare;
- 1377 • un *Hamiltoniano finale o del problema* $\mathbf{H}_f$ che codifica la funzione obiet-
1378 tivo in modo che lo stato fondamentale del sistema sia un autostato di
1379 $\mathbf{H}_f$ con autovalore minimo. Ovvero, lo stato fondamentale del sistema
1380 corrisponde a una soluzione ottimale di P;
- 1381 • un *percorso di evoluzione adiabatica*, una funzione $s(t)$ che decresce da 1
1382 a 0 mentre il tempo va da 0 a t_f .

1383 L'algoritmo CAQ è definito da $\mathbf{H}(t)$ che crea una transizione graduale (adia-
1384 batica) da $\mathbf{H}_i$ a $\mathbf{H}_f$ secondo

$$\mathbf{H}(t) = s(t) \cdot \mathbf{H}_i + (1 - s(t)) \cdot \mathbf{H}_f. \quad (2.6)$$

1385 Per creare l'Hamiltoniano iniziale applichiamo a tutti i qubit l'operatore
1386 di Pauli X (σ_x nell'Equazione 1.10) —questo operatore descrive l'osservabile
1387 spin parallelo all'asse x , di conseguenza— gli autovettori associati sono $|+\rangle$
1388 e $|-\rangle$. In particolare, $\mathbf{H}_i$ è definito come

$$\mathbf{H}_i = \sum_{i=1}^n \frac{1}{2} (1 - X_{q_i})$$

1389 dove X_{q_i} indica l'applicazione della porta di Pauli X al qubit i . Lo stato fon-
1390 damentale è quindi formato da tutti gli n qubit nello stato $|+\rangle$. Ciò significa
1391 che è stato creato uno stato in cui gli autostati nella base standard ($\{|0\rangle, |1\rangle\}$)
1392 sono osservabili con uguale probabilità.

1393 In generale, $\mathbf{H}_i$ è specificato in una base ortogonale a quella dell'Hamiltoniano
1394 finale al fine di creare uno stato di massima sovrapposizione.

*Stato iniziale come
massima sovrapposizione*

1395 Partendo dallo stato fondamentale noto di $\mathbf{H}_i$ ed evolvendo grazie a $s(t)$
1396 fino a $\mathbf{H}_f$, il *teorema adiabatico* garantisce che, sotto certe condizioni, Q rimar-
1397 rà sempre nello stato fondamentale di $\mathbf{H}(t)$, e al tempo t_s saremo in grado
1398 di leggere nella configurazione dei qubit la soluzione del problema P.

1399 A prima vista potrebbe sembrare che questa formulazione limiti le capa-
1400 cità del CAQ; in realtà, possiamo riscrivere naturalmente molti problemi in
1401 NP come problemi di ottimizzazione. Nella Sezione 2.1.3 abbiamo menzio-
1402 nato il problema della fattorizzazione in numeri primi come candidato per
1403 dimostrare la superiorità del calcolo quantistico. Questo problema può es-
1404 sere tradotto in un problema di ottimizzazione in cui, dato un intero N , si
1405 devono trovare due interi $x, y > 1$ tali da minimizzare $f(x, y) = (N - x \cdot y)^2$.

1406 2.3.2 Risultati teorici

1407 Vediamo alcuni risultati su cui si basa il CAQ, che definiscono le classi di
1408 problemi affrontabili e stabiliscono la relazione tra CAQ e il modello basato
1409 su porte.

1410 *Teorema adiabatico*

1411 Il teorema adiabatico garantisce, sotto condizioni abbastanza restrittive, che
1412 il calcolo effettuato dal CAQ termini effettivamente nello stato fondamentale

1413 di $\mathbf{H}_f$. Consideriamo l'algoritmo $\mathbf{H}(t)$ che evolve nel tempo $t : 0 \rightarrow t_f$; ripa-
 1414 rametrizziamo il flusso del tempo con $s(t)$ strettamente decrescente da 1 a 0
 1415 mentre $t : 0 \rightarrow t_f$. L'Hamiltoniano associato a $s(t)$ è $\tilde{\mathbf{H}}(s)$. Infine, chiamiamo
 1416 $\tau(s)$ il tasso con cui $\tilde{\mathbf{H}}(s)$ evolve in funzione di $s(t)$. Denotando con $|\Phi_s\rangle$
 1417 lo stato del sistema in funzione di $s(t)$, possiamo riscrivere l'equazione di
 1418 Schrödinger 1.15 per il nostro sistema come²⁰:

$$\frac{\partial}{\partial s} |\Phi_s\rangle = -i\tau(s)\tilde{\mathbf{H}}(s) |\Phi_s\rangle.$$

1419 Assumendo che durante l'intera evoluzione del sistema lo stato fonda-
 1420 mentale sia sempre unico, chiamiamo $\delta(s)$ il *gap spettrale*: la differenza tra
 1421 gli autovalori dello stato fondamentale e del primo stato eccitato, in qualsiasi
 1422 istante s . Chiamiamo δ_m il *gap spettrale minimo* $\delta_m = \min_s \delta(s)$. Assumiamo
 1423 che

$$\tau(s) \ll \frac{\left\| \frac{\partial}{\partial s} \tilde{\mathbf{H}}(s) \right\|}{\delta_m^2} \quad (2.7)$$

1424 dove $\|\cdot\|$ è la norma matriciale indotta dalla metrica L_2 .

1425 Denotando con $|\Phi_g\rangle$ lo stato fondamentale dell'Hamiltoniano finale, il
 1426 *teorema adiabatico* afferma che:

1427 **Theorem 9.**

Teorema adiabatico

- 1428 • Se il sistema si trova nello stato fondamentale a $s = 0$, e
 - 1429 • se δ_s è strettamente maggiore di 0 per tutto il tempo s , e
 - 1430 • se il processo evolve abbastanza lentamente da rispettare l'Equazione 2.7
- 1431 allora il processo terminerà nello stato fondamentale $|\Phi_{t_f}\rangle = |\Phi_g\rangle$ con probabilità
 1432 sufficientemente alta.

1433 **Complessità temporale**

1434 Per ottenere un limite superiore al tempo computazionale t_f per un pro-
 1435 blema P di dimensione n , è necessario trovare un limite superiore per il
 1436 numeratore $\left\| \frac{\partial}{\partial s} \tilde{\mathbf{H}}(s) \right\|$ e un limite inferiore per il denominatore δ_m^2 nell'E-
 1437 quazione 2.7.

1438 Il numeratore è limitato dalla massima differenza tra gli autovalori otteni-
 1439 bili dagli Hamiltoniani $\mathbf{H}_i$ e $\mathbf{H}_f$. È possibile dimostrare che questa differenza
 1440 è polinomiale nella dimensione n del problema.

Gap spettrale 1441 In generale, l'analisi del gap spettrale minimo δ_m^2 per un dato algoritmo
 1442 e $\mathbf{P} \neq \mathbf{NP}$ $\mathbf{H}(s)$ e una data istanza I di un problema P è difficile. In particolare, quan-
 1443 do si cerca di sviluppare un algoritmo $\mathbf{H}_L(s)$ per decidere un linguaggio
 1444 $L \in \mathbf{NP}$, non è noto se δ_m^2 sia polinomiale o esponenziale per $\mathbf{H}_L(s)$. Stabi-
 1445 lirlo significherebbe dimostrare $\mathbf{P} \neq \mathbf{NP}$ oppure $\mathbf{P} = \mathbf{NP}$ in questo modello
 1446 computazionale.

²⁰ Abbiamo ommesso il termine $\hbar$ perché è solo una costante moltiplicativa e lo poniamo uguale a 1. Questo è un approccio spesso adottato in meccanica quantistica per semplificare i passaggi aritmetici.

1447 *Relazione con il modello basato su porte*

1448 Studiando le relazioni tra CAQ e calcolo a porte quantistiche, è stato dimo-
 1449 strato che una versione universale del CAQ può simulare il modello basato
 1450 su porte con al più un rallentamento polinomiale. È stato inoltre dimo-
 1451 strato che vale anche il contrario: il modello a porte quantistiche può simulare
 1452 qualsiasi calcolo CAQ in modo efficiente. Questi due teoremi mostrano che
 1453 il calcolo adiabatico quantistico e il calcolo a porte quantistiche sono modelli
 1454 computazionali polinomialmente equivalenti. La classe di problemi risolta
 1455 efficientemente dagli algoritmi CAQ è quindi **BQP**.

*Equivalenza tra
CAQ e modello a
porte quantistiche*

1456 2.3.3 Quantum annealing

1457 Il *quantum annealing* (QA) è un approccio basato su euristiche che consente
 1458 di risolvere problemi di ottimizzazione combinatoria. È un sistema ispirato
 1459 al *simulated annealing*. Il simulated annealing è un algoritmo stocastico per
 1460 trovare l'energia minima di un sistema ispirato dalla termodinamica.

1461 Esiste una stretta relazione tra CAQ e QA; in particolare, è stabilito che
 1462 un algoritmo per il quantum annealing può essere implementato nativamen-
 1463 te con l'insieme di istruzioni fornito dal CAQ. I due approcci sono così
 1464 strettamente connessi che i due termini sono spesso usati in modo inter-
 1465 cambiabile, e in questo lavoro possiamo considerarli anche come sinonimi.
 1466 Esistono tuttavia differenze, non tanto nell'algoritmo, quanto nel modello
 1467 computazionale di riferimento:

- 1468 • Un algoritmo CAQ è progettato per essere implementato nel modello
 1469 CAQ universale. Questo è il modello computazionale discusso nella se-
 1470 zione precedente ed è polinomialmente equivalente al modello basato
 1471 su porte quantistiche;
- 1472 • Un algoritmo QA mira a risolvere un certo tipo di problemi: i problemi
 1473 di ottimizzazione combinatoria. Ciò consente di imporre restrizioni
 1474 sull'Hamiltoniano finale e di rilassare le condizioni del modello CAQ.

1475 In un algoritmo QA, l'Hamiltoniano finale $\mathbf{H}_f$ rappresenta uno stato fonda-
 1476 mentale classico, ovvero senza sovrapposizione al momento della lettura dei
 1477 risultati. La condizione che viene rilassata rispetto al CAQ, invece, è l'adia-
 1478 baticità dell'evoluzione del sistema fisico. In particolare, non è più richiesto
 1479 che l'intera computazione avvenga nello stato fondamentale di $\mathbf{H}(t)$.

*Differenza tra
CAQ e QA*

1480 *Annealing*

1481 Esaminiamo ora le idee alla base dei sistemi basati su annealing, partendo
 1482 dal simulated annealing sviluppato per i modelli computazionali classici,
 1483 per poi vedere come il quantum annealing si inserisce nella discussione e
 1484 quali vantaggi quest'ultimo ha rispetto al modello classico.

1485 **SIMULATED ANNEALING:** Questo approccio stocastico consente di trovare
 1486 il minimo di una funzione multivariata. Il metodo è ispirato alla termodina-
 1487 mica e mira a trovare la configurazione più stabile del sistema, ovvero quella

con energia minima. La funzione da minimizzare rappresenta quindi l'energia del sistema ed è generalmente composta da molti minimi locali separati da barriere energetiche. Per superare queste barriere, viene fornita energia al sistema. Nel simulated annealing, questa energia è energia termica.

Il sistema fisico parte da una certa configurazione (non ottimale) e con una certa temperatura (simulata) T . Il sistema cambia configurazione secondo una certa euristica e nella nuova configurazione avrà una nuova energia; sia ΔE la differenza tra l'energia nella seconda configurazione e l'energia nella prima. Se $\Delta E < 0$ la nuova configurazione viene accettata. Se $\Delta E > 0$ la transizione non viene immediatamente scartata ma viene accettata con una certa probabilità uguale al fattore di Boltzmann $e^{-\frac{\Delta E}{k_b T}}$ dove k_b è la costante di Boltzmann. Osserviamo che più alta è la temperatura, più alta è la probabilità di superare barriere energetiche elevate. Man mano che la temperatura diminuisce —assumendo quindi di avvicinarci al minimo globale— le transizioni verso configurazioni a energia più alta diventano più rare. Quando raggiungiamo $T = 0$, l'energia totale è data solo dall'energia interna del sistema, e, al termine dell'annealing, il sistema si trova nello stato di energia interna minima.

Il tempo τ necessario per evolvere da $T = T_i$ a $T = 0$ è chiamato *tempo di rilassamento termico del sistema*. Se il sistema è *ergodico*²¹, il tempo di rilassamento cresce linearmente con la dimensione del problema (mentre le possibili configurazioni crescono esponenzialmente); in questo caso il simulated annealing è un algoritmo efficiente per trovare lo stato fondamentale del sistema. Tuttavia, se il sistema non è ergodico, qualsiasi fluttuazione termica a temperatura finita richiederebbe un tempo di rilassamento infinito per portare il sistema alla configurazione di energia minima; ovvero, se il sistema non è ergodico $\tau \rightarrow \infty$.

Limiti del
simulated annealing

QUANTUM ANNEALING: Il quantum annealing, come la sua versione classica, può essere visto come un'euristica che si muove all'interno di uno spazio di soluzioni cercando quella migliore. Il modo in cui lo spazio viene esplorato da un algoritmo QA è descritto da una catena di Markov parametrizzata. Per passare da una configurazione all'altra, non viene più utilizzata la temperatura T ma un *coefficiente di campo trasversale* Γ (chiamato anche *coefficiente di tunneling*). Come T , anche Γ parte da un valore elevato e decresce fino a zero seguendo una certa pianificazione.

Il quantum annealing introduce un *campo trasversale* o *Hamiltoniano disordinante* $\mathbf{H}_d$ che non commuta con $\mathbf{H}_f$ ed è scalato con il coefficiente di tunneling. L'Hamiltoniano che descrive l'evoluzione temporale del sistema è quindi:

$$\mathbf{H}(t) = \mathbf{H}_f + \Gamma(t)\mathbf{H}_d. \quad (2.8)$$

L'Hamiltoniano disordinante $\mathbf{H}_d$ introduce energia cinetica nel processo di annealing sotto forma di fluttuazioni quantistiche dello spazio delle soluzioni. Diminuire $\Gamma(t)$ avvicina il sistema a $\mathbf{H}_f$ smorzando le fluttuazioni quantistiche. La Figura 2.10 mostra graficamente il vantaggio del QA rispetto alla controparte classica: valori elevati di $\Gamma(t)$ consentono al calcolo di sfuggire ai minimi locali attraverso il *tunneling* attraverso le colline invece

²¹ Per maggiori informazioni sui sistemi ergodici vedere [7].

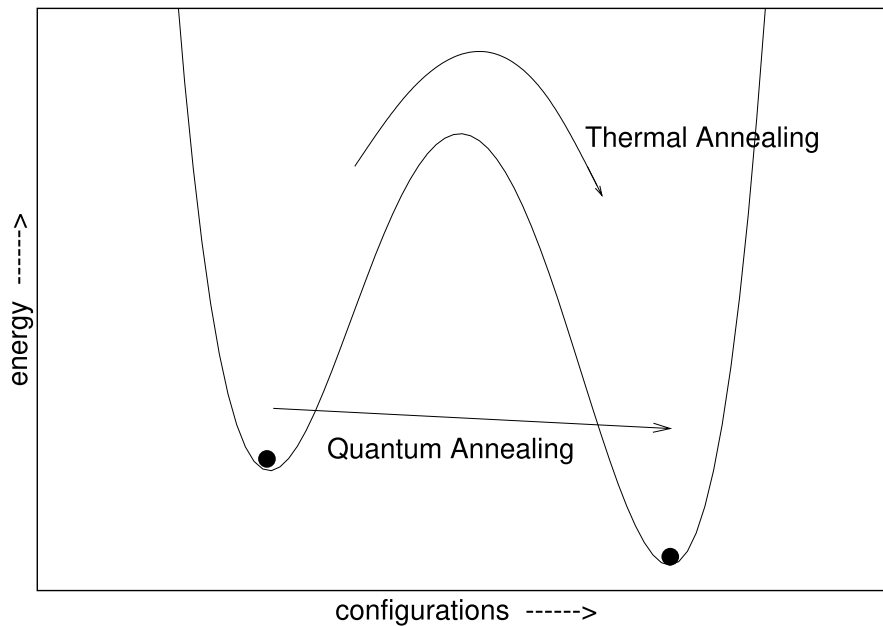


Figura 2.10: Simulated annealing e quantum annealing [5]

1533 di scalarle gradualmente. Ciò consente all'algoritmo di muoversi più ve-
 1534 locemente e più lontano nello spazio delle soluzioni nelle fasi iniziali del
 1535 processo.

1536 Un altro vantaggio del QA è che, durante l'evoluzione, il sistema non si
 1537 trova in un singolo stato, ma in una sovrapposizione di stati la cui probabilità
 1538 è definita da $\mathbf{H}(t)$. Questa idea viene adottata anche dagli algoritmi che
 1539 implementano il QA su architetture classiche. In questo caso si parla di
 1540 *simulated quantum annealing* e l'obiettivo è cercare di catturare la proprietà di
 1541 sovrapposizione rappresentando molti stati in parallelo²².

*Simulated quantum
annealing*

1542 **Formulazione di un problema per l'annealing**

1543 Vediamo come formulare un problema in modo che possa essere risolto con
 1544 un *quantum annealer* (una macchina capace di implementare il QA). Otter-
 1545 remo due formulazioni equivalenti che descrivono problemi risolvibili con
 1546 quantum annealing, simulated annealing o simulated quantum annealing.
 1547 Per ora ci concentriamo sulla descrizione teorica della formulazione. Per
 1548 un'implementazione pratica compatibile con un'architettura QA, si veda la
 1549 Sezione 4.3.

1550 **MODELLO DI ISING:** è un modello matematico che descrive le transizioni
 1551 di fase e altre proprietà di un sistema fisico che evolve nel tempo. Il proble-
 1552 ma è definito da n particelle disposte su un grafo $G = (V, E)$ che può essere
 1553 descritto come una griglia d -dimensionale. Ogni particella può trovarsi in
 1554 uno stato di spin $\in \{+1, -1\}$. Chiamiamo *configurazione di spin* $s = s_1, \dots, s_n$
 1555 un'assegnazione di spin alle particelle.

1556 Due tipi di forze possono agire sul sistema descritto:

*Forze sulle
particelle*

²² Una discussione più approfondita sul simulated quantum annealing va oltre lo scopo di questo lavoro, ma può essere trovata in [8].

- 1557 • forze esterne h_i applicate alle singole particelle;
- 1558 • forze di interazione J_{ij} tra vicini nella griglia (due vertici v_i e v_j collegati
- 1559 da un arco e_{ij}). Se non ci sono archi tra v_i e v_j assumiamo $J_{ij} = 0$.

1560 L'energia di una data configurazione è definita da

$$\mathbf{H}(s) = \sum_i h_i \cdot s_i + \sum_{j < i} J_{ij} \cdot s_i s_j . \quad (2.9)$$

1561 Vediamo come codificare questa funzione energia in un quantum annealer.
 1562 Considerando una collezione di n qubit $Q = q_1, \dots, q_n$ disposti nella griglia
 1563 definita dal grafo $G = (V, E)$, lo stato di Q è $|\Phi\rangle = |\phi_1, \dots, \phi_n\rangle$. Possiamo
 1564 tradurre questa energia nell'Hamiltoniano del problema H_f :

$$\mathbf{H}_f = \sum_{i \in V} h_i \cdot Z q_i + \sum_{(i,j) \in E} J_{ij} \cdot Z q_i Z q_j . \quad (2.10)$$

1565 Dove $Z q_i$ indica l'applicazione della porta di Pauli Z al qubit i . Questo è
 1566 un Hamiltoniano valido per un algoritmo QA e un quantum annealer è in
 1567 grado di trovare la soluzione di energia minima.

1568 **PROBLEMA QUBO:** Il modello di Ising è definito su spin $s_1 \in \{+1, -1\}$.
 1569 Una formulazione equivalente, ma più vicina a quella usuale in informatica,
 1570 è il problema di *Ottimizzazione Binaria Quadratica Non Vincolata* (forma QU-
 1571 BO). In questa formulazione consideriamo n variabili booleane $x_i \in \{0, 1\}$
 1572 e, considerando una matrice triangolare superiore Q_{ij} di pesi di dimensione
 1573 $n \times n$, vogliamo minimizzare la funzione

$$f_{\text{QUBO}}(x) = \sum_{i,j} Q_{ij} \cdot x_i x_j . \quad (2.11)$$

Dalla forma QUBO alla forma Ising e viceversa
 1574 È possibile dimostrare che un problema in forma QUBO ha sempre una
 1575 versione corrispondente nella forma del modello di Ising, e per passare da
 1576 una formulazione all'altra si impone $s_i = 1 - 2x_i$ [9]. È anche possibile
 1577 dimostrare che lo stato fondamentale del problema in forma QUBO è lo
 1578 stesso di quello in forma Ising a meno di un offset finito δ_e

$$\min_{s \in \{-1, +1\}} \sum_i h_i \cdot s_i + \sum_{j < i} J_{ij} \cdot s_i s_j = \min_{x \in \{0, 1\}} \sum_{i,j} Q_{ij} \cdot x_i x_j + \delta_e . \quad (2.12)$$

1579 2.4 CONCLUSIONE

1580 In questo lavoro abbiamo giustificato l'interesse per il calcolo quantistico mo-
 1581 strando come questo nuovo paradigma si posizioni all'interno dello studio
 1582 della complessità computazionale e nella definizione delle classi di problemi
 1583 computazionali.

1584 Abbiamo iniziato definendo i modelli teorici che descrivono la computa-
 1585 zione classica, come la macchina di Turing e il modello a circuiti. Partendo
 1586 da questi, abbiamo classificato i problemi computazionali in base alle risorse
 1587 necessarie per risolverli.

1588 Abbiamo quindi introdotto teoricamente il calcolo quantistico (attraverso
1589 i circuiti quantistici) e mostrato come le classi di problemi computaziona-
1590 li cambiano quando si considera questo nuovo paradigma. In particolare,
1591 abbiamo osservato che dimostrare i vantaggi computazionali del paradig-
1592 ma quantistico porterebbe a risultati significativi anche nella computazione
1593 classica, rendendo possibile dimostrare che $P \neq PSPACE$.

1594 Il capitolo prosegue poi definendo il paradigma di calcolo quantistico ba-
1595 sato su porte, questa volta in modo meno teorico, osservando come un qubit
1596 è definito in un modello computazionale e cosa significa applicare una porta
1597 quantistica a un sistema di qubit per modificarne lo stato. La sezione sulle
1598 porte quantistiche si conclude con l'algoritmo di Deutsch, che mostra una
1599 prima accelerazione (anche se solo polinomiale) dei computer quantistici
1600 rispetto alle architetture classiche.

1601 Abbiamo concluso il capitolo esaminando un altro approccio al calcolo
1602 quantistico, questa volta un approccio analogico: il CAQ. Abbiamo descritto
1603 in cosa consiste un algoritmo CAQ e quali condizioni devono essere soddi-
1604 sfatte affinché questo algoritmo ci permetta di ottenere la soluzione al nostro
1605 problema. Infine, siamo passati dal modello CAQ universale agli algoritmi
1606 QA, perdendo generalità ma rilassando i vincoli imposti alla computazio-
1607 ne. Abbiamo derivato due forme per formulare un problema adatto al QA
1608 (forma QUBO e forma Ising) e ne abbiamo evidenziato l'equivalenza.

1609 Queste due formulazioni che abbiamo ottenuto sono precisamente l'obiet-
1610 tivo del resto del nostro lavoro: trasformare la conoscenza strutturata, e una
1611 query su questa conoscenza, in una formulazione adatta al QA.

1613 In this chapter we explain what kind of knowledge base (KB) an ontology
 1614 is, how to build an ontology, and why this knowledge representation is
 1615 important. To clarify and demonstrate why ontologies are useful, we present
 1616 an example of a foundational ontology¹, briefly discussing its utility.

1617 The rest of the chapter is about reasoning on ontologies; we discuss the
 1618 semantics of the formal language used to represent knowledge, what we
 1619 mean when saying interpretation of a KB, and the complexity of finding an
 1620 interpretation.

1621 3.1 KNOWLEDGE BASE

1622 In the field of information technologies, an ontology is a structured represen-
 1623 tation of knowledge about a certain domain of interest; however, the study
 1624 of knowledge began much before informatics. To better understand what an
 1625 ontology is, let's start with the philosophical definition and then point out
 1626 the differences between this vision and the IT one.

1627 3.1.1 Ontology in philosophy

1628 Ontology was born as a branch of philosophy. In this context it is the
 1629 science of what is, of the kinds and structures of objects, properties, events,
 1630 processes, and relations in every area of reality [10].

1631 The goal of an ontology is to give a definitive and exhaustive classifica-
 1632 tion of entities in all spheres of being. With the term "definitive" we mean
 1633 that an ontology should answer questions such as: "What classes of entities
 1634 are needed for a complete description and explanation of all the goings-on
 1635 in the universe?" With the term "exhaustive", instead, we mean that all
 1636 types of entities and relations between these entities are included in our
 1637 ontology [10].

1638 3.1.2 Ontology in computer science

1639 Thanks to the advent of the internet and the development of bigger and
 1640 bigger software used by bigger and bigger groups of users, what we might
 1641 call the Tower of Babel problem emerged. Each research group develops
 1642 its KB with terms and concepts shared and accepted only inside the group.
 1643 For example, different databases may use identical labels but with diffe-

1 a very general template that can be used as base (foundation) to build an ontology about a specific domain (more about that in Section 3.2.2).

rent meanings, and the same meaning may be expressed with different names [10].

To address the incompatibility problem between software, databases, and research groups, ontologies have become an important research topic in computer science where the goal is to define standards for data exchange, information integration, and interoperability [11].

In this field the term ontology gains a new meaning:

Definition 1. *Ontologies represent a formal and explicit specification of a shared conceptualization [12].*

In this definition the keywords are:

CONCEPTUALIZATION: an ontology creates an abstract model identifying and defining only the relevant concepts;

EXPLICIT: the types of concepts and constraints on their use are explicitly defined;

FORMAL: an ontology should be machine-readable;

SHARED: the knowledge represented by the ontology has to be accepted by a group of people, ideally by everyone.

When we use an ontology to represent knowledge we are describing a graph where entities are bound together through relationships, and classified according to a formal description of the world [13]. Knowledge bases expressed with this formalism are divided into two components [14]:

T-BOX: stores a set of universally quantified assertions (inclusion assertions) stating general properties of concepts and roles;

A-BOX: contains assertions on individual objects (instance assertions).

The T-Box is the conceptualization of the world, while the A-Box is a certain instance of the world we have modelled in the T-Box.

We can see some similarities between an ontology and a database: the T-Box can be seen as the Entity-Relation schema and the A-Box as the set of all entries of the database. There is, however, a logical difference between the world represented by an ontology and the world represented by a database.

Databases make the *closed world assumption*: everything that is not present in the database is automatically false; for example, if a person does not appear in a bank registry it means that that person is not a client of the bank.

Ontologies, on the other hand, make the *open world assumption* [15], which means, for example, that we can assert that a certain person is a parent even if we have not specified any son or daughter.

3.1.3 OWL Language

OWL 2 Web Ontology Language is an ontology language for the Semantic Web with a formally defined meaning [16]. Thanks to OWL we can model classes and relations between classes (T-Box) and individuals with their specific properties and relations between individuals (A-Box).

OWL is a declarative language and defines the state of the world in a logical way. In particular, we are interested in OWL DL where the meaning of ontologies expressed with this language is assigned in a Description Logic style. OWL DL is, therefore, decidable and an appropriate tool (so-called reasoner) can then be used to infer further information about that state of the world [16].

OWL DL

OWL per se does not specify any syntax, it states only what can or cannot be expressed in an ontology. The World Wide Web Consortium (W3C) standardizes various syntaxes, some inspired by functional languages, others more suitable for storing on web pages. The only syntax that must be implemented by all tools to be compliant with the OWL standard is the RDF/XML syntax [16] (examples of this syntax are provided in Section 3.2.1).

3.1.4 Importance of ontologies

Ontologies are important in various fields, from interoperability to machine learning.

In the Semantic Web context, ontologies are a main vehicle for data integration, sharing, and discovery [17]. Different research groups can use the same ontology to share a unified vocabulary that helps build common knowledge and helps to better integrate the results obtained by each group.

In a more commercial scenario, an ontology can be used as a translation layer between different databases or software that are built by different teams and use different vocabularies.

In the machine learning field an ontology could be used to support the sharing and reuse of formally represented knowledge among neuro-symbolic AI systems [12].

3.2 EXAMPLE ONTOLOGIES

To help understanding the structure of ontologies and to show a practical example of ontology, we present two ontologies: a simple ontology about family relationships and DOLCE, a foundational ontology.

3.2.1 Simple ontology

This simple ontology about parental relationships shows the basic structure of an ontology, helping to understand the graph structure of these KBs and the relations between the T-Box and A-Box.

In Figure 3.1, we can see the T-Box of the ontology: this structure specifies what our domain of interest is, and what entities could possibly populate our world. This ontology is about people, so the main class/concept is **People**: this class has several subclasses that represent parents, children, and married people. We can assert that a person belongs to the married class without specifying the partner (open world assumption) but we can also infer that a person belongs to the parents class because we have created a relationship of type `parent_of` between that person and another person.

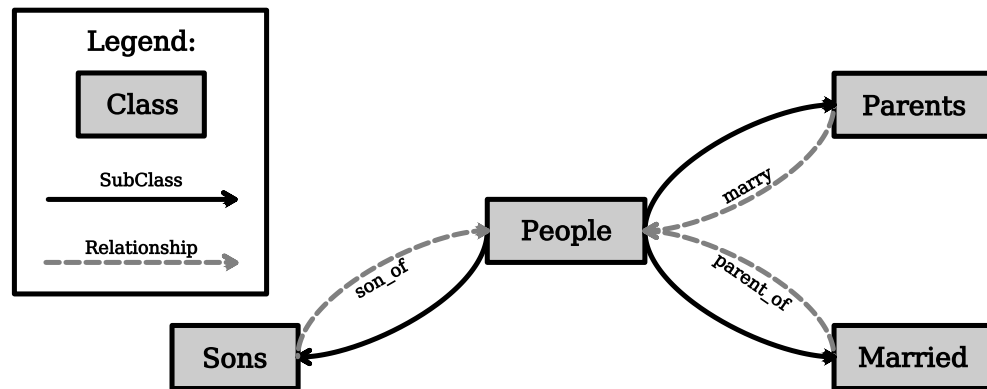


Figura 3.1: Graph for T-Box

1727 OWL allows us to express rules to infer when a member of a class belongs
 1728 also to another class. The following code shows (in the RDF/XML syntax)
 1729 the definition of the class `Parent`²:

```

1 <owl:Class rdf:about="http://people#Parent">
2   <owl:equivalentClass>
3     <owl:Restriction>
4       <owl:onProperty rdf:resource="http://people#parent_of"/>
5       <owl:someValuesFrom rdf:resource="http://people#Person"/>
6     </owl:Restriction>
7   </owl:equivalentClass>
8   <rdfs:subClassOf rdf:resource="http://people#Person"/>
9 </owl:Class>

```

Listing 3.1: Definition of parents

1730 At line 8 we can see that `Parent` is a subclass of `People`, and at lines 4 and
 1731 5 it is specified that a parent is a person that is `parent_of` another person.
 1732 Other interesting properties that we can specify thanks to OWL constructs
 1733 are:

- 1734 • relation `marry` is symmetric;
- 1735 • relation `parent_of` is the inverse of `son_of`;
- 1736 • we can specify a domain and a range for relations.

1737 Now we can populate the ontolo-
 1738 gy by adding individuals and rela-
 1739 tions between individuals. For this
 1740 small example we take inspiration
 1741 from the Simpson family, and in the
 1742 family tree (Figure 3.2 on the right)
 1743 we can see the small portion of the
 1744 family represented. To show what
 1745 we mean by open world assump-
 1746 tion we have asserted that Jackie is
 1747 a married person even if in our representation there is no husband.

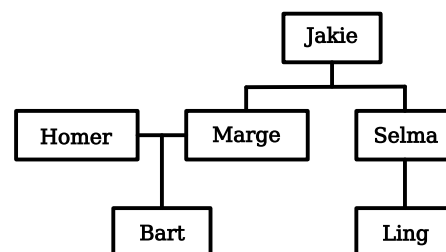


Figura 3.2: Simpson family tree

² The complete code of the ontology can be seen at <https://github.com/DavideCamino/Thesis/blob/main/Code/Ontology/people.rdf>.

Our ontology covers a small domain, the types of entities that populate our model are very limited; the next example shows the commitment of engineering an ontology to represent virtually anything in the universe.

3.2.2 DOLCE ontology

DOLCE (Descriptive Ontology for Linguistic and Cognitive Engineering) is a top-level (foundational) ontology [18]; this means that this ontology describes fundamental aspects of reality and should be used as a base for constructing an ontology about a particular domain of interest. For this reason DOLCE defines only the T-Box; the user will then expand the T-Box with specific classes and relations of interest, and lastly will populate the A-Box.

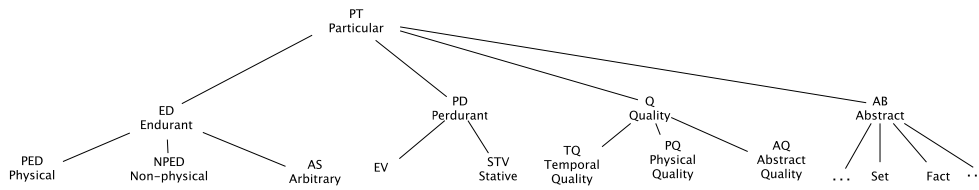


Figure 3.3: First layer of DOLCE taxonomy [18]

STRUCTURE OF DOLCE: in DOLCE we can model the modification of objects during time; for this reason DOLCE distinguishes between *endurants* and *perdurants*. Endurants may acquire or lose properties and parts through time, perdurants are fixed in time [18]. With a simplification we can see endurants as the physical entities that are modified by the passing of time (like objects, animals, and people) and perdurants as events that, once they have passed, cannot be changed anymore (like a tennis match or a conference).

*Endurants and
perdurants*

The relation connecting endurants and perdurants is called participation. A physical entity can be in time by participating in a perdurant, and perdurants happen in time by having endurants as participants [18].

Another important aspect of DOLCE is the way we attribute a property to an entity; this is done by using qualities, which are what can be perceived and measured. To do so we can assert that a certain entity has a specific quality and then, when it is possible, quantify that quality.

IMPORTANCE OF DOLCE: foundational ontologies can be useful in several fields, from conceptual modeling to natural language processing. DOLCE, today, is used in a variety of domains where it provides the general categories and relations needed to give a coherent view of reality [18].

3.3 REASONING ON ONTOLOGY

In Section 3.1.3 we have introduced the standard language to encode an ontology; in order to infer new information, starting from the one we already have, we need to better specify the semantics of OWL DL.

3.3.1 $\mathcal{SROIQ}$ DL

The semantics of OWL DL extends the semantics of the description logic (DL) $\mathcal{SROIQ}$ to provide support for datatypes and punning [19]. For constructs available both in OWL DL and in $\mathcal{SROIQ}$ the semantics correspond exactly.

Description logics allow the modeling of the domain of interest with three kinds of entity: concepts, roles, and individual names. These entities correspond to unary predicates, binary predicates, and constants in first-order logic [15]. From the point of view of ontology and OWL, concepts are classes, roles are relationships, and individual names are the individuals that can belong to one or more classes.

Constructors of $\mathcal{SROIQ}$ DL $\mathcal{SROIQ}$ is one of the most expressive description logics where we have constructors for:

- transitive roles: $\mathcal{S}$
- role inclusions, local reflexivity, universal role, symmetry, asymmetry, role disjointness, reflexivity, and irreflexivity: $\mathcal{R}$;
- nominals: $\mathcal{O}$;
- inverse roles: $\mathcal{I}$;
- qualified number restrictions: $\mathcal{Q}$.

For example, we can construct the ontology shown in Figures 3.1 and 3.2 with a set of assertions like:

`person(selma)            married(jackie)            parent_of(marge, bart)`

Each of these statements is called an axiom and the set of all axioms constitutes our KB.

3.3.2 Interpretation of a knowledge base

An interpretation I consists of a domain Δ^I and an interpretation function $\cdot^I$ that maps:

$$\begin{aligned} \text{concept } A &\rightarrow A^I \subseteq \Delta^I \\ \text{role } R &\rightarrow R^I \subseteq \Delta^I \times \Delta^I \\ \text{named individual } a &\rightarrow a^I \in \Delta^I \end{aligned}$$

Meaning of interpretation In other words I assigns a fixed meaning to all entities in the KB [15]. By having a fixed meaning, we can say if an axiom α holds in I or not; in the first case we say that I satisfies α and we write $I \models \alpha$.

If all axioms in an ontology are satisfied by I we say that I is a *model* of the ontology. An ontology is consistent if it accepts at least one model.

A reasoner should at least be capable of saying if an ontology is consistent, but we are also interested in querying knowledge to retrieve new information.

QUERY INTERPRETATION: Considering a KB K , a query q consists of axiom templates where $\mathcal{SROIQ}$ axioms are composed of concept names, role names, and individual names, but also of concept variables, role variables, and individual variables. A solution for the query is an interpretation μ that allows us to rewrite all variables in q with names; we denote with $\mu(q)$ the result of the substitution.

The evaluation of q over K is a set of solutions μ with: [20]

$$\{ \mu | K \cup \mu(q) \text{ is a } \mathcal{SROIQ} \text{ knowledge base and } K \models \mu(q) \}$$

In other words μ binds all free variables of q to names present in K [20].

A naive approach to find the solution to a query is to simply test for each possible solution mapping μ , if $K \models \mu(q)$; however, in the worst case, the number of mappings that have to be tested is exponential in the number of variables in the query [20].

3.3.3 Complexity of reasoning

Since presenting an actual algorithm for reasoning on ontologies is out of the scope of this work, we only give some hints about the reasons for the complexity and then present the theoretical results that prove the problem of reasoning on ontology is at least PSpace-hard.

It is easy to convince oneself that the more axioms there are in an ontology, the fewer interpretations exist that satisfy all axioms. On the other hand, if an ontology has fewer models, the more axioms hold in all of them and the more logical consequences follow from the ontology.

In other words the semantics of description logics are *monotonic*: the more knowledge we embed in an ontology, the more results it returns [15]. A more formal view is given in [21], where two *sources of complexity* are identified:

- OR-branching: the presence of disjunctive constructors;
- AND-branching: the presence of qualified existential and universal quantifiers.

The AND-branching is responsible for the exponential size of a single interpretation, and the OR-branching is responsible for the exponential number of different interpretations.

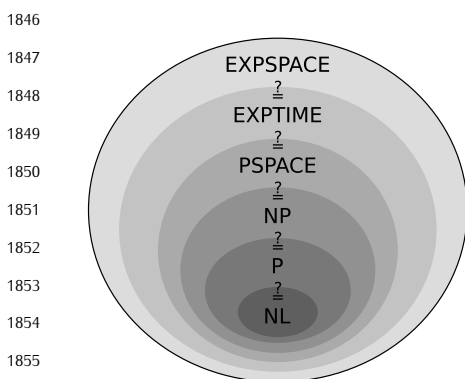


Figura 3.4: Complexity classes

To discuss the complexity of reasoning we take into account the description logic $\mathcal{ALC}$; this DL is a restriction of $\mathcal{SROIQ}$ [15], so its complexity is a lower bound for $\mathcal{SROIQ}$. It is possible to prove the **PSPACE**-hardness of satisfiability in $\mathcal{ALC}$ [21], therefore also $\mathcal{SROIQ}$ DL is at least **PSPACE**-hard. This result shows that, unless **PSPACE** = **P**, the exponential time complexity of any algorithm that makes inference on an ontology cannot be improved.

$$\mathcal{SROIQ} \in \mathbf{PSPACE}$$

1858 For those interested in some numerical examples to better understand
1859 what this class of complexity means in a real context, [22] presents the
1860 reasoner HermiT and evaluates its performance on some real ontologies.

1861 3.4 CONCLUSIONS

1862 In this chapter we have explained what an ontology is and we have motiva-
1863 ted the interest in this field.

1864 We showed that an ontology can be useful both in academic research and
1865 in industry. Moreover, being a formal and machine-readable structure, it
1866 can be queried and used to perform logically demonstrable reasoning whose
1867 subject is precisely the knowledge represented within the ontology.

1868 We have shown both theoretically and with examples what can be expres-
1869 sed in an ontology and what cannot. We have formally defined what the
1870 interpretation of a KB is and showed what a query and its results are.

1871 Lastly, we have characterized the complexity of reasoning on ontologies.
1872 This complexity is what motivated us to search for other paradigms to infer
1873 new knowledge starting from an ontology. In the next chapters we will build
1874 the tools necessary to achieve this goal.

1875

1876

Parte II

TOOLS

4

CONFIGURAZIONE DELL'AMBIENTE

In questo capitolo descriviamo l'ambiente, le librerie e gli strumenti che utilizziamo per eseguire i nostri test.

Nelle sezioni seguenti installiamo gli SDK per sviluppare e interagire con i computer quantistici di IBM e D-Wave. Presentiamo inoltre altri due strumenti utili per scrivere facilmente problemi di ottimizzazione.

4.1 AMBIENTE PYTHON

Il linguaggio usato per interfacciarsi con i computer quantistici è solitamente Python. In questa sezione creiamo un ambiente virtuale in Python per comunicare con il computer quantistico IBM e il computer quantistico D-Wave.

Per i nostri test gestiamo gli ambienti Python con `conda`. Iniziamo creando l'ambiente virtuale denominato `quantum` e attivandolo con:

```
$ conda create --name quantum python=3.12 pip
$ conda activate quantum
```

Per i nostri test e per seguire i vari esempi presentati sia da IBM che da D-Wave, è utile anche poter eseguire un notebook Jupyter. Possiamo installare Jupyter con:

```
$ pip install jupyter
```

4.2 IBM QISKIT

Per programmare un'architettura basata su porte e per accedere ai computer quantistici IBM utilizziamo lo stack software *Qiskit*. Il nome *Qiskit* è un termine generale che si riferisce a una raccolta di software per eseguire programmi su computer quantistici.

I componenti principali sono *Qiskit SDK* e *Qiskit Runtime*. Il primo è completamente open source e consente allo sviluppatore di definire il proprio circuito; il secondo è un servizio cloud per eseguire calcoli quantistici sui computer quantistici IBM.

4.2.1 Hello World

Seguendo la documentazione IBM¹ possiamo installare l'SDK e il Runtime con:

¹ <https://quantum.cloud.ibm.com/docs/en/guides/install-qiskit>

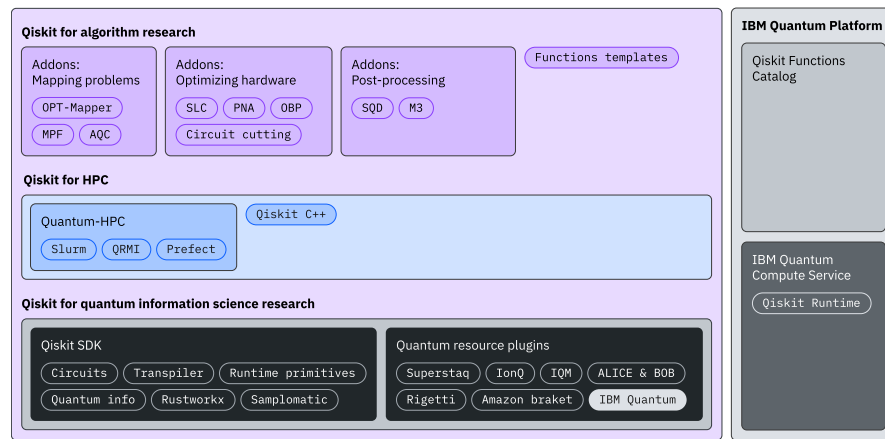


Figura 4.1: Stack software Qiskit

1907

```
$ pip install qiskit matplotlib
↪ qiskit[visualization]
$ pip install qiskit-ibm-runtime
$ pip install qiskit-aer
```

1908 L'ultimo comando installa Aer, che è un simulatore ad alte prestazioni per
 1909 circuiti quantistici scritti in Qiskit. Aer include modelli di rumore realistici,
 1910 e lo utilizzeremo in seguito per testare il nostro circuito.

1911 A volte lo stack Qiskit soffre di incompatibilità tra i vari componenti soft-
 1912 ware che compongono l'ambiente. Al momento della stesura, gli ultimi pac-
 1913 chetti sembrano funzionare senza alcun problema. Per i nostri test utilizze-
 1914 remo qiskit: 2.2.3, qiskit-ibm-runtime: 0.43.1 e qiskit-aer: 0.17.2.

1915 Se la configurazione ha successo siamo ora in grado di eseguire un piccolo
 1916 test per costruire uno stato di Bell (due qubit entangled)². Il codice seguente
 1917 assembla le porte, mostra il circuito finale e usa un sampler per simulare su
 1918 CPU il risultato di 1024 esecuzioni del programma.

```
1 from qiskit import QuantumCircuit
2 from qiskit.primitives import StatevectorSampler
3
4 qc = QuantumCircuit(2)
5 qc.h(0)
6 qc.cx(0, 1)
7 qc.measure_all()
8
9 sampler = StatevectorSampler()
10 result = sampler.run([qc], shots=1024).result()
11 print(result[0].data.meas.get_counts())
12 qc.draw("mpl")
```

Listing 4.1: Costruzione dello stato di Bell

1919 4.2.2 Transpilazione

1920 Ogni Unità di Elaborazione Quantistica (QPU) ha una topologia specifica.
 1921 Dobbiamo riscrivere il nostro circuito quantistico per adattarlo alla topo-

² Otterremo lo stato $|\Psi^+\rangle$ (Equazione 2.5)

1922 gia del dispositivo selezionato su cui vogliamo eseguire il nostro program-
 1923 ma. Questa fase di riscrittura, seguita da un'ottimizzazione, è chiamata
 1924 transpilazione.

1925 Considerando un hardware finto (in modo da non necessitare di una chia-
 1926 ve API) possiamo transpilare il circuito quantistico `qc`, dal codice precedente,
 1927 per adattarlo alla topologia di una QPU specifica. Il Listato 4.2 implementa
 1928 la transpilazione, e la Figura 4.2 mostra i circuiti generati: (a) è il circuito
 1929 che abbiamo definito nel Listato 4.1 e (b) è la versione transpilata in cui la
 1930 porta di Hadamard viene sostituita per adattarsi alla topologia effettiva della
 1931 QPU.

```

1 from qiskit_ibm_runtime.fake_provider import FakeWashingtonV2
2 from qiskit.transpiler import generate_preset_pass_manager
3
4 backend = FakeWashingtonV2()
5 pass_manager = generate_preset_pass_manager(backend=backend)
6
7 transpiled = pass_manager.run(qc)
8 transpiled.draw("mpl")

```

Listing 4.2: Transpilazione

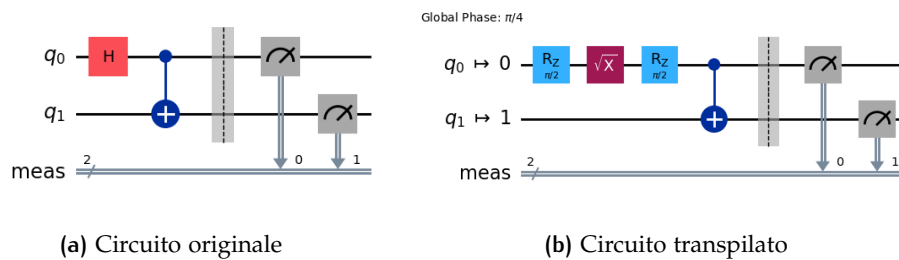


Figura 4.2: Esempio di transpilazione

1932 4.2.3 Esecuzione

1933 Per testare il nostro circuito transpilato utilizziamo Aer, che ci permette di si-
 1934 mulare anche il rumore dell'hardware quantistico reale. Il Listato 4.3 mostra
 come simulare l'esecuzione.

```

1 from qiskit_aer.primitives import SamplerV2
2
3 sampler = SamplerV2.from_backend(backend)
4 job = sampler.run([transpiled], shots=1024)
5 result = job.result()
6 print(f"counts Bell circuit: {result[0].data.meas.get_counts()}")

```

Listing 4.3: Esecuzione simulata

1935
 1936 Se osserviamo i risultati dell'esecuzione possiamo notare che alcune rispo-
 1937 ste presentano qubit non entangled; questo è causato dal rumore (simulato)
 1938 del dispositivo quantistico. Un tipico output dell'esecuzione potrebbe essere:

*Risultati con
rumore simulato*

```

1939 > counts Bell circuit: {'00': 504, '11': 503, '01': 10,
  ↪ '10': 7}

```

Dove gli stati 01 e 10 non dovrebbero essere presenti in un'esecuzione ideale senza errori.

4.3 D-WAVE OCEAN

Per definire un problema di ottimizzazione che può essere risolto su un computer quantistico D-Wave utilizziamo lo stack software Ocean. Ocean ci permette anche di interagire con l'hardware D-Wave, inviare un problema e simulare l'esecuzione su una CPU classica.

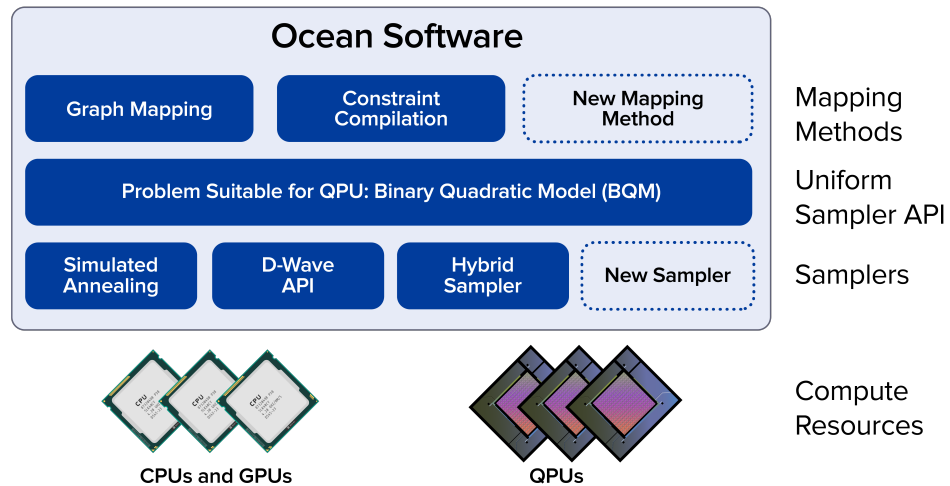


Figura 4.3: Stack software Ocean

Tutti gli strumenti che implementano i passi necessari per risolvere il proprio problema su una CPU, un computer quantistico D-Wave o un solver ibrido quantistico-classico possono essere installati con:

```
$ pip install dwave-ocean-sdk
```

Dopo l'installazione, eseguendo il comando `dwave setup` verrà avviato un prompt interattivo che ci guida attraverso una configurazione completa. Durante la configurazione è anche possibile aggiungere un token API o connettersi all'account D-Wave per importare direttamente una chiave per utilizzare l'hardware quantistico.

4.3.1 Hello World

Per presentare un semplice programma di ottimizzazione consideriamo il problema della copertura minima dei vertici (MVC). Dato un grafo $G = (V, E)$, il problema chiede di trovare un sottoinsieme $V' \subseteq V$ tale che, per ogni arco $\{u, v\} \in E$, almeno uno tra u e v appartenga a V' , e il numero di nodi in V' ($|V'|$) sia il più basso possibile.

MVC come problema di Ising La riduzione da MVC a una formulazione Ising è ben nota. La funzione di costo che vogliamo minimizzare può essere espressa da:

$$\text{cost} = \sum_{i=1}^{|V|} v_i + 2 \cdot \sum_{\{i,j\} \in E} (1 - v_i - v_j + v_i v_j)$$

dove $v_i \in \{-1, 1\}$ e $v_i = 1$ significa che $v_i \in V'$, altrimenti $v_i = -1$.

Come tutti i problemi in forma Ising possiamo esprimere il costo come una matrice simmetrica, quindi la nostra funzione diventa

$$\text{cost} = \mathbf{v}^T \times \mathbf{M} \times \mathbf{v}$$

dove $\mathbf{v}$ è il vettore contenente le variabili binarie v_i .

La figura mostra un grafo di esempio (4.4a) e la corrispondente matrice (4.4b) che esprime la funzione di costo.

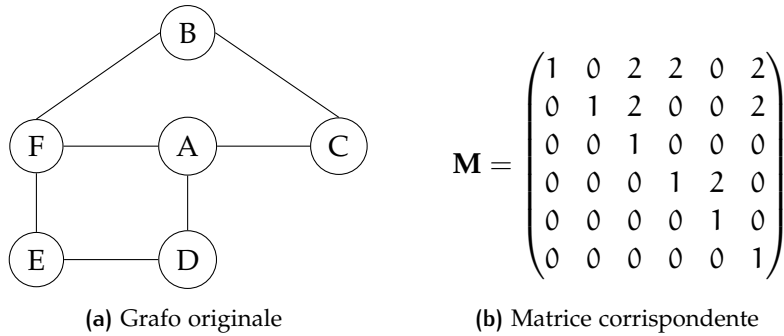


Figura 4.4: Formulazione Ising

Il codice seguente presenta una possibile implementazione del modello Ising descritto sopra. Abbiamo definito due dizionari per memorizzare i coefficienti della matrice. L'ultima riga di codice trova dieci possibili risposte al problema usando la funzione di simulated annealing implementata da D-Wave.

*Simulazione
dell'esecuzione*

```
1 from dwave.samplers import SimulatedAnnealingSampler
2 linear = {'A': 1, 'B': 1, 'C': 1, 'D': 1, 'E': 1, 'F': 1}
3 quadratic = {('B', 'C'): 2, ('B', 'F'): 2, ('C', 'A'): 2, ('D',
4   ↪ 'A'): 2, ('E', 'D'): 2, ('E', 'F'): 2, ('F', 'A'): 2}
5 sampler = SimulatedAnnealingSampler()
6 result = sampler.sample_ising(linear, quadratic, num_reads=10)
```

Listing 4.4: Esempio Ising

Se stampiamo i risultati con `print(result.aggregate())` possiamo osservare qualcosa di simile a questo:

```
1777 A B C D E F energy num_oc.
      0 -1 -1 +1 +1 -1 +1 -14.0      6
      1 +1 +1 -1 -1 +1 -1 -14.0      4
      ['SPIN', 2 rows, 10 samples, 6
      ↪ variables]
```

I due risultati diversi rappresentano le due risposte corrette alla nostra particolare istanza del problema MVC.

4.4 PYQUBO E QUBOVERT

Nel Listato 4.4 abbiamo costruito manualmente la matrice che rappresenta la funzione che vogliamo minimizzare. Può essere utile disporre di strumenti

che ci permettano di lavorare a un livello più alto, definendo funzioni di costo come le Equazioni 2.9 e 2.11 che abbiamo definito nella sezione sul quantum annealing (Sezione 2.3.3).

Considerando di nuovo il problema MVC, la funzione obiettivo tende a minimizzare il numero di nodi nel nostro sottoinsieme, mentre la penalità aumenta il costo se tralasciamo alcuni archi. Questa interpretazione ci permette di trasformare il modello Ising nel modello QUBO più familiare —dal punto di vista di un informatico—, dove tutte le variabili $x_i \in \{0, 1\}$. Vediamo come PyQUBO e qubover ci aiutano in questo compito.

4.4.1 PyQUBO

Leggendo dalla documentazione sul sito di PyQUBO³, PyQUBO ci permette di creare QUBO o modelli Ising da espressioni matematiche flessibili in modo semplice. Alcune delle caratteristiche di PyQUBO sono:

- basato su Python (backend in C++);
- completamente integrato con Ocean SDK;
- validazione automatica dei vincoli;
- segnaposto per la regolazione dei parametri.

Possiamo installare PyQUBO con `$ pip install pyqubo` e riscrivere il nostro problema MVC definendo l'Hamiltoniano che vogliamo minimizzare.

```

1  from pyqubo import Binary, Placeholder, Constraint
2  from dwave.samplers import SimulatedAnnealingSampler
3
4  A, B, C, D, E, F = Binary('A'), Binary('B'), Binary('C'),
   ↪ Binary('D'), Binary('E'), Binary('F')
5
6  H_objective = (A + B + C + D + E + F)
7  H_penalty = Constraint(((1 - A - C + A*C) + \
8  (1 - A - D + A*D) + \
9  (1 - A - F + A*F) + \
10 (1 - B - C + B*C) + \
11 (1 - B - F + B*F) + \
12 (1 - D - E + D*E) + \
13 (1 - E - F + E*F)) , label='cnstr0')
14
15 L = Placeholder('L')
16 H = H_objective + L*H_penalty
17 H_internal = H.compile()
18 bqm = H_internal.to_bqm(feed_dict={'L': 2})
19
20 sampler = SimulatedAnnealingSampler()
21 result = sampler.sample(bqm, num_reads=10)

```

Listing 4.5: Riscrittura del MVC con pyQUBO

2002

³ <https://pyqubo.readthedocs.io/en/latest/>

Il Listato 4.5 presenta una possibile reimplementazione del Listato 4.4, in cui vediamo anche come PyQUBO si interfaccia con Ocean SDK (riga 17), e come creare (righe 14–16) e istanziare (riga 17) un Hamiltoniano parametrico.

4.4.2 qubovert

Come scritto nella documentazione⁴, qubovert è il pacchetto unico per formulare, simulare e risolvere problemi in forma booleana e di spin. Usando la nostra nomenclatura, forma booleana e forma di spin sono rispettivamente la forma QUBO e la forma Ising.

Qubovert ci permette di definire vari tipi di problemi di ottimizzazione che possono essere risolti per forza bruta, con il simulated annealing di qubovert, o con il solver di D-Wave. I modelli definiti in qubovert sono:

- QUBO: Ottimizzazione Booleana Quadratica Non Vincolata;
- QUSO: Ottimizzazione di Spin Quadratica Non Vincolata (modello Ising);
- PUBO: Ottimizzazione Booleana Polinomiale Non Vincolata;
- PUSO: Ottimizzazione di Spin Polinomiale Non Vincolata;
- PCBO: Ottimizzazione Booleana Polinomiale Vincolata;
- PCSO: Ottimizzazione di Spin Polinomiale Vincolata.

Oltre ai modelli generici, qubovert dispone di una libreria di famosi problemi NP-completi mappati nelle forme QUBO e Ising.

```

1 from qubovert import boolean_var
2 from dwave.samplers import SimulatedAnnealingSampler
3
4 A, B, C, D, E, F = boolean_var('A'), boolean_var('B'),
   ↪ boolean_var('C'), boolean_var('D'), boolean_var('E'),
   ↪ boolean_var('F')
5
6 model = A + B + C + D + E + F
7 model.add_constraint_OR(A, C, lam=2)
8 model.add_constraint_OR(A, D, lam=2)
9 model.add_constraint_OR(A, F, lam=2)
10 model.add_constraint_OR(B, C, lam=2)
11 model.add_constraint_OR(B, F, lam=2)
12 model.add_constraint_OR(D, E, lam=2)
13 model.add_constraint_OR(E, F, lam=2)
14
15 qubo = model.to_qubo()
16 dwave_qubo = qubo.Q
17
18 sampler = SimulatedAnnealingSampler()
19 result = sampler.sample_qubo(dwave_qubo, num_reads=10)

```

Listing 4.6: Riscrittura del MVC con qubovert

⁴ <https://qubovert.readthedocs.io/en/latest/index.html>

Il Listato 4.6 mostra una possibile implementazione del problema MVC usando gli strumenti forniti da qubovert. Qubovert ci permette di esprimere il nostro problema come PCBO; utilizziamo questa formulazione per esprimere i vincoli in modo più naturale. Nel nostro esempio ci assicuriamo che ogni arco sia coperto semplicemente imponendo che almeno uno dei nodi collegati dall'arco sia presente nella soluzione. Questo vincolo viene ripetuto per ogni arco del grafo (righe 7–13). I vincoli sono codificati nella funzione di costo usando una funzione di penalità moltiplicata per uno scalare: il *moltiplicatore di Lagrange*. Per specificare il moltiplicatore di Lagrange usiamo la parola chiave `lam`.

Qubovert, come PyQUBO, può interfacciarsi con Ocean SDK, trasformando un problema PCBO in un problema QUBO (riga 15) e poi riscrivendolo nel formato accettato dal solver (o sampler) di D-Wave.

4.5 CONCLUSIONE

In questo capitolo abbiamo configurato un ambiente per eseguire i nostri futuri esperimenti e test. Abbiamo anche mostrato alcuni piccoli esempi per presentare le caratteristiche principali e testare gli strumenti che utilizzeremo nel nostro lavoro.

Seguire questa configurazione consente a chiunque di ricreare esattamente la stessa configurazione che utilizziamo, evitando (per quanto ne sappiamo e testiamo) incompatibilità tra i pacchetti Python.

2045 QA-Prolog è uno strumento che consente di scrivere un programma in un
 2046 linguaggio di programmazione logica ed eseguirlo su un quantum annealer.
 2047 QA-Prolog recupera anche i risultati restituiti dal quantum annealer e li
 2048 presenta in modo naturale e comprensibile.

2049 In questo capitolo introduciamo il progetto e forniamo alcuni riferimenti
 2050 ad altri lavori correlati. Descriviamo in modo esteso la pipeline di trasfor-
 2051 mazioni che parte da un codice Prolog e termina con un Hamiltoniano H_f ,
 2052 come abbiamo descritto nella Sezione 2.3.3. Presentiamo poi il nostro con-
 2053 tributo al progetto, discutendo le modifiche apportate al codice originale di
 2054 QA-Prolog per ripristinare la compatibilità con il framework moderno usato
 2055 per interfacciarsi con il quantum annealer D-Wave e per supportare l'ultima
 2056 versione della libreria utilizzata nel progetto.

2057 Il capitolo si conclude con una guida all'installazione che garantisce un
 2058 ambiente funzionante e riproducibile per eseguire esperimenti, sia per que-
 2059 sto capitolo che per i successivi.

2060 5.1 IL PROGETTO

2061 QA-Prolog è un progetto sviluppato da Scott Pakin¹ nel periodo 2017 – 2019.
 2062 Parte dalla domanda: “È possibile esprimere la programmazione logica con
 2063 vincoli nella forma accettata dall'hardware di quantum annealing?” [1].

2064 La speranza è che, anche se oggi viviamo nell'era *NISQ*² del calcolo quan-
 2065 tistico, i quantum annealer siano più facilmente scalabili rispetto ai compu-
 2066 ter quantistici basati su porte [23], e QA-Prolog potrebbe migliorare l'esecu-
 2067 zione dei programmi Prolog sostituendo il backtracking con un annealing
 2068 completamente parallelo verso uno stato soluzione [24].

2069 5.1.1 Motivazione

2070 Come abbiamo mostrato nel Capitolo 2, programmare un computer quanti-
 2071 stico non è un compito semplice. Esprimiamo il nostro algoritmo in modo
 2072 molto vicino al basso livello.

2073 Su un quantum annealer dobbiamo definire una funzione di costo, senza
 2074 vincoli (che devono essere trasformati in una funzione di penalità). Anche se
 2075 esistono librerie che ci permettono di esprimere queste funzioni in modo più
 2076 semplice, dobbiamo almeno trovare una rappresentazione QUBO del nostro
 2077 problema.

2078 Ancora più difficile è la situazione sui computer quantistici basati su por-
 2079 te. Il programmatore deve costruire un circuito quantistico porta per porta,

*Porte quantistiche
e FPGA*

¹ Los Alamos National Laboratory: pakin@lanl.gov.

² Noisy intermediate-scale quantum computing.

un approccio simile a quello adottato con le FPGA (Field Programmable Gate Arrays) [25]. Possiamo infatti vedere una forte analogia riassunta nella Tabella 5.1.

FPGA	Computer a porte quantistiche
blocchi logici programmabili che implementano funzioni logiche	porte quantistiche
routing programmabile che collega le funzioni logiche	possibilità di definire l'ordine delle porte
blocchi I/O collegati alla logica	qubit di input e qubit di output che possono essere misurati

Tabella 5.1: Parallelismo tra FPGA e porte quantistiche

Le FPGA sono componenti a cui la maggior parte degli informatici non è abituata e probabilmente al di fuori dell'interesse di un programmatore. Allo stesso modo, la difficoltà di programmare un computer quantistico potrebbe essere un significativo deterrente nell'attrarre nuovi ricercatori nel campo.

Oggi sono disponibili alcune "porte ad alto livello" che racchiudono più porte di basso livello in schemi utili. Esistono anche, sia per i computer quantistici basati su porte che per i quantum annealer, alcuni template di problemi noti che necessitano solo di una messa a punto per essere utili per un problema specifico.

Nonostante questi tipi di astrazioni, programmare un computer quantistico è difficile e molto vicino al linguaggio macchina.

5.1.2 Prolog

Possiamo vedere QA-Prolog come un compilatore da Prolog a H_f , dove lo stato fondamentale di H_f è la soluzione del nostro programma Prolog. Prima di iniziare con il processo di compilazione, è utile comprendere le caratteristiche principali di Prolog, perché non è un linguaggio di programmazione imperativo come C o Java, ma *dichiarativo* come SQL; inoltre, Prolog è un *linguaggio di programmazione logica*. Ciò significa che il nucleo della programmazione non è dire al computer cosa fare, ma dirgli cosa è vero e chiedergli di cercare di trarre conclusioni [26].

Discutere in dettaglio le caratteristiche di Prolog e dei linguaggi di programmazione logica va oltre lo scopo di questo lavoro. Ulteriori informazioni su Prolog possono essere trovate in *Programming in PROLOG* [26] e *Learn Prolog Now!* [27]. Per chi fosse interessato ai linguaggi di programmazione logica, si consigliano *Logic for problem solving* [28] e *Introduction to logic programming* [29].

In Prolog non specifichiamo passo per passo un algoritmo che risolve il nostro problema, descriviamo invece la relazione formale tra gli oggetti del nostro problema e quali relazioni devono essere vere nella nostra soluzione [26].

Programmare in Prolog consiste in:

Linguaggio di programmazione logica

Elementi di programma Prolog

- elencare *fatti* sugli oggetti e le relazioni tra gli oggetti;
- specificare *regole* per derivare nuovi fatti da quelli già asseriti;
- porre domande (*query*) sugli oggetti e le loro relazioni.

Da queste caratteristiche possiamo capire cosa significa “dichiarativo”: il programma è un elenco di affermazioni sul nostro problema (il nostro dominio di interesse); la relazione tra un programma Prolog e un’ontologia è molto stretta. In Prolog codifichiamo una KB³ fatta di fatti e regole; nel Capitolo 6 possiamo vedere un esempio completo di riscrittura da un’ontologia OWL a una KB Prolog.

PREDICATI ARITMETICI: Per comprendere meglio alcune caratteristiche di QA-Prolog che differiscono dal Prolog di base, spieghiamo qui come asserire l’uguaglianza tra argomenti quando si fa riferimento a interi. I predicati offerti da Prolog sono:

- `+Expr1 == +Expr2`: vero se l’espressione `Expr1` valuta a un numero uguale a `Expr2`;
- `-Number is +Expr`: vero quando `Number` è il valore a cui `Expr` valuta.

I segni prima degli argomenti sono *indicatori di modalità degli argomenti*. Il segno `+` specifica che l’argomento seguente deve essere istanziato, `-` indica che l’argomento è un output [30]. Ciò significa che non possiamo avere variabili quando si usa il predicato `==` perché tutto deve essere noto al momento della valutazione. Usando `is` l’unica variabile può essere il valore di `Number`; `Expr`, d’altra parte, deve essere noto.

*Indicatori di
modalità degli argomenti*

ESEMPIO: Presentiamo ora un programma Prolog di base per mostrare la sintassi e l’utilizzo delle query. Codifichiamo una KB su una festa, descriviamo due persone, Yolanda e Mia, asserendo cosa stanno facendo, e fornendo alcune regole per derivare altre informazioni sul nostro piccolo scenario.

```

1  sings(mia).
2  listens2Music(yolanda).
3  party.
4
5  dance(yolanda):- listens2Music(yolanda).
6  happy(yolanda):- dance(yolanda).
7  happy(mia):- sings(mia).
8
9  smile(X) :- happy(X).
```

Listing 5.1: KB di base

Nel Listato 5.1, adattato da [27], le righe da 1 a 3 sono fatti: stiamo asserendo che Yolanda sta ascoltando musica, Mia sta cantando e c’è una festa. Le altre righe sono regole: possiamo identificare le regole dal segno `:-` che divide la *testa* della regola a sinistra dal *corpo* a destra. La testa di una regola è vera se il corpo è vero.

*Interpretazione delle
righe Prolog*

³ Base di Conoscenza

Ad esempio, la regola alla riga 5 può essere letta come: “Se Yolanda sta ascoltando musica, allora Yolanda sta ballando”. La riga 9 mostra l’uso di una variabile: le variabili iniziano con una lettera maiuscola e sono segnaposto per le informazioni. Possiamo leggere questa regola come “Se qualcuno è felice, sorride”.

Possiamo interrogare la nostra KB, chiedendo ad esempio se Yolanda è felice. In SWI-Prolog [31] possiamo interagire con l’interprete, e la query che valutiamo è `?- happy(yolanda).` (il punto fermo fa parte della sintassi e indica all’interprete che la query è completa). Prolog risponderà `yes.`; questo perché Yolanda sta ascoltando musica, e se sta ascoltando musica balla, e se balla è felice.

Per ragionamento analogo dovrebbe essere chiaro perché il risultato di `?- smile(X).` è `X = mia; X = yolanda.`, dove `;` significa disgiunzione logica: `0`.

5.1.3 Caratteristiche di QA-Prolog

QA-Prolog non supporta tutte le funzionalità di Prolog, ma abbastanza da rendere possibile la programmazione logica di base [1].

QA-Prolog supporta atomi e interi positivi ma non numeri in virgola mobile, stringhe o liste. Supporta operazioni aritmetiche e relazionali, e le regole possono fare riferimento ad altre regole ma non in modo ricorsivo. QA-Prolog supporta l’unificazione, il backtracking e i predicati composti da più clausole [1].

QA-Prolog supporta anche alcune funzionalità non presenti nella versione base di Prolog. In particolare, è possibile eseguire operazioni su variabili anche prima che siano istanziate [1]; ciò significa che QA-Prolog è più potente nella manipolazione di variabili libere. Possiamo comprendere questa caratteristica guardando al Paragrafo 5.1.2 dove abbiamo mostrato che le espressioni aritmetiche possono avere al massimo una variabile libera —predicato `is`— nell’operando sinistro.

Cenni di computazione
reversibile

In QA-Prolog possiamo avere variabili sia a sinistra che a destra del predicato⁴, in questo modo possiamo passare l’input alla formula per ottenere il risultato, ma anche assegnare un valore al risultato per ricavare l’input. Possiamo già vedere che QA-Prolog consente una computazione reversibile: dall’input all’output ma anche dall’output all’input.

5.1.4 Lavori correlati

Esistono altri tentativi di sviluppare un linguaggio di programmazione ad alto livello per computer quantistici, sia per il modello a porte quantistiche che per i quantum annealer.

Quantum Prolog [35] dimostra che è possibile esprimere l’equivalente di una versione pura di Prolog su relazioni finite in termini di un modello di calcolo quantistico discreto. Questo lavoro si rivolge ai computer quantistici a porte e si concentra sull’equivalenza matematica della programmazione re-

⁴ In Prolog è possibile ottenere lo stesso risultato con librerie come CLP(FD) [32, 33] e CLP(Z) [34]

lazionale e del calcolo quantistico discreto sul campo dei booleani [1]. Quantum Prolog, tuttavia, rimane un lavoro teorico con l'obiettivo di dimostrare alcune proprietà matematiche, non esiste un'implementazione e quindi non è adatto per eseguire esperimenti.

C to D-Wave [36] riutilizza la pipeline che discuteremo, sostituendo il passo iniziale. L'articolo affronta la difficoltà di programmare i quantum annealer presentando un framework di compilazione che compila un sottoinsieme di codice C in istruzioni macchina quantistiche da eseguire su un quantum annealer.

5.2 PIPELINE

Siamo ora pronti a descrivere la pipeline di trasformazioni che ci porta da un programma Prolog a un Hamiltoniano H_f .

La catena di trasformazioni è mostrata nella Figura 5.1, dove l'ultimo passo (in arancione) è il quantum annealer capace di trovare lo stato fondamentale di H_f . In viola possiamo vedere i vari formati di file lungo la pipeline, e in giallo il software che esegue la riscrittura. La maggior parte di questo software è realizzata ad hoc per la pipeline di QA-Prolog; vedremo invece che Yosys è uno strumento utilizzato nella progettazione di circuiti digitali.

Da un punto di vista ad alto livello, la pipeline riscrive una KB iniziale espressa in Prolog in oggetti diversi. Il significato logico delle entità che costruiamo durante la pipeline è:

1. Programma Prolog (KB);
2. Circuito digitale ad alto livello in Verilog;
3. Circuito digitale a basso livello in formato EDIF;
4. Hamiltoniano simbolico nel formalismo qasm;
5. Hamiltoniano fisico.
6. Esecuzione su un'unità di elaborazione quantistica

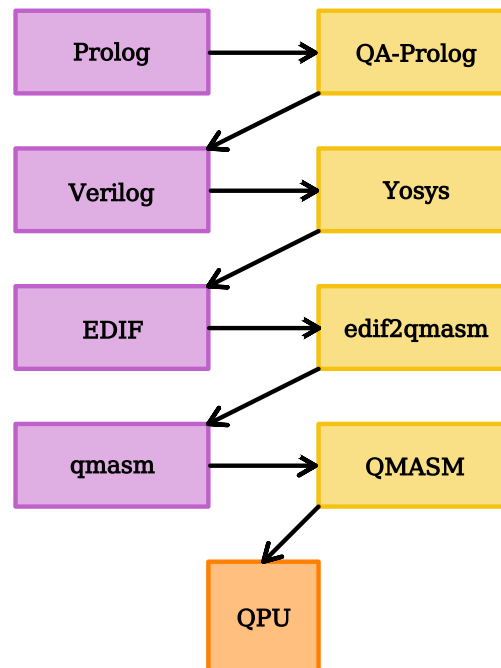


Figura 5.1: Pipeline di QA-Prolog

Iniziamo ad analizzare la pipeline dall'ultimo passo, quello più vicino alla QPU.

2227 5.2.1 QMASM

2228 QMASM è un *quantum macro assembler* [37]; elabora un Hamiltoniano sim-
 2229 bolico e assembla un Hamiltoniano fisico che può essere incorporato su un
 2230 quantum annealer D-Wave. È stato sviluppato in Python da Scott Pakin
 2231 con l'obiettivo di colmare una lacuna negli strumenti per la creazione di
 2232 programmi D-Wave. QMASM è un livello di astrazione che consente al pro-
 2233 grammatore di non doversi preoccupare di impostare manualmente i pesi
 2234 dei punti specifici e le intensità degli accoppiatori sulla topologia fisica.

2235 Questo software può essere considerato un assembler nel senso che map-
 2236 pa una rappresentazione simbolica delle operazioni (linguaggio assembly)
 QMASM come macro 2237 nel linguaggio macchina. QMASM non è solo un assembler, ma estende
 assembler 2238 le sue funzionalità includendo le macro: blocchi di linguaggio assembly
 2239 parametrizzati e nominati che un programma può istanziare più volte [37].

2240 **CARATTERISTICHE:** QMASM fornisce una serie di funzionalità per sem-
 2241 plicare la programmazione D-Wave a basso livello [37]. Inoltre, possiamo
 2242 anche utilizzare QMASM per assemblare un Hamiltoniano fisico che possia-
 2243 mo in seguito manipolare o risolvere usando metodi classici o altri sistemi
 2244 quantistici.

2245 Alcune delle funzionalità più utili e interessanti di QMASM sono:

- 2246 • i qubit sono referenziati simbolicamente, non numericamente, sia nel
 2247 codice sorgente che quando QMASM riporta i risultati dell'esecuzione;
- 2248 • i qubit possono essere fissati a `true` o `false`;
- 2249 • i pattern di qubit possono essere incapsulati in macro e istanziati ripe-
 2250 tutamente;
- 2251 • gruppi di macro possono essere incapsulati in librerie e riutilizzati in
 2252 più programmi;
- 2253 • QMASM può escludere automaticamente dai risultati le soluzioni note
 2254 come errate e mostra solo i qubit "interessanti".

2255 Grazie a questo insieme di funzionalità, QMASM è già un utile livello di
 2256 astrazione che semplifica lo sviluppo di programmi destinati a un annealer,
 2257 classico o quantistico.

Esempio di 2258 **ESEMPIO:** Consideriamo un esempio che mostra il potenziale di QMASM
 problema di soddisfacibilità 2259 e che è utile anche per i passi successivi della pipeline. Il problema della sod-
 2260 disfacibilità è ben noto come problema NP-completo [38], quindi è un buon
 2261 candidato per il nostro esempio. Prendiamo in considerazione la semplice
 2262 formula:

$$y = x_1 \wedge \neg(x_2 \vee x_3) \quad (5.1)$$

2263 QMASM ci permette di definire una macro per ogni operatore logico ne-
 2264 cessario a rappresentare la formula e poi assemblare la formula finale chia-
 2265 mando queste macro. Nei Listati 5.2 chiamiamo A e B i qubit di input e Y il

qubit di output. I pesi sono specificati come problema Ising (Sezione 2.3.3), che è il formato predefinito per i file sorgente QMASM⁵.

Ci sono diversi dettagli interessanti su queste macro:

- il simbolo `#` identifica una riga di commento non elaborata;
- il simbolo `$` è usato per contrassegnare un qubit come ausiliario: a meno che non sia esplicitamente richiesto dal programmatore, i risultati intermedi non vengono riportati nelle soluzioni;
- grazie alla direttiva `!assert` possiamo informare QMASM sui vincoli della soluzione. Questa direttiva non modifica i pesi, ma consente al programmatore di escludere dalle soluzioni quelle sicuramente errate.

<pre> 1 # Y = A AND B 2 !begin_macro AND 3 !assert \$Y=\$A&\$B 4 \$A -0.5 5 \$B -0.5 6 \$Y 1 7 8 \$A \$B 0.5 9 \$A \$Y -1 10 \$B \$Y -1 11 !end_macro AND </pre> (a) Porta AND	<pre> 1 # Y = NOT A 2 !begin_macro NOT 3 !assert \$Y=!\$A 4 \$A \$Y 1.0 5 !end_macro NOT </pre> (b) Porta NOT	<pre> 1 # Y = A OR B 2 !begin_macro OR 3 !assert \$Y=\$A \$B 4 \$A 0.5 5 \$B 0.5 6 \$Y -1 7 8 \$A \$B 0.5 9 \$A \$Y -1 10 \$B \$Y -1 11 !end_macro OR </pre> (c) Porta OR
--	--	---

Listing 5.2: Definizione della T-Box

Rispetto a quanto fatto nel Listato 4.4, definire i pesi in QMASM è molto più semplice e il risultato è più leggibile.

È possibile verificare, per ogni macro nei Listati 5.2, che data una configurazione dei qubit di input, il valore di Y che minimizza l'energia corrisponde all'output della porta logica che stiamo modellando. Ad esempio, il polinomio esplicito che descrive l'operatore AND (dove abbiamo rimosso il simbolo $\$$ per semplicità di lettura) è:

Interpretazione della macro come polinomio

$$H = -\frac{1}{2} \cdot (A + B) + Y + \frac{1}{2} \cdot (AB) - (AY + BY)$$

Dove $A, B, Y \in \{-1, 1\}$ perché stiamo descrivendo un problema Ising.

I Listati 5.2 mostrano anche l'uso della direttiva `!assert` (riga 3 di ogni listato): considerando la porta OR, imponiamo che in una soluzione valida $Y = A \vee B$, le soluzioni che non verificano questo vincolo vengono automaticamente scartate da QMASM.

Assemblare le macro

Possiamo salvare queste macro in un file denominato `gates.qasm` e usarlo per risolvere il nostro problema. Per calcolare la formula $y = x_1 \wedge \neg(x_2 \vee x_3)$ utilizzeremo alcuni risultati intermedi: iniziamo con $x_4 = (x_2 \vee x_3)$, poi applichiamo la negazione $x_5 = \neg x_4$, e infine calcoliamo il risultato come $y = x_1 \wedge x_5$.

⁵ come specificato in <https://github.com/lanl/qasm/wiki/File-format>.

```

1  # Solve circuit-satisfiability problem.
2
3  !include <gates>
4
5  !use_macro OR x2_or_x3
6      x2_or_x3.$A = x2
7      x2_or_x3.$B = x3
8      x2_or_x3.$Y = $x4
9
10 !use_macro NOT not_x4
11     not_x4.$A = $x4
12     not_x4.$Y = $x5
13
14 !use_macro AND x1_and_x5
15     x1_and_x5.$A = x1
16     x1_and_x5.$B = $x5
17     x1_and_x5.$Y = y

```

Listing 5.3: Soddisfacibilità del circuito

Il codice QMASM che implementa questa procedura è riportato nel Listato 5.3. Questo esempio mostra come usare le macro: istanziamo una macro e le diamo un nome con `!use_macro <nome_macro> <nome_istanza>` (ad esempio riga 5), e poi istanziamo i qubit definiti nella macro con i nostri qubit effettivi. Ad esempio, considerando `!use_macro OR` alla riga 5, possiamo vedere che usiamo x_2 e x_3 come input e un qubit ausiliario x_4 come output. Un altro dettaglio da sottolineare è la direttiva `!include` (riga 3), che importa tutte le porte usate in questo file sorgente.

Eseguire un "programma"
QMASM

Infine, possiamo interrogare il quantum annealer (o in questo caso un solver classico) per trovare una soluzione al nostro problema di soddisfacibilità. Per farlo fissiamo la variabile di output y per essere sicuri che nella soluzione il suo valore sarà true. Possiamo interrogare il solver con:

```
qasm --run --pin="y := true" --solver="sim_anneal" circ_sat.qasm
```

dove `circ_sat.qasm` è il nome del file del nostro codice sorgente.

QMASM si interfaccia direttamente con il framework Ocean e può interrogare uno dei solver messi a disposizione da D-Wave. Il recupero delle soluzioni si traduce in una chiamata alle funzioni della libreria di Ocean molto simile a quella mostrata nel Listato 4.4 alla riga 5. QMASM offre anche la possibilità di impostare il tempo di annealing e il numero di letture direttamente dalla riga di comando; in questo caso sono stati usati i valori predefiniti, che sono rispettivamente il tempo di annealing predefinito impostato da D-Wave per il solver specifico e 1000 campioni.

I risultati sono riportati nel Listato 5.4. Qui possiamo vedere che il solver ha trovato correttamente la soluzione, ma solo 642 volte su 1000 campioni predefiniti. Questo è causato dalla natura stocastica dell'annealing, simulato o quantistico. QMASM ha rimosso automaticamente le soluzioni che non rispettano la direttiva `!assert` o che non hanno l'energia minima.

QMASM offre già alcune potenti astrazioni per lavorare con i quantum annealer. Aggiungiamo ora due livelli aggiuntivi che consentono uno stile di programmazione più simile ai paradigmi a cui gli informatici sono abituati,

```
# x1 --> 12
# x2 --> 3
# x3 --> 4
# y --> [True]
Solution #1 (energy = -20.0000, tally = 642):

Variable  Value
-----  -
x1        True
x2        False
x3        False
y         True
```

Listing 5.4: Risultati della soddisfacibilità del circuito

2323 mantenendo al contempo un forte controllo sulle dimensioni delle variabili
2324 e quindi sul numero di qubit utilizzati.

2325 5.2.2 Yosys e edif2qasm

2326 Questi passi della pipeline prendono come input un programma *Verilog*
2327 *HDL* (Hardware Description Language) e lo trasformano in un Hamiltoniano
2328 simbolico.

2329 Aggreghiamo due passi perché Yosys non è uno strumento sviluppato per
2330 questa pipeline ed è usato principalmente per ottimizzare i risultati interme-
2331 di delle trasformazioni. Inoltre, Yosys e edif2qasm lavorano sulla stessa
2332 entità logica: un circuito digitale che, in questi passi della pipeline, viene
2333 convertito in un Hamiltoniano simbolico.

2334 Verilog è un linguaggio di descrizione hardware che offre diversi livelli di *Verilog*
2335 astrazione progettuale. Il livello più alto è *comportamentale* e utilizza costrutti
2336 di programmazione come assegnazioni, condizionali e cicli while. Il livello
2337 più basso è una connessione di porte (*netlist*) [39].

2338 I linguaggi di descrizione hardware non sono qualcosa con cui la maggior
2339 parte dei programmatori ha familiarità, ma offrono alcuni vantaggi quando
2340 si mira a un quantum annealer [40]:

- 2341 • forniscono un controllo preciso sulla larghezza dei bit per non sprecare
2342 alcun qubit;
- 2343 • possono essere compilati in un piccolo insieme di primitive: le porte
2344 che possiamo definire con QMASM.

2345 In questi passi della pipeline iniziamo usando costrutti comportamentali,
2346 poi generiamo automaticamente una netlist come output dei *sintetizzatori*.

2347 Il sintetizzatore utilizzato è Yosys [41], un software libero e open source
2348 per la sintesi di Verilog HDL.

2349 Le due caratteristiche più utili di Yosys sono la possibilità di specifica-
2350 re una *libreria di celle*, l'insieme di porte usate per sintetizzare la netlist, e
2351 l'uso dello strumento esterno *Berkeley ABC* [42] (incorporato in Yosys), che
2352 fornisce ulteriori ottimizzazioni del codice.

2353 Yosys riduce un programma Verilog a una netlist EDIF che usa solo un *Selezione delle porte*
2354 insieme specificato di porte; poi edif2qasm riduce la netlist in un Hamilto-

2355 niano simbolico. Questo Hamiltoniano usa un insieme di macro che defini-
 2356 scono l'energia per ogni tipo di porta che Yosys può usare⁶. Il risultato finale
 2357 è un Hamiltoniano simbolico che può essere usato come input per QMASM.

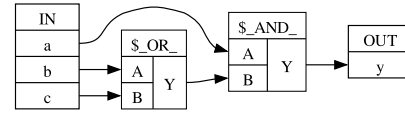
2358 **ESEMPIO:** Considerando di nuovo la semplice formula $y = x_1 \wedge \neg(x_2 \vee$
 2359 $x_3)$ (Equazione 5.1).

```

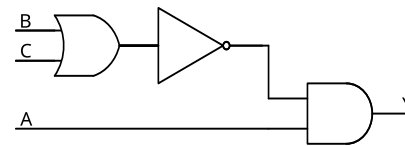
1 module sat (A, B, C, Y);
2   input A, B, C;
3   output Y;
4
5   assign Y = A & ~(B | C);
6 endmodule

```

(a) Codice Verilog



(b) Netlist sintetizzata



(c) Circuito corrispondente

Figura 5.2: Elaborazione con Yosys

2360 La Figura 5.2 mostra una possibile implementazione Verilog della formu-
 2361 la (a), la netlist che Yosys ha prodotto dal codice Verilog (b), e, solo a sco-
 2362 po illustrativo, il corrispondente circuito digitale implementato con porte
 2363 logiche (c).

2364 Ora possiamo passare la netlist ottenuta a edif2qasm per ottenere il
 codice riportato nel Listato 5.5.

```

1 !include <stdcell>
2
3 !begin_macro sat
4   !use_macro AND $id00006
5   !use_macro NOT $id00004
6   !use_macro OR $id00005
7   $id00004.A = $id00005.Y # $or$sat.v:5$1_Y
8   $id00005.A = b
9   $id00005.B = c
10  $id00006.A = a
11  $id00006.B = $id00004.Y # $not$sat.v:5$2_Y
12  $id00006.Y = y
13 !end_macro sat
14
15 !use_macro sat sat

```

Listing 5.5: Hamiltoniano simbolico

2365 Confrontando il Listato 5.5 generato automaticamente e quello 5.3 scritto
 2366 manualmente, possiamo osservare alcune differenze:

- 2368 • nel Listato 5.5 definiamo la nuova macro `sat`: ogni modulo nel pro-
 2369 gramma Prolog viene convertito in una macro;

6 L'insieme effettivo di porte estende le porte logiche *and*, *or* e *not* ed è disponibile su: <https://github.com/lanl/edif2qasm/blob/master/stdcell.qasm>.

- il codice generato è leggermente meno leggibile ma più compatto. Non è necessario generare nuove variabili ausiliarie, ma si usano direttamente come input per le macro l'output di altre macro.

Anche se meno leggibile, possiamo commentare il codice nel Listato 5.5. Alle righe 4, 5 e 6 dichiariamo di aver bisogno delle macro AND, NOT e OR, dando a ogni macro un nome come \$id<numero>⁷. Alla riga 7 assegniamo come input A della macro NOT l'output Y della macro OR. Dati questi indizi, il resto del codice dovrebbe essere chiaro.

5.2.3 QA-Prolog

Questo è l'ultimo passo della pipeline, quello che fornisce il massimo livello di astrazione dall'hardware che trova la soluzione al nostro problema.

QA-Prolog, come edif2qasm, è scritto in GO [1, 40] e compila un programma Prolog in uno Verilog; ogni fatto o regola viene convertito in un modulo. Una volta che QA-Prolog ha compilato il codice sorgente Prolog, inclusa la query che può essere specificata dalla riga di comando, in Verilog, QA-Prolog invoca Yosys, edif2qasm e QMASM, come illustrato nella Figura 5.1 [1].

QMASM esegue il programma dell'utente su un sistema D-Wave o su un simulatore e riporta il valore di ogni simbolo che appare nel file sorgente QMASM. QA-Prolog mappa questi elenchi di booleani in interi e atomi nominati, associa quei valori alle variabili nominate nella query dell'utente e riporta all'utente tutte le variabili e i loro valori esattamente come farebbe un tipico ambiente Prolog [1].

*Risultati in
formato leggibile*

ESEMPIO 1: Il primo esempio che presentiamo mostra una piccola KB che può essere riassunta come “il nemico del mio nemico è mio amico”.

```

1 hates(alice, bob).
2 hates(bob, charlie).
3
4 enemy(X, Y) :- hates(X, Y).
5 enemy(X, Y) :- hates(Y, X).
6
7 friend(X, Y) :-
8     enemy(X, Z),
9     enemy(Z, Y).
```

Listing 5.6: KB il nemico del mio nemico è mio amico

Il codice è riportato nel Listato 5.6. Grazie alla natura dichiarativa di Prolog possiamo comprendere immediatamente il significato della nostra KB: descriviamo le relazioni tra tre persone, assertiamo che se una persona odia un'altra sono nemici (righe 4 e 5), e che se due persone condividono un nemico sono amici (righe 7, 8 e 9).

Possiamo ora interrogare la KB per trovare se ci sono due persone che sono amiche con:

⁷ Il simbolo \$ dice a QMASM che non siamo interessati al valore dei qubit all'interno della macro.

```
QA-Prolog --qasm-args="--solver=tabu" --query='friends(P1, P2).'
↪ --work-dir=~/.work" friends.pl
```

In questo comando possiamo vedere che QA-Prolog ci permette di interrogare la KB con la stessa sintassi che useremmo per interagire con un interprete Prolog. Come previsto, QA-Prolog *lega* le variabili `P1` e `P2` ad Alice e Charlie.

Un altro dettaglio da sottolineare è il parametro `--work-dir=<percorso>`, che specifica la cartella in cui QA-Prolog produce tutti i file e il solver: utilizziamo un *algoritmo di ricerca tabu* perché il simulated annealing implementato in Ocean non è in grado di trovare una soluzione nemmeno in questo piccolo scenario⁸. Diciamo piccolo perché, dallo studio riportato in [43], possiamo osservare che le ontologie del mondo reale definiscono un numero di classi, individui e proprietà⁹ che può essere dell'ordine delle centinaia di migliaia.

Ispezionando la directory di lavoro, possiamo verificare l'output di ogni passo della pipeline. Nell'Appendice A.1 è mostrata la riscrittura di una KB Prolog in un programma Verilog: è possibile osservare in dettaglio come ogni fatto o regola sia stato trasformato. Se ispezioniamo la directory di lavoro possiamo vedere l'output di ogni passo della pipeline; in particolare, per vedere esattamente come la KB viene riscritta in un programma Verilog, nell'Appendice A.1 è presente il risultato della compilazione, ed è possibile osservare in dettaglio come ogni fatto o regola sia stato trasformato.

*Problema di
soddisfacibilità
in QA-Prolog*

ESEMPIO 2: Il secondo esempio conclude la nostra trasformazione del problema di soddisfacibilità (Equazione 5.1); purtroppo, questo non è molto didattico perché il programma Prolog adatto all'elaborazione con QA-Prolog introduce un overhead considerevole. Ciò è dovuto al fatto che in Prolog possiamo semplicemente importare `library(clpb)` [44] e poi formulare la query `?- sat(A * (~(B + C))).`; per usare QA-Prolog, invece, dobbiamo implementare gli operatori logici definendo le loro tavole di verità, per poi assemblare gli operatori nella nostra formula logica. Questo porta all'overhead. Il Listato 5.7 mostra l'implementazione che usiamo nel nostro test.

Possiamo ora eseguire QA-Prolog con la query `sat(A, B, C, true)` per trovare l'assegnazione corretta delle variabili booleane che soddisfa la nostra formula logica. Anche in questo caso siamo costretti a usare l'algoritmo di ricerca tabu. Se ora esploriamo la directory di lavoro di QA-Prolog possiamo vedere che la matrice simbolica generata dalla pipeline è molto più complessa di quella riportata nel Listato 5.5, possiamo interpretare questo come il costo dell'astrazione.

5.2.4 Panoramica

Abbiamo descritto una pipeline di riscritture che, strato dopo strato, permette di astrarre dall'hardware che effettivamente risolve il nostro problema, spostandosi verso un linguaggio, e quindi un modo di ragionare, a livello più alto. Parliamo di riscritture perché l'oggetto che stiamo manipolando rimane logicamente invariato, proprio come è essenziale che le soluzioni che

⁸ Ulteriori considerazioni in merito nel Capitolo 8.

⁹ Che trasformeremo in fatti e regole nel Capitolo 6.

```

1  logicAnd(false, false, false).
2  logicAnd(false, true, false).
3  logicAnd(true, false, false).
4  logicAnd(true, true, true).
5
6  logicNot(false, true).
7  logicNot(true, false).
8
9  logicOr(false, false, false).
10 logicOr(false, true, true).
11 logicOr(true, false, true).
12 logicOr(true, true, true).
13
14 satFormula(A, B, C, Y) :-
15     logicOr(B, C, X),
16     logicNot(X, Z),
17     logicAnd(A, Z, Y).

```

Listing 5.7: Soddisfacibilità in Prolog

2443 stiamo cercando rimangano invariate; il problema e le sue soluzioni sono
 2444 semplicemente espressi in linguaggi diversi man mano che si scende nella
 2445 pipeline.

2446 I vantaggi dell'astrazione sono essenzialmente gli stessi ottenuti passando
 2447 dal linguaggio assembly a un linguaggio di alto livello: maggiore facilità
 2448 di espressione, migliore leggibilità e la possibilità di lavorare con paradigmi
 2449 già familiari. Gli svantaggi sono gli overhead che naturalmente emergono
 2450 durante il processo di traduzione. QA-Prolog è ancora un prototipo, e se
 2451 oggi i compilatori producono codice macchina migliore di quello che un
 2452 programmatore potrebbe scrivere manualmente, è perché un notevole sforzo
 2453 e molti anni di ricerca sono stati investiti per ottenere risultati eccellenti. La
 2454 speranza è che QA-Prolog e i lavori correlati rappresentino un primo passo
 2455 nella stessa direzione nell'ambito del calcolo quantistico.

2456 5.3 AGGIORNAMENTO AL PROGETTO

2457 In questa sezione descriviamo brevemente gli aggiornamenti apportati al
 2458 progetto per ripristinare la compatibilità con il framework Ocean e con le
 2459 altre librerie utilizzate. Forniamo anche una breve guida all'installazione
 2460 degli strumenti per garantire che tutti i pacchetti funzionino correttamente
 2461 e che i nostri esperimenti siano riproducibili.

2462 5.3.1 Ripristino di QMASM

2463 L'ultimo commit a QA-Prolog è stato effettuato nel 2019, e l'ultimo a QMA-
 2464 SM nel 2021. Da allora, i pacchetti e le librerie sono cambiati e la compati-
 2465 bilità con la pipeline di QA-Prolog si è interrotta.

2466 Il componente più fragile della pipeline è QMASM, perché è quello che
 2467 interagisce con framework esterni che possono cambiare. In particolare, lo

*Codice
obsoleto*

sviluppo di Ocean è molto attivo, e alcune funzioni usate in QMASM sono state rimosse, deprecate o spostate in altri pacchetti.

Le incompatibilità riscontrate tra gli import usati in QMASM e le librerie esterne sono:

- nelle librerie `dwave.cloud` e `hybrid`;
- nelle librerie che definiscono i sampler D-Wave (solver classici);
- nella libreria `scipy.stats`.

La documentazione disponibile per Ocean¹⁰ e per `scipy`¹¹ ci ha aiutato a risolvere queste incompatibilità: nella maggior parte dei casi la soluzione consisteva semplicemente nel correggere il nome della libreria.

Un altro piccolo bug, causato da un nome diverso di un solver nell’helper della riga di comando e nel codice effettivo, è stato corretto ripristinando la corrispondenza.

5.3.2 Correzione dell’interazione `edif2qasm-QMASM`

Durante l’esecuzione della pipeline completa abbiamo riscontrato un errore causato da macro non definite nel file sorgente QMASM.

```
1 // Define hates(atom, atom).
2 module \hates/2 (A, B, Valid);
3   ...
4 endmodule
```

(a) Codice Verilog

```
1 # hates/2
2 !begin_macro id00011
3   ...
4 !end_macro id00011
```

(b) Definizioni delle macro

```
1 # enemies/2
2 !begin_macro id00010
3   !use_macro hates/2 $id00032 # hates_PWYwG/2
4   ...
5 !end_macro id00010
```

(c) Utilizzo delle macro

Listing 5.8: Errore di macro non definite

L’errore può essere visto nei Listati 5.8: `edif2qasm` riscrive la netlist generata da (a) nella macro (b); qui `hates/2` è solo un commento, il nome effettivo della macro è `id00011` (specificato alla riga 2). In (c), tuttavia, possiamo vedere che la macro precedentemente definita viene chiamata simbolicamente e non con il suo nome effettivo (riga 3). Questo produce ovviamente un errore: il nome `hates/2` nel file QMASM non ha alcun significato, è solo un commento.

Correzione dei nomi
delle macro

Per risolvere il problema abbiamo aggiunto un passo di preprocessing prima del parsing con QMASM. Durante il preprocessing tutti i nomi simbolici vengono sostituiti con il nome effettivo della macro corrispondente.

¹⁰ Disponibile su <https://docs.dwavequantum.com/en/latest/index.html>.

¹¹ Disponibile su <https://docs.scipy.org/doc/scipy/index.html>.

```

1 with open(file, 'r') as file_in:
2     first = file_in.readline()
3     second = file_in.readline()
4     while(second_row != ""):
5         if first.startswith("#") and second.startswith("!begin_macro"):
6             self.name[first[2:-1]] = second[len("!begin_macro "):-1]
7             first_row = second_row
8             second_row = file_in.readline()
9
10 with open(file, 'r') as file_in:
11     doc = file_in.read()
12     lines = doc.splitlines()
13     for line in lines:
14         for word in line.split():
15             if word in self.name.keys():
16                 line = line.replace(word, self.name[word])
17     self.new_lines.append(line)

```

Listing 5.9: Nucleo del preprocessore

Il codice principale del preprocessore è riportato nel Listato 5.9. Il programma scansiona il file sorgente due volte: durante la prima lettura viene costruito un dizionario Python *nome_simbolico-nome_effettivo*, poi, durante la seconda lettura, tutte le istanze del nome simbolico vengono sostituite con il nome effettivo.

Ora tutti i componenti dovrebbero funzionare correttamente, e possiamo installare tutto il software necessario ed eseguire alcuni esperimenti.

5.3.3 Guida all'installazione

Per avere un ambiente funzionante in cui eseguire i nostri esperimenti, dobbiamo installare tutto il software richiesto dalla pipeline di QA-Prolog. Nel Capitolo 4 abbiamo configurato un ambiente Python con tutti gli strumenti essenziali; per iniziare l'installazione della pipeline di QA-Prolog dobbiamo avere almeno Ocean SDK installato come descritto nella Sezione 4.3.

Il primo componente di cui abbiamo bisogno è QMASM: possiamo installare la versione aggiornata e corretta da <https://github.com/DavideCamino/qasm.git>. La procedura di installazione consiste nei seguenti tre comandi:

```

$ git clone
↪ https://github.com/DavideCamino/qasm.git
$ cd qasm
$ python setup.py install

```

Successivamente abbiamo bisogno di Yosys e GO: Yosys fa parte della pipeline, e GO ci consente di compilare `edif2qasm` e QA-Prolog. Sia Yosys che GO sono disponibili sui loro siti ufficiali <https://yosyshq.net/> e <https://go.dev/>, inoltre sulla maggior parte delle distribuzioni GNU/Linux Yosys e GO possono essere installati direttamente dal gestore di pacchetti della distribuzione.

Il comando `go install` installa gli eseguibili Go nella directory predefinita `$HOME/go/bin`. È utile aggiungere la sottodirectory `bin` al `PATH`. Questo può

2520 essere fatto modificando il file `.bashrc` (o quello corrispondente per la shell
2521 specifica), aggiungendo:

```
2522 PATH=$PATH:$HOME/go/bin  
export PATH
```

2523 Infine possiamo installare `edif2qasm` e `QA-Prolog` con:

```
2524 $ go install  
↪ github.com/lanl/edif2qasm@latest  
$ go install  
↪ github.com/lanl/QA-Prolog@latest
```

2525 5.4 CONCLUSIONE

2526 In questo capitolo abbiamo presentato `QA-Prolog`, una pipeline di riscrittura
2527 che consente la trasformazione di un programma Verilog in un Hamiltoniano
2528 il cui stato fondamentale rappresenta la soluzione del programma originale. Abbiamo
2529 discusso in dettaglio i vari passi della pipeline, mostrando anche come alcune
2530 parti siano state modificate per renderle nuovamente compatibili con i framework
2531 attuali. Infine, abbiamo descritto come installare una versione funzionante della
2532 pipeline.

2533 Dalla nostra discussione emerge che `QA-Prolog` è uno stack software modulare
2534 in cui è possibile sostituire un componente della pipeline con un altro per
2535 modificare il risultato; ad esempio, abbiamo presentato un lavoro che parte da
2536 C invece che da Prolog.

2537 Questo lavoro ha il potenziale di fornire una base per molti altri esperimenti,
2538 che possono fare affidamento su un'infrastruttura già implementata e funzionante,
2539 al fine di sviluppare nuove estensioni, sia a monte che a valle, della pipeline.
2540

2541

Parte III

2542

EXPERIMENTS

In questo capitolo presentiamo un esempio completo di riscrittura, partendo da un'ontologia espressa in OWL (Sezione 3.1.3), e arrivando a una formulazione Prolog adatta alla pipeline presentata nel capitolo precedente.

Seguendo la letteratura, iniziamo con la traduzione dell'ontologia in Prolog, discutendo come tradurre concetti come classi, sottoclassi, individui e relazioni. Ci concentriamo poi sull'incompatibilità di questa prima versione con QA-Prolog, evidenziamo le ragioni di tale incompatibilità e proponiamo possibili soluzioni.

Presentiamo alcuni esempi di query per dimostrare l'equivalenza delle due trasformazioni. Infine forniamo osservazioni sul risultato finale ottenuto dopo le trasformazioni eseguite da QA-Prolog.

6.1 ONTOLOGIA DI PARTENZA

Il dominio di interesse della nostra ontologia è il regno animale; questo dominio funge da pretesto per una descrizione più approfondita di quella mostrata nell'esempio dell'albero genealogico (Sezione 3.2.1). Vedremo come descrivere le specie animali, le loro abitudini alimentari e le relazioni tra gli individui.

6.1.1 Struttura dell'ontologia

In questa ontologia descriviamo una piccola parte del regno animale. La gerarchia di classi più complessa rappresenta la tassonomia degli animali: caratterizza gli esseri viventi con precisione crescente fino a identificarne la specie.

Le altre classi si concentrano invece sulle abitudini alimentari degli animali. Distinguiamo tra prede e predatori, e tra animali carnivori ed erbivori (in questo piccolo esempio, per semplicità, ignoriamo gli onnivori. La ragione sarà chiara nella prossima sezione).

Infine, consideriamo le relazioni tra gli animali. Definiamo due tipi di relazioni: relazioni parentali che collegano due animali nei ruoli di genitore e figlio, e relazioni legate alle abitudini alimentari che collegano due animali nei ruoli di predatore e preda.

La T-Box della nostra ontologia può essere vista nella Figura 6.1, definiamo anche alcuni individui e le loro relazioni, per popolare la nostra ontologia¹.

¹ Il codice di questa ontologia è disponibile su <https://github.com/DavideCamino/Thesis/blob/main/Code/Ontology/animals.rdf>.

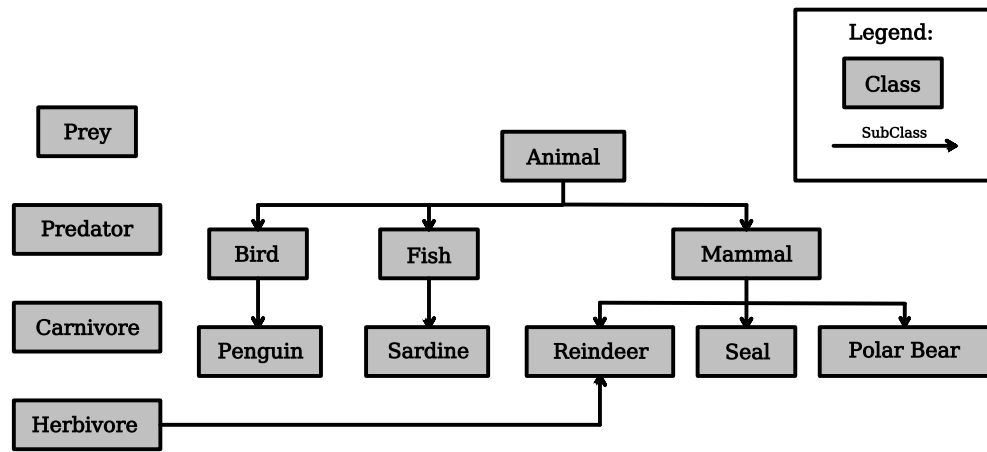


Figura 6.1: T-Box dell'ontologia

2577 6.1.2 Vincoli dell'ontologia

2578 In questa sezione presentiamo i vincoli su classi e individui rappresentati
 2579 nella nostra ontologia. Questi vincoli ci permettono di fare inferenza sul-
 2580 l'ontologia e derivare nuove informazioni da quelle già presenti. Grazie
 2581 a questi vincoli possiamo determinare se tutte le informazioni contenute
 2582 nell'ontologia sono coerenti con il modello teorico che stiamo descrivendo.

2583 I vincoli definiti nell'ontologia animale sono:
 2584 dell'ontologia

- 2584 • Vincoli sulle classi:

2585 – carnivoro ed erbivoro sono classi disgiunte;

- 2586 • Vincoli sugli individui:

2587 – Se l'animale A caccia l'animale B, A è un predatore e carnivoro e
 2588 B è una preda.

2589 Dai vincoli definiti possiamo vedere che preda e predatore non sono classi
 2590 disgiunte e essere una preda non significa automaticamente che l'animale è
 2591 erbivoro. Questo è coerente con il regno animale in cui un certo animale
 2592 potrebbe essere sia preda che predatore.

2593 Dovrebbe essere chiaro anche perché abbiamo omesso gli onnivori dalla
 2594 nostra descrizione: questa semplificazione ci permette di descrivere erbi-
 2595 voro e carnivoro come disgiunti e di dedurre che un animale che caccia è
 2596 automaticamente carnivoro.

2597 6.1.3 Contenuto dell'ontologia

2598 Esaminiamo ora alcuni fatti rappresentati nell'ontologia; questi serviranno
 2599 come esempi per mostrare come funzionano il controllo di integrità delle
 2600 informazioni raccolte nella KB e i processi di ragionamento.

2601 Iniziamo considerando le classi. Come mostrato nella T-Box della Figu-
 2602 ra 6.1, gli animali che possiamo descrivere sono: sardine, pinguini, foche,
 2603 orsi polari e renne. Dalla T-Box vediamo anche che abbiamo specificato so-
 2604 lo la renna come erbivora; le abitudini alimentari delle altre specie possono
 2605 essere dedotte attraverso la relazione di caccia.

Esaminiamo ora alcuni fatti sugli individui che popolano la nostra KB:

- Beary è un orso polare e caccia Sealy e Penguy;
- Sealy è una foca e caccia Penguy;
- Penguy è un pinguino e caccia Sardy;
- Sardy è una sardina;
- Reiny è una renna e caccia Sardy;
- Reiny_b e Reiny_c sono renne, Reiny_b è figlio di Reiny e Reiny_c è figlio di Reiny_b.

Da queste informazioni possiamo dedurre che Beary e Reiny sono predatori, Sealy e Penguy sono sia predatori che prede; e Sardy è una preda.

CONTRADDIZIONE: Una contraddizione è stata deliberatamente introdotta nell'ontologia. Dal fatto che Reiny caccia, consegue che è un animale carnivoro; tuttavia, la sua specie è etichettata come erbivora. Reiny è quindi sia erbivora che carnivora. Inoltre, erbivori e carnivori sono classi disgiunte. Questo rende la KB inconsistente.

Fonte di inconsistenza

Se utilizzassimo un reasoner per verificare la consistenza della KB, ad esempio HermiT [22], otterremmo un messaggio di errore che ci informa che l'ontologia è inconsistente e che i risultati del reasoner non possono quindi essere considerati affidabili.

Il reasoner è anche in grado di fornire una spiegazione dell'inconsistenza (mostrata nella Figura 6.2²), dalla quale è chiaro che il problema è che Reiny è contemporaneamente erbivora e carnivora.

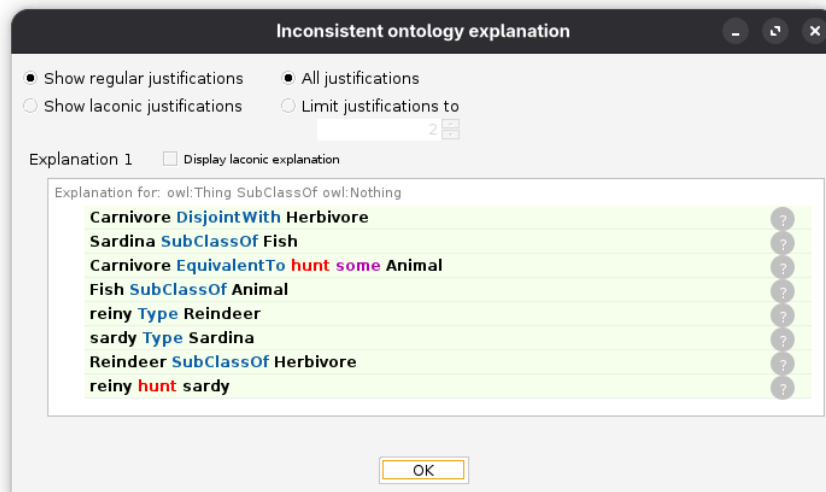


Figura 6.2: Finestra di dialogo degli errori in Protégé

Abbiamo introdotto questa inconsistenza al fine di mostrare —nelle sezioni seguenti— come sia possibile anche in Prolog rilevare questo tipo di

² La figura mostra la finestra di dialogo di Protégé [45], un editor di ontologie ampiamente usato.

2630 errore e identificare le contraddizioni nella KB. Se rimuoviamo il fatto “Reiny
2631 caccia Sardy” l’ontologia tornerà consistente.

2632 6.2 VERSIONE PROLOG

2633 Per tradurre l’ontologia in una KB Prolog, seguiamo il lavoro presentato
2634 in [46]. Questo articolo discute in profondità come replicare i vari costrutti
2635 forniti da OWL al fine di ottenere una KB logicamente equivalente in Prolog.

2636 In questa sezione vedremo come tradurre i concetti di classe e sottoclasse,
2637 classi disgiunte e relazioni tra individui; presenteremo regole per inferire
2638 nuove informazioni, come è stato fatto nell’ontologia. Infine mostreremo
2639 come rilevare errori e inconsistenze nella KB.

2640 Il nostro obiettivo non è percorrere l’intera traduzione di un’ontologia in
2641 Prolog, ma piuttosto convincerci che tale traduzione è possibile.

2642 L’ontologia originale include solo un piccolo sottoinsieme dei costrutti di-
2643 sponibili in OWL. Questo perché la traduzione in Prolog è solo il primo
2644 passo; nelle sezioni seguenti modificheremo il programma Prolog per ren-
2645 derlo compatibile con QA-Prolog e ottenere una rappresentazione adatta a
2646 un quantum annealer.

2647 Argomenti avanzati come la open world assumption, le classi enumerate,
2648 la quantificazione esistenziale, le regole di cardinalità, le gerarchie cicliche e
2649 altro sono trattati in [46].

2650 6.2.1 Classi e sottoclassi

2651 Iniziamo la nostra KB descrivendo la T-Box dell’ontologia: qui definiamo la
2652 gerarchia delle classi. Il Listato 6.1 mostra come costruire la gerarchia: (a)
2653 è un elenco di fatti che definisce le classi (ad esempio, la riga 1 può essere
2654 letta come “animale è una classe.”), (b) definisce la struttura delle classi (la
2655 riga 1 si legge come “mammifero è una sottoclasse di animale.”).

```
1 class(animal).
2 class(mammal).
3 class(polar_bear).
4 class(seal).
5 class(reindeer).
6 class(fish).
7 class(sardine).
8 class(bird).
9 class(penguin).
10 class(herbivore).
11 class(carnivore).
12 class(pre).
13 class(predator).
```

(a) Definizioni delle classi

```
1 subclass(mammal, animal).
2 subclass(bird, animal).
3 subclass(fish, animal).
4 subclass(polar_bear, mammal).
5 subclass(seal, mammal).
6 subclass(reindeer, mammal).
7 subclass(sardine, fish).
8 subclass(penguin, bird).
9 subclass(reindeer, herbivore).
```

(b) Definizioni delle sottoclassi

Listing 6.1: Definizione della T-Box

2656 Dobbiamo ora creare una regola per garantire che la relazione di sottoclas-
 2657 se soddisfi la proprietà transitiva:

*Proprietà di transitività
 della relazione di sottoclasse*

X sottoclasse di $Y \wedge Y$ sottoclasse di $Z \Rightarrow X$ sottoclasse di Z .

2658 Nella nostra ontologia, ad esempio, questa regola è necessaria per stabilire
 2659 che un pinguino è un animale, ma non un mammifero. Il modo più naturale,
 2660 in un linguaggio di programmazione logica, per imporre la proprietà transi-
 2661 tiva è attraverso una definizione ricorsiva. Il Listato 6.2 mostra come viene
 2662 definita la regola: le righe 1 e 2 implementano il caso base della ricorsione,
 2663 mentre le righe 4, 5 e 6 implementano la chiamata ricorsiva.

Regola ricorsiva

```

1 subClass(C1, C2) :-
2     subclass(C1, C2).
3 subClass(C1, C2) :-
4     subclass(C1, C3),
5     subClass(C3, C2).
```

Listing 6.2: Definizione ricorsiva del predicato sottoclasse

2664 È importante sottolineare che il predicato che stiamo definendo nel Lista-
 2665 to 6.2 è diverso da quello usato nel Listato 6.1. Il predicato `subclass(C1, C2)`
 2666 (dove `C1` e `C2` sono classi) è usato per costruire la gerarchia delle sottoclassi
 2667 con un elenco di fatti. Il predicato `subClass(C1, C2)` cattura la proprietà tran-
 2668 sitiva che ci interessa. Quest'ultimo è il predicato che deve essere usato nelle
 2669 query e in qualsiasi regola successiva ogni volta che è necessario ragionare
 2670 sulla gerarchia delle classi.

2671 6.2.2 Individui e le loro relazioni

2672 Definiamo gli individui assegnandoli a una classe; per farlo usiamo il predi-
 2673 cato `isa(I, C)` dove `I` è un individuo e `C` è una classe. Le relazioni, invece,
 2674 sono definite attraverso il predicato `hasproperty(I1, R, I2)` che significa che
 2675 l'individuo `I1` è nella relazione `R` con l'individuo `I2`. Il Listato 6.3 è una
 2676 raccolta di fatti che popolano la nostra KB e che definiscono le relazioni tra
 2677 individui.

```

1 isa(beary, polar_bear).
2 isa(sealy, seal).
3 isa(reiny, reindeer).
4 isa(reiny_b, reindeer).
5 isa(reiny_c, reindeer).
6 isa(sardy, sardine).
7 isa(penguy, penguin).
```

(a) Definizioni degli individui

```

1 hasproperty(beary, hunt, sealy).
2 hasproperty(beary, hunt, penguy).
3 hasproperty(sealy, hunt, penguy).
4 hasproperty(penguy, hunt, sardy).
5 hasproperty(reiny, hunt, sardy).
6 hasproperty(reiny_b, sonOf, reiny).
7 hasproperty(reiny_c, sonOf, reiny_b).
```

(b) Definizioni delle relazioni

Listing 6.3: Definizione della A-Box

2678 Il predicato `isa(I, C)` ci permette di definire la classe di un individuo,
 2679 ma non tiene conto della gerarchia delle classi. Ad esempio, sappiamo che
 2680 Beary è un orso polare, ma non possiamo dedurre che è un mammifero o un
 2681 animale.

Definizione del
predicato *isa*

Per considerare anche le superclassi della classe di un individuo, definiamo il predicato *isa(I, C)*. Questo predicato, grazie alla definizione ricorsiva di *subClass(C1, C2)*, permette di assegnare a un individuo tutte le classi ereditate dalla classe definita tramite il predicato *isa(I, C)*. Il Listato 6.4 mostra l'implementazione della regola *isa(I, C)*.

```

1  isa(I, C) :-
2      isa(I, C).
3  isa(I, C) :-
4      isa(I, C2),
5      subClass(C, C2).
```

Listing 6.4: Predicato *isa*

Come per i predicati *subClass(C1, C2)* e *subClass(C1, C2)*, anche *isa(I, C)* deve essere usato solo nella definizione della classe di un individuo, mentre *isa(I, C)* deve essere usato nelle query e nelle regole future.

6.2.3 Altre regole

La KB che abbiamo costruito finora contiene classi, individui e relazioni tra individui; vogliamo ora aggiungere regole che ci permettano di estrarre nuova conoscenza dai fatti contenuti nell'ontologia.

In particolare, presentiamo ora le regole per: assegnare una classe a un individuo, identificare gli antenati di un individuo, definire due classi come disgiunte e trovare inconsistenze nella KB.

ASSEGNAZIONE DI UNA CLASSE: Le regole definite nel Listato 6.5 ci permettono di assegnare la classe *carnivore* e *predatore/preda* a un individuo. Queste regole sono un esempio di come i linguaggi di programmazione logica siano facili da interpretare: possiamo leggere il Listato 6.5 come "Se un animale caccia, è *carnivore* ed è un *predatore*. Se viene cacciato, è una *preda*."

```

1  isa(I, carnivore) :-
2      hasproperty(I, hunt, _).
3  isa(I, predator) :-
4      hasproperty(I, hunt, _).
5  isa(I, prey) :-
6      hasproperty(_, hunt, I).
```

Listing 6.5: Regole di attribuzione delle classi

2702

ANTENATI: Analogamente a quanto fatto per la definizione delle sottoclassi *subClass(C1, C2)*, possiamo definire il predicato *hasproperty(I1, ancestor, I2)*. Il Listato 6.6 mostra l'implementazione di base.

Caratteristiche del
predicato *antenato*

Sebbene in un'ontologia che tratta il regno animale questo predicato possa sembrare di scarso interesse, ci fornisce un pretesto per:

- definire un predicato che può essere arricchito con altre regole: nel Listato 6.6 definiamo solo il passo base e la chiamata ricorsiva, ma

```

1  hasproperty(I1, ancestor, I2) :-
2      hasproperty(I2, sonOf, I1).
3  hasproperty(I1, ancestor, I2) :-
4      hasproperty(I2, sonOf, I3),
5      hasproperty(I1, ancestor, I3).

```

Listing 6.6: Definizione di antenato

immaginando di costruire un'ontologia che tratta le persone, possiamo arricchire il predicato con numerose regole che riflettono le relazioni familiari tra esseri umani;

- avere un predicato che può scatenare molte chiamate ricorsive³: il numero di individui che popolano un'ontologia può essere considerevole e può crescere nel tempo; inoltre, se si considerano anche relazioni complesse —come quelle suggerite nel punto precedente— il numero di antenati di un individuo può essere molto elevato e non facilmente noto a priori.

CLASSI DISGIUNTE E INCONSISTENZA: Nella nostra ontologia, l'inconsistenza logica è dovuta al fatto che la renna Reiny è sia erbivora che carnivora, ma le due classi sono disgiunte. Vediamo quindi come esprimere questa disgiunzione in Prolog e come catturare l'inconsistenza nella nostra KB.

Il Listato 6.7 mostra come definire due classi disgiunte e le conseguenze che ciò ha per gli individui. Alla riga 1 vediamo che per affermare che carnivori ed erbivori sono classi disgiunte è sufficiente un singolo fatto. Le altre righe mostrano le conseguenze che la disgiunzione ha sugli individui: se le classi *C* e *C2* sono disgiunte (riga 4) e *I* appartiene alla classe *C2* (riga 5), allora *I* non appartiene alla classe *C* (riga 3). `logicNot(P)` è un predicato che indica che la proposizione *P* è falsa.

Fornire una spiegazione dettagliata del predicato `logicNot(P)` va oltre lo scopo di questo lavoro; in [46] l'introduzione di `logicNot(P)` è giustificata al fine di portare la open world assumption in Prolog.

```

1  disjointClasses(carnivore, herbivore).
2
3  logicNot(isA(I, C)) :-
4      disjointClasses(C, C2),
5      isA(I, C2).

```

Listing 6.7: Classi disgiunte

Identificare l'inconsistenza della nostra KB significa trovare un'affermazione che sia contemporaneamente vera e falsa. In particolare, Reiny è carnivora perché caccia (riga 5 del Listato 6.3), ma non può esserlo perché è una renna e le renne sono erbivore (riga 9 del Listato 6.1).

È possibile identificare una proposizione che sia simultaneamente vera e falsa con la regola mostrata nel Listato 6.8. La testa della regola è una lista composta da una stringa (che serve solo a stampare un messaggio informativo) e dalla proposizione *P*. Affinché il predicato sia vero, la proposizione *P* deve essere contemporaneamente falsa e vera.

Regola per il rilevamento dell'inconsistenza

³ Sarà utile nella Sezione 6.3.2.

```
error(['A term cannot be true and false.', P]) :- logicNot(P), P.
```

Listing 6.8: Regola di errore

Possiamo ora verificare la consistenza della KB chiedendo all'interprete Prolog se esiste una P che sia simultaneamente falsa e vera. Il risultato della query è mostrato nel listato seguente:

```
?- error(X).
2745 X = ['A term cannot be true and false.', isA(reiny,
      ↪ carnivore)].
```

6.2.4 Risultato finale

Grazie a questo processo di traduzione, abbiamo ottenuto una KB espressa in Prolog equivalente all'ontologia originale espressa in OWL. Sebbene con alcune semplificazioni, siamo riusciti a implementare in Prolog i costrutti più importanti offerti da OWL, e abbiamo costruito un insieme di regole per eseguire inferenza sulla KB appena creata. Il processo di traduzione può essere ulteriormente generalizzato a costrutti OWL più avanzati seguendo [46].

6.3 VERSIONE QA-PROLOG

La KB che abbiamo appena ottenuto⁴ non è compatibile con la pipeline di QA-Prolog. Come presentato nella Sezione 5.1.3, QA-Prolog non implementa tutte le funzionalità di Prolog. In particolare, le liste e i termini complessi non sono supportati, e le regole non possono essere ricorsive.

In questa sezione vedremo il nostro contributo su come modificare le regole non supportate al fine di renderle compatibili con la pipeline descritta nel Capitolo 5.

6.3.1 Liste e termini complessi

Nella nostra KB abbiamo introdotto una singola lista, nel Listato 6.8, esclusivamente per stampare un messaggio aggiuntivo che spiegasse la proposizione inconsistente. La lista può semplicemente essere rimossa, oppure possiamo usare una variabile aggiuntiva che verrà legata al messaggio che vogliamo stampare.

I termini complessi sono predicati che hanno uno o più predicati come argomenti. Il termine complesso presente nella nostra KB è `logicNot(isA(I, C))` introdotto nel Listato 6.7 riga 3. Abbiamo usato questo predicato per affermare che se un individuo appartiene a una classe $C1$, disgiunta dalla classe $C2$, allora l'individuo non può appartenere a $C2$.

Possiamo sostituire il termine complesso con un nuovo predicato che rappresenta lo stesso concetto: `isNotA(I, C)`. Questo nuovo predicato ci obbli-

⁴ Disponibile su https://github.com/DavideCamino/Thesis/blob/main/Code/2_QA-prolog/3_QA-prolog/KB1/animal.pl.

2775 ga a modificare anche la regola per catturare le inconsistenze nella KB. Il
 2776 Listato 6.9 mostra il risultato finale.

```

1  isNotA(I, C) :-
2      disjointClasses(C, C2),
3      isA(I, C2).
4
5  error(I, C, M) :-
6      isA(I, C),
7      isNotA(I, C),
8      M = 'Individual X cannot belong and not belong to class Y'.
```

Listing 6.9: Rimozione di liste e termini complessi

2777 Eliminare il predicato `logicNot(P)` dove `P` è una proposizione, e quindi
 2778 possibilmente un predicato, ci obbliga a definire un nuovo predicato per
 2779 ogni affermazione che vogliamo negare, e a verificare ogni coppia *predicato-*
 2780 *negazione_predicato* per controllare eventuali inconsistenze.

2781 Nella nostra KB, solo l'appartenenza a una classe può generare inconsi-
 2782 stenza, quindi una singola regola è sufficiente. In generale, potrebbero esse-
 2783 re necessarie altre definizioni del predicato `error` per tenere conto di tutte le
 2784 possibili cause di inconsistenza⁵.

2785 6.3.2 Unfolding

2786 Nella nostra KB ci sono due regole definite ricorsivamente: `subClass(C1, C2)`
 2787 e `hasproperty(I1, R, I2)`.

2788 Per eliminare la ricorsione, possiamo *svolgere* le chiamate ricorsive. La
 2789 procedura è analoga al *loop unrolling* eseguito dai compilatori moderni per
 2790 velocizzare l'esecuzione del codice macchina. Il loop unrolling consiste nel
 2791 trasformare un ciclo di istruzioni in una sequenza ripetuta di istruzioni; que-
 2792 sto aumenta la lunghezza del codice ma riduce il numero di verifiche della
 2793 condizione del ciclo.

*Unfolding e
loop unrolling*

2794 L'unfolding consiste nel sviluppare la ricorsione ripetendo esplicitamen-
 2795 te la chiamata alla funzione; in questo modo la chiamata ricorsiva viene
 2796 sostituita da molte chiamate alla funzione A nella funzione B.

2797 **SOTTOCLASSI:** Per eliminare la ricorsione nella regola `subClass(C1, C2)`
 2798 possiamo rimuovere la regola con la chiamata ricorsiva, sostituendola con
 2799 un numero sufficiente di altre regole per coprire l'intera gerarchia di classi;
 2800 ogni nuova regola aggiunge una chiamata e permette di salire di un livello
 2801 nella gerarchia.

2802 La nostra ontologia ha una profondità massima di tre, quindi sono neces-
 2803 sarie al massimo due chiamate ricorsive per partire da qualsiasi nodo nella
 2804 gerarchia di classi e raggiungere la radice. Il Listato 6.10 mostra la versione
 2805 aggiornata della regola `subClass(C1, C2)` (per chiarire la procedura, l'unfol-
 2806 ding è sviluppato fino a tre chiamate, anche se erano necessarie solo due per
 2807 coprire l'intera gerarchia di classi).

5 Tuttavia, questo ci dà anche la possibilità di definire messaggi di errore più precisi.

```

1  subClass(C1, C2) :-
2      subclass(C1, C2).
3  subClass(C1, C3) :-
4      subclass(C1, C2),
5      subclass(C2, C3).
6  subClass(C1, C4) :-
7      subclass(C1, C2),
8      subclass(C2, C3),
9      subclass(C3, C4).

```

Listing 6.10: Unfolding delle sottoclassi

ANTENATO: Il problema con il predicato `hasproperty(I1, R, I2)` riguarda la relazione antenato-discendente. Infatti, la sua definizione era molto simile alla regola `subClass(C1, C2)`. In questo caso, tuttavia, la ricorsione è presente anche nel passo base, come si può vedere nelle righe 1 e 2 del Listato 6.6. Il Listato 6.11 mostra una possibile soluzione. Di questo codice è importante notare:

- a differenza del Listato 6.10, definiamo molti nuovi predicati: questo è utile se ci fossero più modi per definire nuovi antenati partendo da quelli già identificati⁶;
- è stato aggiunto il predicato `hasProperty(I1, R, I2)`: questo deve essere usato nelle query, mentre `hasproperty(I1, R, I2)` è usato solo per definire le relazioni tra individui;
- ci siamo fermati, arbitrariamente, dopo aver costruito un albero genealogico di profondità dieci; in generale non c'è modo di sapere quante chiamate ricorsive debbano essere svolte per essere sicuri di catturare tutti gli antenati di un individuo.

6.3.3 Criterio di arresto

Come accennato sopra, non è sempre possibile sapere in anticipo quante chiamate ricorsive verranno eseguite per catturare tutti gli aspetti che ci interessano. Qui discutiamo alcuni criteri di arresto guardando solo alla KB⁷. In generale, possiamo distinguere due casi: T-Box e A-Box.

T-BOX: La T-Box descrive il nostro dominio di interesse e dovrebbe essere stabile. Una volta sviluppata una descrizione del mondo adatta al nostro scopo, è raro che cambi; nel nostro caso, una modifica della T-Box potrebbe avvenire solo se venissero scoperte nuove specie, il che non è frequente.

Se le regole con chiamate ricorsive coinvolgono la T-Box, può essere possibile conoscere il numero massimo di chiamate ricorsive necessarie per catturare tutte le informazioni rilevanti ed eseguire un unfolding sufficiente a consentire un numero adeguato di chiamate alla funzione.

Un esempio di questo criterio di arresto è stato mostrato nella riscrittura della regola `subClass(C1, C2)`. Una volta nota la gerarchia delle classi, è

⁶ Il Listato A.2 in appendice mostra un esempio per chiarire.

⁷ Per un'analisi delle prestazioni dell'impatto dell'unfolding vedere la Sezione 6.4.3.

```

1 ancestor1(I1, I2) :-
2     hasproperty(I2, sonOf, I1).
3 ancestor2(I1, I2) :-
4     ancestor1(I1, I3),
5     hasproperty(I2, sonOf, I3).
6 ancestor3(I1, I2) :-
7     ancestor2(I1, I3),
8     hasproperty(I2, sonOf, I3).
9     ...
10 ancestor9(I1, I2) :-
11     ancestor8(I1, I3),
12     hasproperty(I2, sonOf, I3).
13
14 ancestor_unfold(I1, I2) :-
15     ancestor1(I1, I2);
16     ancestor2(I1, I2);
17     ...
18     ancestor9(I1, I2).
19
20 hasProperty(I1, P, I2) :-
21     hasproperty(I1, P, I2).
22 hasProperty(I1, ancestor, I2) :-
23     ancestor_unfold(I1, I2).

```

Listing 6.11: Unfolding degli antenati

2839 possibile calcolare la profondità massima e usarla come criterio per fermare
 2840 il processo di unfolding.

2841 **A-BOX:** La A-Box contiene gli individui che popolano l'ontologia. Per que-
 2842 sto motivo, la A-Box è molto più soggetta a cambiamenti rispetto alla T-Box
 2843 e, come discusso quando si è presentata la relazione antenato-discendente,
 2844 le relazioni tra individui possono essere anche molto complesse.

*La A-Box è più
impegnativa nel
processo di unfolding*

2845 Ogni volta che un individuo viene aggiunto alla KB o le sue relazioni
 2846 vengono modificate, la profondità delle chiamate ricorsive può cambiare.

2847 Per quanto riguarda la A-Box, il criterio di arresto per l'unfolding deve
 2848 essere deciso in anticipo o stimato tramite euristiche ove possibile. Non
 2849 è possibile conoscere in anticipo il numero massimo di chiamate ricorsive
 2850 senza eseguire un ragionamento, ma questo sarebbe controproducente poi-
 2851 ché l'obiettivo della traduzione è eseguire il ragionamento su un quantum
 2852 annealer.

2853 6.3.4 Equivalenza tra le KB

2854 Prima di sostituire la KB con costrutti non supportati e regole ricorsive con
 2855 quella modificata, dobbiamo garantire che vi sia equivalenza tra le due KB.

2856 Per mostrare l'equivalenza tra le chiamate ricorsive e la versione svol-
 2857 ta, sarebbe possibile costruire una dimostrazione basata sul principio di
 2858 induzione, possibilmente con l'ausilio di un proof assistant.

2859 Una tale dimostrazione costruttiva non è l'obiettivo di questo lavoro. Per
 2860 convincerci dell'equivalenza e tenere conto anche delle altre modifiche, te-

2861 stiamo il comportamento delle query che usano le regole riscritte per garan-
 2862 tirne la correttezza.

2863 Per quanto possibile, il processo di unfolding è stato condotto in modo
 2864 da renderlo trasparente all'utente finale. Il risultato è stato raggiunto per le
 2865 query relative alla gerarchia delle sottoclassi e al recupero degli antenati di
 2866 un individuo.

2867 Per quanto riguarda il rilevamento degli errori, tuttavia, le varie regole
 2868 che definiscono il predicato `error` possono avere un numero diverso di pa-
 2869 rametri. Per ogni numero di parametri sarà necessario testare il predicato
 2870 al fine di garantire che tutte le inconsistenze vengano identificate. Non è
 2871 necessario testare per ogni definizione del predicato; ad esempio, se ci sono
 2872 due definizioni con 3 parametri e una con 4, sarà necessario testare `error`
 2873 due volte.

2874 Il Listato 6.12 mostra le query, che usano le regole modificate per renderle
 2875 compatibili con la pipeline di QA-Prolog, e i loro risultati:

- 2876 • In (a) recuperiamo tutte le superclassi della renna, usando la versione
 2877 svolta del predicato `subClass(C1, C2)`;
- 2878 • In (b) costruiamo l'albero genealogico dell'individuo `Reiny_c` per testa-
 2879 re la nuova versione di `subClass(C1, C2)`;
- 2880 • In (c) usiamo il predicato `error(X, Y, Z)` con la definizione che evita
 2881 liste e termini complessi.

```
1 ?- subClass(reindeer, X).
2   X = mammal ;
3   X = herbivore ;
4   X = animal ;
5   false.
```

(a) Sottoclassi

```
1 ?- hasProperty(X, ancestor, reiny_c).
2   X = reiny_b ;
3   X = reiny ;
4   false.
```

(b) Antenato

```
1 ?- error(X, Y, Z).
2   X = reiny,
3   Y = carnivore,
4   Z = 'Individual X cannot belong and not belong to class Y'
```

(c) Errore

Listing 6.12: Risultati delle query

2882 Il risultato delle query è quello atteso: abbiamo ottenuto tutti e soli i ri-
 2883 sultati che avremmo ottenuto anche interrogando la KB originale. Questo ci
 2884 convince intuitivamente dell'equivalenza delle due KB.

2885 6.4 RISULTATO FINALE

2886 La KB ottenuta⁸ è compatibile con la pipeline di QA-Prolog. In questa
 2887 sezione percorriamo l'intera pipeline di trasformazione fino all'ottenimen-

⁸ Disponibile su https://github.com/DavideCamino/Thesis/blob/main/Code/2_QA-prolog/3_QA-prolog/KB1/animal_unfold.pl.

2888 to dell'Hamiltoniano che rappresenta la KB insieme alla query associata, e
 2889 discutiamo i risultati ottenuti.

2890 Formulare una query è essenziale perché definisce gli output del nostro
 2891 programma e, quindi, ci consente di costruire il circuito digitale, che è il
 2892 primo passo della pipeline.

2893 Per valutare il risultato della traduzione, non consideriamo il risultato
 2894 della valutazione della query perché, come già mostrato nel capitolo sulla
 2895 pipeline nella Sezione 5.2.3, i simulatori offerti dal framework Ocean sono
 2896 in grado di trovare la soluzione solo per istanze di problemi molto piccole
 2897 e vedremo, nella Sezione 6.4.2, che otterremo istanze di problemi con un
 2898 numero molto elevato di qubit.

2899 Ciò che consideriamo invece è la dimensione del problema di minimiz-
 2900 zazione ottenuto al termine della pipeline, estraendo il numero di qubit
 2901 che compongono il polinomio da minimizzare, e cercando di fornire una
 2902 giustificazione per le diverse dimensioni che otteniamo.

2903 6.4.1 Query

2904 Costruiamo diverse query che testano diversi fatti e regole definiti nella no-
 2905 stra KB. Consideriamo sia query semplici, la cui risposta può essere trovata
 2906 semplicemente scansionando i fatti esplicitamente elencati nella KB, sia que-
 2907 ry sempre più complesse che richiedono l'applicazione di una o più regole
 2908 di inferenza.

2909 Le query che testiamo sono:

- 2910 1. `?- class(animal).`
- 2911 2. `?- class(X).`
- 2912 3. `?- subClass(seal, X).`
- 2913 4. `?- isA(sardy, X).`
- 2914 5. `?- error(X, Y, Z).`
- 2915 6. `?- hasProperty(I1, ancestor, reiny_c).`

2916 Nei risultati faremo riferimento alle query direttamente con il numero ad
 2917 esse associato in questo elenco.

2918 6.4.2 Risultati

2919 Come si può vedere dalla Fi-
 2920 gura 6.3, il numero di qubit
 2921 necessari per rappresentare
 2922 il problema varia in modo si-
 2923 gnificativo, da poche decine,
 2924 per le prime query, a quasi
 2925 diecimila qubit, per la query
 2926 relativa agli antenati di un
 2927 individuo.

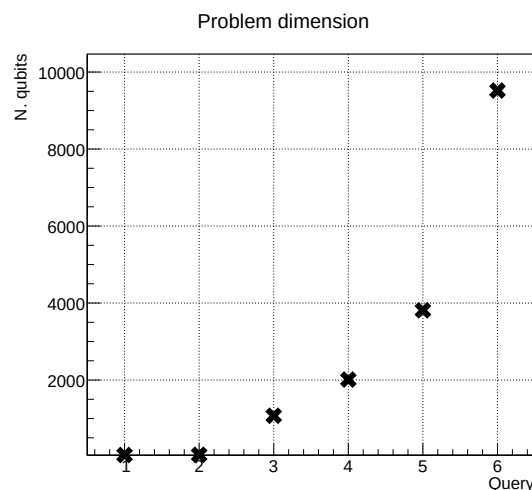


Figura 6.3: Dimensione del problema

2928 La grande differenza nel
 2929 numero di qubit utilizzati è
 2930 dovuta alla complessità del-
 2931 la query, a quanti fatti e re-
 2932 gole devono essere utilizzati
 2933 per trovarne la soluzione.

2934 Per query semplici come
 2935 le prime due, è sufficiente scansionare l'elenco dei fatti. La query 3, che
 2936 genera un problema con 1072 variabili booleane, richiede la costruzione del-
 2937 l'intera gerarchia delle classi. La query 4 (2013 qubit) richiede anch'essa la
 2938 gerarchia delle classi; inoltre, devono essere considerate tutte le regole che
 2939 ci permettono di definire un individuo come appartenente a una certa classe
 2940 (regole del Listato 6.5).

2941 La query 5 aggiunge, a tutte le regole necessarie per risolvere la query 4,
 2942 anche la disgiunzione tra classi e la regola che definisce le inconsistenze. Il
 2943 numero di variabili aumenta quindi a 3812. Infine, la query 6 usa le relazioni
 2944 tra individui e in particolare richiede la relazione che definisce gli antenati,
 2945 che è stata svolta nel Listato 6.11. Proprio a causa dell'unfolding, il numero
 2946 di regole aumenta significativamente e, di conseguenza, la dimensione del
 2947 problema cresce fino a 9519 qubit.

2948 Dai risultati ottenuti, è chiaro che, a seconda della query, otteniamo Ha-
 2949 miltoniani di dimensioni molto diverse. Questo è vantaggioso perché ci
 2950 permette di non considerare l'intera KB per ogni query, ma di estrarre solo
 2951 la parte necessaria per rispondere alla nostra domanda specifica.

2952 Ci si può quindi chiedere quale passo della pipeline sia responsabile di
 2953 questa estrazione. Osservando i file nella directory di lavoro di QA-Prolog,
 2954 si può notare che il file Verilog è lo stesso per tutte le query, mentre il file
 2955 EDIF, sintetizzato da Yosys (Sezione 5.2.2), è radicalmente diverso da una
 2956 query all'altra.

Ottimizzazione
 di Yosys 2957 L'ottimizzazione avviene quindi a livello del circuito logico: una volta
 2958 definito l'output del nostro circuito attraverso la query, Yosys è in grado di
 2959 scartare tutte quelle parti del circuito descritte in Verilog che non sono neces-
 2960 sarie per definire l'output. Yosys genera un file EDIF usando solo il modulo
 2961 Verilog connesso, in termini di circuito, all'output. Questo passo, come evi-
 2962 denziato dal grafico nella Figura 6.3, è in grado di ridurre significativamente
 2963 la dimensione del problema.

2964 6.4.3 Impatto dell'unfolding sulle prestazioni

2965 Se osserviamo il Listato 6.10 e il Listato 6.11, possiamo vedere che l'unfol-
 2966 ding dei due predicati, `subClass(C1, C2)` e `hasProperty(I1, ancestor, I2)`, è
 2967 molto simile. Tuttavia, la dimensione del problema generato dalle query
 2968 che usano questi predicati (query 3 e 6 nella Figura 6.3) è significativamente
 2969 diversa, ovvero tipicamente minore, da una query all'altra. La differenza
 2970 principale tra i due predicati è il numero di volte in cui l'unfolding è stato
 2971 effettuato.

2972 In questa sezione studiamo la dimensione del problema generato dalla
 2973 pipeline di QA-Prolog in funzione del numero di chiamate ricorsive svolte.

Al fine di isolare il più possibile l'impatto dell'unfolding, consideriamo la KB minimale e la query mostrate nel Listato 6.13.

```
1 class(animal).
2 class(mammal).
3
4 subclass(mammal, animal).
```

(a) File Prolog (KB)

```
1 ?- subclass(mammal, animal)
```

(b) Query

Listing 6.13: Input unfolding

QA-Prolog impone un numero massimo di clausole nel corpo di una regola; per superare questo limite siamo costretti a sviluppare l'unfolding come abbiamo fatto per la relazione `hasProperty(I1, ancestor, I2)` (Listato 6.11).

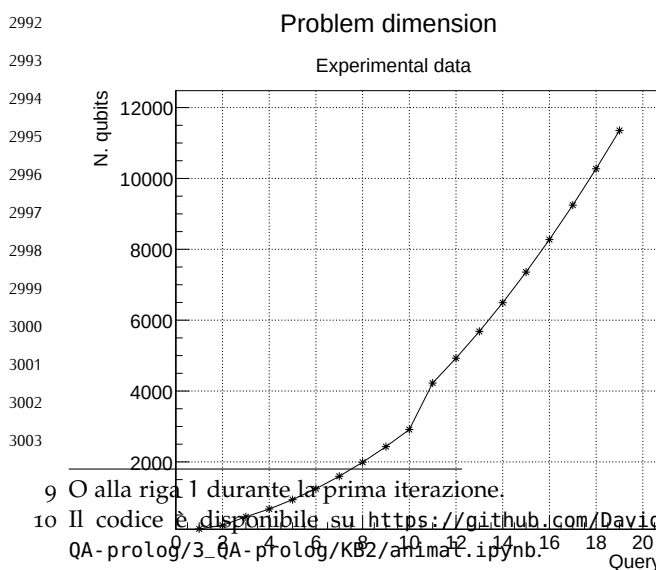
```
1 rule_0 = 'subClass0(C0,C1) :- \n\tsubclass(C0, C1).\n'
2 for i in range(1, n_unfold):
3     rule = 'subClass'+ str(i) + '(C0,C2) :-\n' + \
4           '\tsubClass' + str(i-1) + '(C0,C3),\n' + \
5           '\tsubclass(C3, C2).\n'
```

Listing 6.14: Nucleo dello script di unfolding

Poiché conosciamo l'unica regola ricorsiva della KB, possiamo automatizzare il processo di unfolding. Il Listato 6.14 mostra il nucleo della generazione delle regole: per `n_unfold` volte (riga 2) generiamo una nuova regola (riga 3) usando la regola definita nell'iterazione precedente⁹ (riga 4), aggiungendo poi un livello (riga 5).

La dimensione del problema, in termini di numero di variabili booleane, è stata valutata per alberi di profondità da due a venti. Per eseguire questi esperimenti, non solo la generazione delle regole è stata automatizzata, ma anche l'esecuzione dell'esperimento stesso¹⁰.

RISULTATI QUALITATIVI: Nella Figura 6.4 è possibile osservare l'impatto dell'unfolding sulle dimensioni del problema. Come si può notare, all'aumentare del numero di regole, aumenta anche il numero di qubit necessari per rappresentare il problema.



Un'altra osservazione qualitativa che si può fare è che il numero di variabili è correlato al numero di regole indipendentemente dalla gerarchia delle classi. Infatti, la KB (Listato 6.13) che stiamo considerando ha una profondità massima di due, avendo solo due classi (una sotto-

Figura 6.4: Impatto dell'unfolding sulla dimensione

3004 classe dell'altra). Possia-
 3005 mo quindi dedurre che
 3006 Yosys è in grado di ot-
 3007 timizzare il circuito scar-
 3008 tando tutti i componenti
 3009 non connessi all'output,
 3010 ma non è in grado di selezionare le regole effettivamente necessarie per
 3011 raggiungere la risposta alla query.

3012 **RISULTATI QUANTITATIVI:** Per comprendere quantitativamente l'impatto
 3013 dell'unfolding sulla dimensione dell'Hamiltoniano finale, tentiamo di adat-
 3014 tare i punti ottenuti sperimentalmente. Per eseguire il fitting usiamo il me-
 3015 todo dei minimi quadrati minimizzando la variabile statistica χ^2 . Le classi
 3016 di funzioni testate erano:

- 3017 • polinomio di primo grado: funzione lineare;
- 3018 • polinomio di secondo grado: parabola;
- 3019 • funzione esponenziale.

*Crescita quadratica
 della dimensione
 del problema*

3020 La funzione che meglio interpola i punti sperimentali è il polinomio di se-
 3021 condo grado. La Figura 6.5a mostra il risultato del fitting. Si può osservare
 3022 un valore molto alto di χ^2 ; questo ci porterebbe a rifiutare l'ipotesi che una
 3023 parabola interpoli correttamente i dati sperimentali. Tuttavia, si deve ricor-
 3024 dare che il test χ^2 presuppone che sia disponibile una stima ragionevole
 3025 dell'errore associato a ciascuna misura [47]: in questo caso non possiamo
 3026 associare alcun errore alla misura nemmeno ripetendo il test molte volte. La
 3027 conclusione cui possiamo comunque giungere è che, tra le classi di funzioni
 3028 studiate, la parabola è quella che meglio interpola i dati.

3029 Osservando più da vicino il grafico nella Figura 6.4, possiamo notare quel-
 3030 lo che sembra un salto nella distribuzione dei punti (tra il punto 10 e 11).
 3031 Possiamo quindi cercare di considerare due diverse funzioni quadratiche
 3032 che approssimano rispettivamente il primo insieme di punti e il secondo. La
 3033 Figura 6.5b mostra il risultato di questa ipotesi.

3034 In questo caso osserviamo un χ^2 molto basso per entrambe le funzioni di
 3035 interpolazione. Anche in questo caso, un tale risultato dovrebbe sorprender-
 3036 ci perché ottenere un adattamento così buono partendo da una distribuzione
 3037 di dati sperimentali è altamente improbabile.

3038 Nel nostro caso, un valore così basso indica che i dati raccolti si trova-
 3039 no perfettamente su una parabola, e l'assenza di errore sperimentale rende
 3040 questo accordo esatto. Il "salto" nella distribuzione dei dati potrebbe essere
 3041 dovuto alla rappresentazione interna delle regole. Al di sotto di un certo
 3042 numero di regole, esse vengono identificate con un certo numero di qubit;
 3043 una volta superato questo limite, sono necessari più qubit per identificare
 3044 ogni regola.

3045 Se questa ipotesi è corretta, dovremmo aspettarci ulteriori salti nell'anda-
 3046 mento del numero di variabili, ma la crescita rimarrà sempre quadratica.
 3047 Inoltre, i segmenti continui saranno ciascuno il doppio del precedente, per-
 3048 ché ogni qubit extra consente di raddoppiare il numero di identificatori unici
 3049 per le regole.

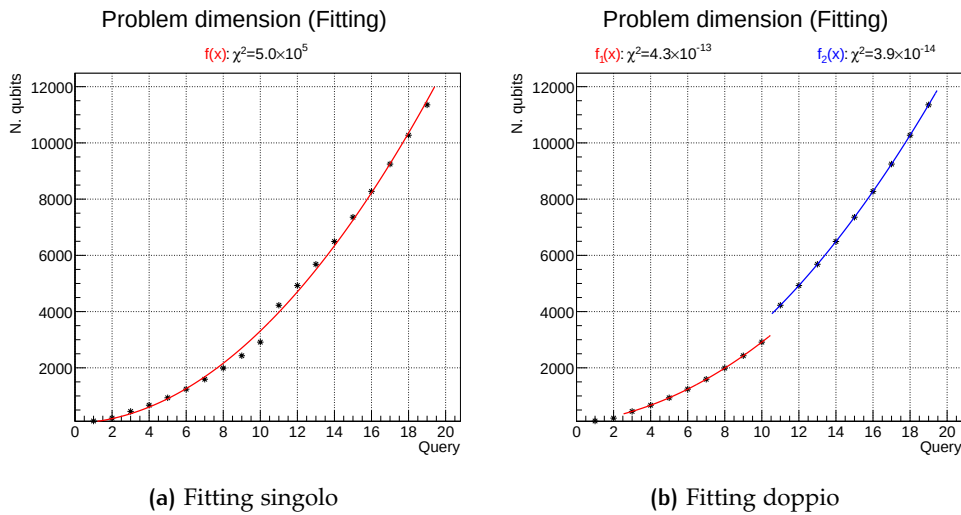


Figura 6.5: Dati adattati

3050 6.5 CONCLUSIONE

3051 In questo capitolo abbiamo mostrato come sia possibile passare da un'onto-
 3052 logia espressa in OWL a una KB espressa in Prolog su cui è possibile evi-
 3053 denziare inconsistenze nei fatti e svolgere ragionamenti come sull'ontologia
 3054 originale.

3055 Abbiamo ricordato alcune delle limitazioni di QA-Prolog, mostrando co-
 3056 me abbiano influenzato la nostra traduzione; abbiamo proposto soluzioni
 3057 che ci hanno portato a riscrivere alcune regole e, soprattutto, a svolgere le
 3058 chiamate ricorsive.

3059 Abbiamo dimostrato che la KB ottenuta è effettivamente compatibile con
 3060 la pipeline di QA-Prolog percorrendo tutti i passi fino all'ottenimento di
 3061 un Hamiltoniano, di cui abbiamo discusso le dimensioni mettendole in rela-
 3062 zione con la query e conseguentemente con i fatti e le regole necessari per
 3063 rispondere a tale query.

3064 Infine, abbiamo studiato l'impatto dell'unfolding sulla dimensione del
 3065 problema. Abbiamo ottenuto una crescita quadratica della variabile boo-
 3066 leana usata per rappresentare il nostro problema in funzione del numero di
 3067 chiamate ricorsive svolte.

3068 A un livello alto, questo capitolo può essere visto come un passo a mon-
 3069 te aggiuntivo della pipeline: QA-Prolog, attraverso successive riscritture in
 3070 formati diversi che rappresentano logicamente sempre lo stesso problema
 3071 con la stessa soluzione, parte da una KB in Prolog e arriva a una rappre-
 3072 sentazione adatta a un quantum annealer. In questo capitolo abbiamo ag-
 3073 giunto la riscrittura da OWL a Prolog, aumentando di un passo la catena di
 3074 riscritture.

3076 Nei capitoli precedenti ci siamo concentrati sull'ottenimento di un Hamil-
 3077 toniano il cui stato fondamentale rappresentasse la soluzione al nostro pro-
 3078 blema. Implicitamente o esplicitamente abbiamo fatto riferimento ai quan-
 3079 tum annealer come macchine per trovare la configurazione che minimizza
 3080 l'energia.

3081 In questo capitolo mostreremo come passare da un problema formulato
 3082 per quantum annealer a un problema formulato per computer basati su por-
 3083 te quantistiche attraverso il *Quantum Approximation Optimization Algorithm*
 3084 (QAOA), proposto per la prima volta da Farhi et al. in [48].

3085 Iniziamo descrivendo l'algoritmo QAOA, evidenziando vantaggi e svan-
 3086 taggi teorici rispetto al CAQ¹. Mostreremo poi come sia possibile tradurre
 3087 un problema formulato in forma QUBO (Sezione 2.3.3), ottenuto al termi-
 3088 ne della pipeline di QA-Prolog, in un problema adatto per i computer IBM
 3089 basati su porte quantistiche.

3090 Infine, testeremo la traduzione proposta e presenteremo alcuni risultati
 3091 sperimentali, discutendoli.

3092 7.1 QAOA

3093 L'algoritmo QAOA è ispirato al CAQ; entrambi cercano di trovare l'autovalore
 3094 minimo di una matrice che rappresenta l'energia di un sistema fisico [49].
 3095 Questa matrice descrive un'osservabile, di conseguenza è un operatore her-
 3096 mitiano e in particolare è l'Hamiltoniano $\mathbf{H}$ del sistema. Il più piccolo au-
 3097 tovalore di $\mathbf{H}$ è l'energia minima del sistema che può essere misurata; per
 3098 questo motivo è chiamato energia dello stato fondamentale.

3099 Come descritto nella Sezione 2.3.1, il CAQ parte da un Hamiltoniano il cui
 3100 stato fondamentale è facile da conoscere ed evolve, adiabatico (per rimane-
 3101 re nello stato fondamentale), verso l'Hamiltoniano con stato fondamentale
 3102 sconosciuto [49]. Il tempo di evoluzione da un Hamiltoniano all'altro può
 3103 essere esponenziale [48]; inoltre, in generale la probabilità di successo del
 3104 CAQ non aumenta all'aumentare del tempo di annealing.

3105 QAOA è un algoritmo modulare destinato a macchine basate su porte
 3106 quantistiche e, una volta ottimizzati i suoi parametri, la qualità del risultato
 3107 aumenta all'aumentare del numero di moduli, chiamati *livelli*.

Livelli di QAOA

3108 Nel modello a porte l'algoritmo è descritto dal circuito e viceversa un
 3109 circuito definisce un algoritmo. D'ora in poi usiamo i termini circuito e
 3110 algoritmo in modo intercambiabile.

¹ Calcolo Adiabatico Quantistico (Sezione 2.3.1).

7.1.1 Circuito

Il circuito QAOA prende in input n qubit, che sono le variabili booleane che vogliamo ottimizzare per minimizzare l'energia rappresentata da $\mathbf{H}$. All'inizio del circuito tutti i qubit sono impostati a zero ($|0\rangle$).

L'algoritmo QAOA inizia preparando lo stato $|s\rangle$ con massima sovrapposizione. È possibile ottenere questo stato partendo dallo stato con n qubit impostati a zero applicando la porta di Hadamard (H) a ciascuno di essi [49]; denotiamo questo operatore con $H^{\otimes n}$. Lo stato $|s\rangle$ è preparato dalla porta blu nella Figura 7.1.

Il circuito QAOA è poi composto da p livelli identici. Ogni livello è composto da due operatori (due porte quantistiche):

- *operatore di separazione di fase* (porte arancioni nella Figura 7.1): $U_P(\gamma) = e^{-i\gamma\mathbf{H}}$ dove $\mathbf{H}$ è l'Hamiltoniano del sistema e $\gamma \in [0, 2\pi]$ è un parametro;
- *operatore di mixing* (porte verdi nella Figura 7.1): $U_M(\beta) = e^{-i\beta\mathbf{B}}$ dove $\mathbf{B}$ è l'operatore che applica le porte di Pauli X a ogni qubit e $\beta \in [0, \pi]$ è un parametro.

L'operatore $U_P(\gamma)$ simula $\mathbf{H}$, mentre $U_M(\beta)$, che non commuta con $U_P(\gamma)^2$, consente di modificare la distribuzione di probabilità dell'output [49].

Usare le porte di Pauli X non è l'unico modo per costruire l'operatore di mixing; in [50, 51] vengono esplorati altri operatori di mixing.

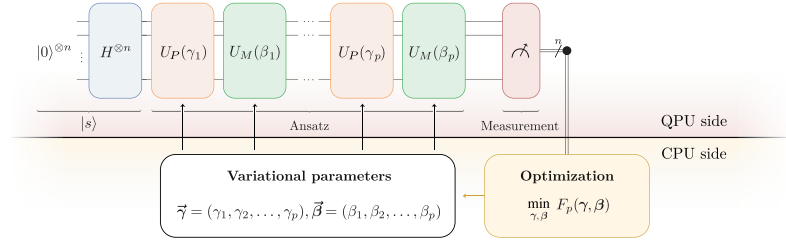


Figura 7.1: Schema QAOA [52]

La Figura 7.1 mostra lo schema generale dell'algoritmo QAOA considerando p livelli.

Dati $\vec{\gamma} = (\gamma_1, \gamma_2, \dots, \gamma_p)$ e $\vec{\beta} = (\beta_1, \beta_2, \dots, \beta_p)$, denotiamo con $|\psi(\vec{\gamma}, \vec{\beta})\rangle$ l'ansatz (una stima della soluzione) di QAOA. L'ansatz è definito come:

$$|\psi(\vec{\gamma}, \vec{\beta})\rangle = U_M(\beta_p)U_P(\gamma_p) \cdots U_M(\beta_1)U_P(\gamma_1)|s\rangle. \quad (7.1)$$

Il circuito termina con la misura; il valore atteso di questa misura è:

Energia del
sistema

$$F(\vec{\gamma}, \vec{\beta}) = \langle \psi(\vec{\gamma}, \vec{\beta}) | \mathbf{H} | \psi(\vec{\gamma}, \vec{\beta}) \rangle. \quad (7.2)$$

$F(\vec{\gamma}, \vec{\beta})$ rappresenta il valore atteso di $\mathbf{H}$ nello stato $|\psi(\vec{\gamma}, \vec{\beta})\rangle$ [49].

Una volta implementato, il valore atteso viene stimato eseguendo il circuito quantistico molte volte. Ogni esecuzione del circuito recupera un'assegnazione per gli n qubit di input; questa assegnazione rappresenta una

$$2 \quad U_P(\gamma) \times U_M(\beta) \neq U_M(\beta) \times U_P(\gamma).$$

possibile soluzione al problema con un'energia associata. L'energia media rappresenta la nostra stima $F(\vec{\gamma}, \vec{\beta})$.

Le prestazioni di QAOA dipendono dal numero di livelli; dati parametri ottimali di QAOA, l'algoritmo converge asintoticamente a soluzioni ottimali all'aumentare del numero di livelli [48].

7.1.2 Ottimizzazione dei parametri

QAOA richiede la regolazione dei parametri per minimizzare il valore obiettivo atteso per un dato numero di livelli. Formalmente i migliori parametri $(\vec{\gamma}^*, \vec{\beta}^*)$ sono definiti come:

$$(\vec{\gamma}^*, \vec{\beta}^*) = \arg \min_{\vec{\gamma}, \vec{\beta}} F(\vec{\gamma}, \vec{\beta}) \quad (7.3)$$

Questo problema di regolazione dei parametri è non convesso e **NP-difficile** [49]. *Limiti dell'ottimizzazione*
Anche con un solo livello, il gradiente di $F(\vec{\gamma}, \vec{\beta})$ svanisce rapidamente; questo fenomeno è noto come barren plateau [53, 54]. Il barren plateau si espande esponenzialmente all'aumentare del numero di qubit, rendendo il processo di regolazione ancora più difficile [54].

Il processo di ottimizzazione dei parametri viene eseguito su una CPU classica piuttosto che su un computer quantistico. Questo rende QAOA un algoritmo ibrido [49].

CONCLUSIONE: Nonostante la difficoltà nell'ottimizzare i parametri, l'algoritmo QAOA presenta numerosi aspetti interessanti e la ricerca in questo campo è molto attiva. Nel nostro lavoro ci offre la possibilità di cambiare l'architettura di riferimento e tentare di risolvere un problema formulato in QUBO —ottenuto con la pipeline di QA-Prolog— usando computer basati su porte quantistiche. Nelle prossime sezioni vedremo come implementare praticamente l'algoritmo QAOA.

7.2 DAL CAQ AL QAOA

Nella sezione precedente ci siamo concentrati sugli aspetti teorici dell'algoritmo QAOA. In questa sezione invece mostriamo come usare il Framework Qiskit di IBM per risolvere un problema di minimizzazione come quelli ottenuti al termine della pipeline di QA-Prolog.

7.2.1 Obiettivo

Al termine della pipeline di QA-Prolog otteniamo un problema in forma QUBO: in questa rappresentazione le variabili x_i da ottimizzare appartengono all'insieme $\{0, 1\}$. Quando passiamo all'architettura basata su porte quantistiche, il valore misurato dei qubit appartiene all'insieme $\{-1, 1\}$. Dobbiamo quindi passare da una rappresentazione in forma QUBO a un problema in forma Ising. Qiskit richiede inoltre che la matrice $\mathbf{H}$ sia espressa come operatori di Pauli.

3178 Con questi passi abbiamo reso l'output di QA-Prolog compatibile con l'in-
 3179 put dell'algoritmo QAOA. Per completare l'implementazione dobbiamo poi
 3180 ottimizzare i parametri definiti nelle Equazioni 7.1 e 7.2 al fine di minimiz-
 3181 zare il valore atteso di H . Infine, dopo aver trovato i parametri ottimali, cam-
 3182 pioneremo un'ultima volta il risultato del circuito quantistico per ottenere il
 3183 risultato del nostro problema.

3184 Per riassumere, per implementare l'algoritmo QAOA dobbiamo:

- 3185 • passare dalla forma QUBO alla forma Ising;
- 3186 • esprimere l'osservabile H usando operatori di Pauli;
- 3187 • ottimizzare i parametri dell'Equazione 7.2;
- 3188 • campionare il circuito quantistico ottimizzato.

3189 7.2.2 Da QUBO a Ising

3190 Nella Sezione 2.3.3 abbiamo visto che esiste sempre una traduzione da un
 3191 problema in forma QUBO alla forma Ising. È possibile implementare questa
 3192 traduzione manualmente seguendo [9]; in questo caso useremo l'implemen-
 3193 tazione fornita dalla libreria Python `qubover`.

```

1 qubo = qubover.utils.matrix_to_qubo(qubo_mat)
2 ising = qubover.utils.qubo_to_quso(qubo)
3 ising_dict = dict(ising)
4 ising_dict.pop(())
5 ising_mat = qubover.utils.qubo_to_matrix(ising_dict)

```

Listing 7.1: Dalla forma QUBO alla forma Ising

3194 Il Listato 7.1 mostra come è possibile passare da una formulazione all'altra
 3195 usando la libreria `qubover`:

- 3196 1. costruiamo un problema QUBO usando come input l'output della pi-
 3197 peline di QA-Prolog (`qubo_mat`);
- 3198 2. trasformiamo il problema QUBO in forma Ising;
- 3199 3. estraiamo il dizionario che assegna i coefficienti numerici Q_{ij} nella
 3200 Formula 2.11;
- 3201 4. scartiamo il termine costante;
- 3202 5. costruiamo la matrice con i pesi del problema in forma Ising.

Scarto dell'
offset energetico

3203 L'aspetto più interessante di questa procedura è la rimozione del termine
 3204 costante; questo valore viene usato per allineare lo stato fondamentale del
 3205 problema in forma QUBO e in forma Ising (Equazione 2.12) alla stessa ener-
 3206 gia. Poiché il nostro obiettivo è trovare lo stato fondamentale, indipenden-
 3207 temente dal suo valore, possiamo ignorare questo offset e considerare solo i
 3208 coefficienti delle variabili.

7.2.3 Hamiltoniano come operatore di Pauli

Si consideri l'Hamiltoniano triangolare superiore $\mathbf{H}$ che descrive il nostro problema in forma Ising. Ogni elemento appartenente alla diagonale $h_{i,i}$ viene tradotto in una porta di Pauli Z che misura la componente z dello spin del qubit i e applica la trasformazione identità a tutti gli altri qubit³. Gli elementi $h_{i,j}$ ⁴ vengono trasformati in porte che misurano lo spin lungo l'asse Z dei qubit i e j , e applicano l'operatore identità agli altri qubit.

Seguendo il tutorial IBM [55] trasformiamo la matrice che descrive il problema in forma Ising in operatori di Pauli. Il Listato 7.2 mostra come è possibile automatizzare questa riscrittura. Alla riga 4 generiamo l'osservabile per gli elementi sulla diagonale e alla riga 6 l'osservabile per gli altri elementi della matrice.

```
1 def build_paulis(matrix):
2     pauli_list = []
3     for i in range(len(matrix)):
4         pauli_list.append(("Z", [i], matrix[i][i]))
5         for j in range(i+1, len(matrix)):
6             pauli_list.append(("ZZ", [i, j], matrix[i][j]))
7     return pauli_list
```

Listing 7.2: Generazione dell'operatore di Pauli

3220

Possiamo infine costruire un singolo osservabile a partire dall'elenco di operatori di Pauli che abbiamo appena costruito. Per farlo usiamo una funzione della libreria fornita da Qiskit. Il Listato 7.3 mostra come ottenere la funzione di costo, dove `pauli_list` sono gli operatori di Pauli che abbiamo generato con il Listato 7.2 e `n_qubits` è il numero di qubit che descrivono il nostro problema.

```
cost_hamiltonian = SparsePauliOp.from_sparse_list(pauli_list, n_qubits)
```

Listing 7.3: Definizione della funzione di costo

3226

7.2.4 Circuito finale

Come visto nella Sezione 7.1.1 l'algoritmo QAOA non usa direttamente la funzione di costo, ma due operatori parametrici (porte $U_P(\gamma)$ e $U_M(\beta)$) ripetuti un certo numero di volte.

Qiskit fornisce al programmatore un costruttore per generare il circuito quantistico che implementa l'algoritmo QAOA; questo costruttore consente di specificare:

- funzione di costo: quella ottenuta nel Listato 7.3;
- `reps`: il numero p di livelli;

³ Ricordando l'osservazione fatta nel Paragrafo 2.2.2 dove abbiamo sottolineato che la porta di Pauli Z è sia unitaria (una porta quantistica valida) che hermitiana (quindi rappresenta un'osservabile).

⁴ Con $j > i$ perché $\mathbf{H}$ è triangolare superiore.

- 3236
- 3237
- 3238
- 3239
- 3240
- 3241
- stato iniziale: sotto forma di circuito quantistico. Se non viene specificato alcuno stato iniziale, la porta di Hadamard viene applicata a ogni qubit;
 - operatore di mixing: questo parametro può essere specificato come operatore base o come circuito quantistico. Se non viene specificato alcun parametro viene applicata la porta di Pauli X.

3242

3243

Il circuito termina misurando il valore dei qubit. Il Listato 7.4 mostra la costruzione del circuito.

```
1 qc = QAOAAnsatz(cost_operator=cost_funcrion, reps=n_reps)
2 qc.measure_all()
```

Listing 7.4: Costruzione del circuito

3244

3245

3246

3247

ESEMPIO: Considerando un problema descritto da tre qubit e il QAOA con due livelli. La Figura 7.2 mostra: in a la matrice che descrive il problema; in b il circuito in forma compatta; in c il circuito espanso per osservare come i coefficienti della matrice siano stati tradotti in osservabili.

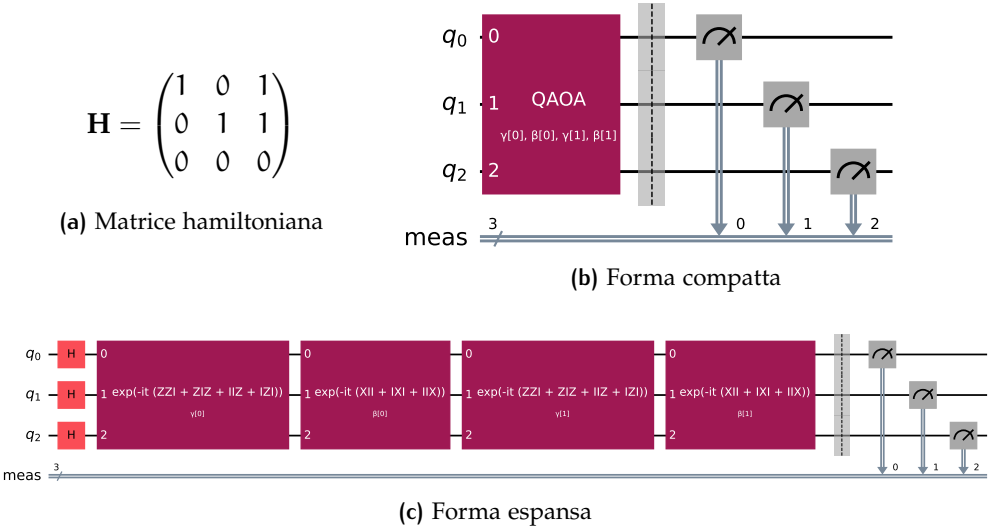


Figura 7.2: Circuito QAOA

3248

3249

3250

3251

3252

3253

3254

Esaminando attentamente gli operatori che costituiscono la porta $U_P(\gamma)$ (primo e terzo blocco nella Figura 7.2c) possiamo osservare che l'ordine dei qubit è invertito. Ad esempio, l'operatore IIZ rappresenta l'elemento di **H** in posizione (0,0): I è la porta identità applicata al terzo e al secondo qubit, e Z è la porta di Pauli Z applicata al primo qubit. L'operatore ZZI rappresenta l'elemento in posizione (1,2). Questa osservazione deve essere tenuta a mente quando si leggono i risultati del circuito.

Stringa di qubit
invertita

3255

7.2.5 Ottimizzazione dei parametri

3256

3257

Definiamo una procedura per ottenere il costo $F(\vec{\gamma}, \vec{\beta})$ (Equazione 7.2) come valore atteso del risultato dell'esecuzione del circuito quantistico definito in

precedenza. Questo costo deve essere minimizzato ottimizzando i parametri $\vec{\gamma}$ e $\vec{\beta}$. Seguendo [55], usiamo la libreria Python `scipy` per minimizzare $F(\vec{\gamma}, \vec{\beta})$.

```

1 def cost_func_estimator(params, ansatz, hamiltonian, estimator):
2     # transform the observable defined on virtual qubits to
3     # an observable defined on all physical qubits
4
5     isa_hamiltonian = hamiltonian.apply_layout(ansatz.layout)
6     pub = (ansatz, isa_hamiltonian, params)
7     job = estimator.run([pub])
8     results = job.result()[0]
9     cost = results.data.evs
10    objective_func_vals.append(cost)
11
12    return cost

```

Listing 7.5: Stima di $F(\vec{\gamma}, \vec{\beta})$

Il Listato 7.5 mostra come ottenere $F(\vec{\gamma}, \vec{\beta})$ come valore atteso:

- alla riga 6 creiamo un *primitive unified block* (`pub`): una tripla contenente il circuito da eseguire (`ansatz`), l'osservabile da misurare (`isa_hamiltonian`) e i parametri del circuito (`params`);
- alla riga 7 stimiamo $F(\vec{\gamma}, \vec{\beta})$ eseguendo più volte il circuito⁵;
- dalla riga 9 alla 12 recuperiamo e restituiamo il risultato.

```

1 with Session(backend=backend) as session:
2     estimator = Estimator(mode=session)
3     estimator.options.default_shots = 1000
4
5     result = minimize(
6         cost_func_estimator,
7         init_params,
8         args=(candidate_circuit, cost_hamiltonian, estimator),
9         method="COBYLA",
10        tol=1e-2,
11    )
12 optimized_circuit = candidate_circuit.assign_parameters(result.x)

```

Listing 7.6: Ottimizzazione di $\vec{\gamma}$ e $\vec{\beta}$

Il Listato 7.6 mostra il processo di minimizzazione: creiamo un estimator che esegue 1000 volte il circuito (righe 2 e 3) e usiamo la funzione della libreria `scipy.optimize.minimize`⁶ per minimizzare $F(\vec{\gamma}, \vec{\beta})$ ottimizzando i parametri $\vec{\gamma}$ e $\vec{\beta}$. Infine creiamo il circuito ottimizzato assegnando al circuito parametrico il valore dei parametri ottimizzati (riga 12). In questo listato, `candidate_circuit` è la versione transpilata⁷ del circuito `qc` ottenuto nel Listato 7.4.

⁵ Il numero di shot è un attributo dell'oggetto `estimator` che definiamo nel Listato 7.6.

⁶ La sua documentazione è disponibile su <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize.html>.

⁷ Sezione 4.2.2

3274 7.2.6 Recupero dei risultati

3275 Ora che abbiamo ottimizzato il valore dei parametri possiamo, un'ultima
3276 volta, eseguire il campionamento del circuito quantistico. Questa volta non
3277 siamo interessati a stimare il valore minimo di $F(\vec{\beta}, \vec{\gamma})$, ma a determinare il
3278 valore dei qubit che genera questo valore.

```
1 sampler = Sampler(mode=backend)
2 sampler.options.default_shots = 10000
3 pub = (optimized_circuit,)
4 job = sampler.run([pub], shots=int(1e4))
5 counts_bin = job.result()[0].data.meas.get_counts()
```

Listing 7.7: Campionamento del circuito

3279 Nel Listato 7.7 creiamo un `Sampler` (riga 1) che esegue il circuito ottimizza-
3280 to 10000 volte (riga 4). In questo caso usiamo un `Sampler`, e non un `Estimator`
3281 come nel Listato 7.6, perché siamo interessati al valore dei qubit piuttosto
3282 che al valore atteso di un'osservabile. Alla riga 5 recuperiamo i risultati.

3283 7.3 ESPERIMENTI

3284 Applichiamo l'algoritmo QAOA per risolvere un problema in forma QUBO
3285 ottenuto con la pipeline di QA-Prolog.

3286 7.3.1 Problema

3287 Si consideri il problema di soddisfacibilità $y = x_1 \wedge \neg(x_2 \vee x_3)$ (Equazio-
3288 ne 5.1). Questo problema è ben adatto all'applicazione di QAOA perché
3289 possiamo leggere la soluzione direttamente dal valore dei qubit⁸.

3290 Come mostrato nell'Esempio 5.2.3, partire dal codice Prolog per risolvere
3291 il problema di soddisfacibilità introduce un overhead significativo. Per mi-
3292 nimizzare il numero di qubit necessari per rappresentare il problema ci con-
3293 centriamo solo sull'ultima parte della pipeline, partendo dal codice Verilog
3294 dell'Esempio 5.2.2.

3295 *Matrici QUBO e Ising* Il problema QUBO che otteniamo è rappresentato da una matrice 11×11 ,
3296 il che significa che abbiamo bisogno di undici qubit per codificare il nostro
3297 problema.

3298 La Figura 7.3 mostra la rappresentazione matriciale del problema sia in
3299 forma QUBO che nella corrispondente forma Ising. Le due rappresentazioni
3300 hanno stati fondamentali con valori diversi, ma corrispondono alla stessa
3301 configurazione dei qubit.

3302 7.3.2 Esecuzione

3303 Per risolvere il problema di soddisfacibilità costruiamo un QAOA con $p = 10$
3304 livelli. Aumentando il numero di livelli, l'algoritmo converge sempre meglio

⁸ Discuteremo la possibilità di usare QA-Prolog per rendere leggibile l'output di QAOA nel Capitolo 8.

$$\mathbf{H} = \begin{pmatrix} 4 & 2 & -4 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -8 \\ 0 & 4 & -4 & 0 & -8 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 4 & 0 & 0 & -8 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 6 & 2 & -4 & -8 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 6 & -4 & 0 & -8 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 \end{pmatrix} \quad \mathbf{H} = \begin{pmatrix} 0.5 & 0.5 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -2 \\ 0 & 0.5 & -1 & 0 & -2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & -2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -0.5 & 0.5 & -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -0.5 & -1 & 0 & -2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

(a) Hamiltoniano QUBO

(b) Hamiltoniano Ising

Figura 7.3: Hamiltoniano del problema

3305 alla soluzione ottimale.

3306 Tuttavia, aumentare il numero di livelli porta anche a tempi di esecuzione
 3307 più lunghi e, se eseguito su hardware quantistico (e non tramite simulatori),
 3308 più è profondo il circuito, più si incontra il fenomeno della decoerenza [6,
 3309 p. 278]⁹. Inoltre, il numero di parametri da ottimizzare è uguale al doppio
 3310 del numero di livelli, quindi aumentare il numero di livelli rende anche
 3311 l'ottimizzazione dei parametri più complessa¹⁰.

3312 La Figura 7.4 mostra il proces-
 3313 so di ottimizzazione dei parametri:
 3314 man mano che vengono trovati para-
 3315 metri migliori, il costo stimato dimi-
 3316 nuisce. In questo caso, i parametri
 3317 sono stati ottimizzati dopo circa 130
 3318 iterazioni. Ogni iterazione compor-
 3319 ta 1000 esecuzioni del circuito quan-
 3320 tistico. Durante questi 1000 shot, il
 3321 circuito rimane invariato e i parame-
 3322 tri $\vec{\gamma}$ e $\vec{\beta}$ sono fissi. Dopo ogni itera-
 3323 zione (ovvero dopo 1000 shot), vie-
 3324 ne stimato il valore atteso dell'ener-
 3325 gia del sistema e i parametri $\vec{\gamma}$ e $\vec{\beta}$
 3326 vengono aggiornati.

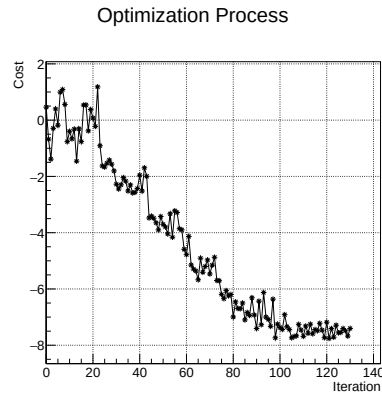
Ottimizzazione dei
parametri

Figura 7.4: Minimizzazione del costo

3327 Dopo la fase di ottimizzazione, otteniamo un'approssimazione dei para-
 3328 metri ottimali $\vec{\gamma}^*$ e $\vec{\beta}^*$ (Equazione 7.3). Possiamo quindi campionare il cir-
 3329 cuito ottimizzato per ottenere la soluzione al nostro problema. Ogni shot,
 3330 sia nella fase di ottimizzazione che nella fase di campionamento finale, uti-
 3331 lizza la stessa configurazione iniziale dei qubit. L'algoritmo inizia con tutti
 3332 i qubit nello stato $|0\rangle$, per poi preparare lo stato $|s\rangle$ applicando la porta di
 3333 Hadamard H a ogni qubit.

3334 La Figura 7.5a mostra i risultati del campionamento (10000 esecuzioni del
 3335 circuito quantistico), evidenziando in viola i quattro risultati più frequenti.

3336 L'istogramma nella Figura 7.5a è difficile da interpretare; inoltre mostra
 3337 il risultato di tutti gli undici qubit che formano il polinomio da minimizza-
 3338 re. Siamo interessati solo al valore delle variabili x_1 , x_2 e x_3 , le altre sono

Recupero dei
risultati

⁹ Anche 10 livelli sono già troppi per il computer quantistico rumoroso di oggi, tuttavia è un numero appropriato per le nostre simulazioni.

¹⁰ Come detto nella Sezione 7.1.2.

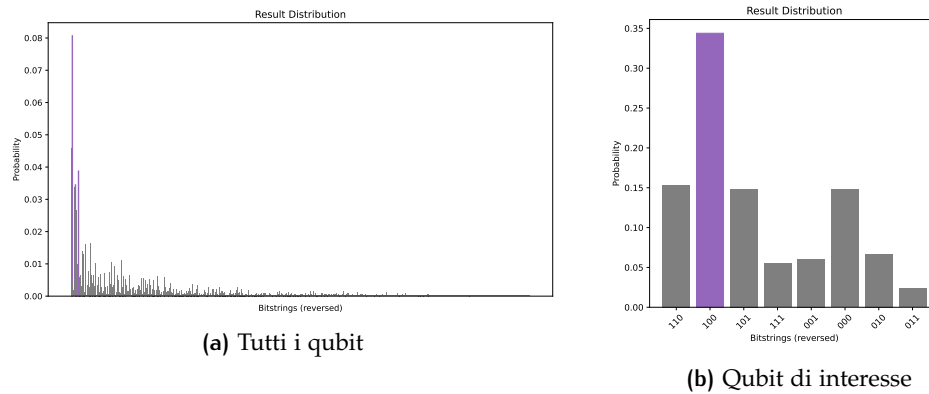


Figura 7.5: Campionamento del circuito

ausiliarie. La Figura 7.5b mostra il risultato del campionamento in cui consideriamo solo le variabili di interesse. Queste sono state estratte verificando quali qubit fossero stati mappati da QMASM alle variabili x_1 , x_2 e x_3 , e tenendo conto che la bitstring ottenuta deve essere letta in ordine inverso.

Il risultato ottenuto è quello atteso; abbiamo infatti ottenuto la stringa 1 0 0 come campione più probabile. Questi valori devono essere associati rispettivamente alle variabili x_1 , x_3 e x_2 , e possiamo facilmente verificare che questa assegnazione soddisfa l'Equazione 5.1.

7.3.3 Risultati

Sia il processo di campionamento che il processo di ottimizzazione dei parametri introducono componenti stocastiche nell'esecuzione di QAOA. Vogliamo quindi verificare, eseguendo l'ottimizzazione dei parametri e il campionamento finale più volte, con quale frequenza otteniamo il risultato atteso¹¹.

Poiché il tempo di esecuzione di ogni esperimento deve essere preso in considerazione, il numero di esperimenti eseguiti è limitato e non consente un'analisi statistica affidabile. Tuttavia, è sufficiente per mostrare come l'algoritmo QAOA non sia sempre in grado di trovare la risposta corretta al problema di minimizzazione.

La Figura 7.6 mostra i risultati (dei qubit rilevanti) di 12 ripetizioni dell'esperimento. Nell'immagine, tutti i casi in cui è stata identificata l'assegnazione corretta delle variabili x_1 , x_2 e x_3 sono evidenziati in verde. I casi in cui il risultato più probabile è errato sono evidenziati in rosso.

Il numero di esperimenti è molto basso e le analisi statistiche avanzate non sarebbero affidabili. Tuttavia, possiamo fare alcune osservazioni qualitative. Tipicamente, nei casi in cui viene trovata la soluzione corretta, la probabilità di ottenere la bitstring corretta è notevolmente più alta rispetto alla configurazione dei bit. Quando invece la soluzione è errata, i diversi risultati sono distribuiti in modo piuttosto uniforme in oltre la metà dei casi (con alcuni evidenti valori anomali).

¹¹ Il codice che automatizza l'esecuzione degli esperimenti è disponibile su https://github.com/DavideCamino/Thesis/blob/main/Code/3_QAOA/circ_sat/circ_sat_qaoa_multiple_run.ipynb.

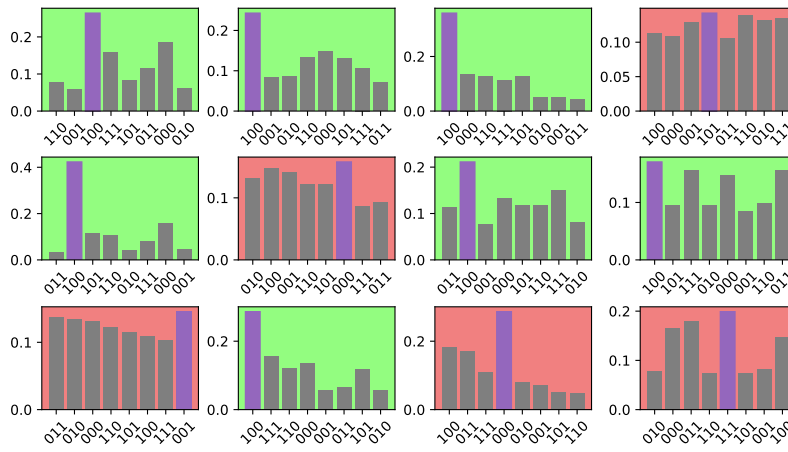


Figura 7.6: Risultato di esperimenti multipli

3368 7.4 CONCLUSIONE

3369 In questo capitolo abbiamo presentato un nuovo algoritmo per risolvere un
 3370 problema di minimizzazione in forma QUBO. Da un punto di vista teori-
 3371 co, abbiamo visto che QAOA garantisce un miglioramento monotono delle
 3372 prestazioni all'aumentare del numero di livelli (a condizione che siano stati
 3373 trovati parametri ottimali), mentre il CAQ non garantisce questa monotonia
 3374 all'aumentare del tempo di annealing. Abbiamo poi presentato gli aspetti cri-
 3375 tici di QAOA, evidenziando come il problema di trovare parametri ottimali
 3376 sia, in generale, NP-difficile.

3377 La discussione si è poi spostata sull'implementazione pratica, e abbiamo
 3378 visto come usare il Framework Qiskit per risolvere su macchine IBM basate
 3379 su porte quantistiche un problema originariamente in forma QUBO. Abbia-
 3380 mo presentato script Python per automatizzare la traduzione, abbiamo visto
 3381 come recuperare i risultati e come interpretarli. Infine, abbiamo discusso
 3382 la qualità delle soluzioni fornite da QAOA ripetendo la regolazione dei pa-
 3383 rametri e il campionamento del circuito ottimizzato, discutendo i risultati
 3384 ottenuti.

3385 Nel capitolo precedente abbiamo mostrato come aggiungere un passo di
 3386 riscrittura a monte della pipeline di QA-Prolog. Anche questo capitolo può
 3387 essere considerato come un ulteriore passo di riscrittura di un problema
 3388 in formati diversi ma equivalenti; questa volta a valle della pipeline abbia-
 3389 mo preso l'output fornito da QMASM e lo abbiamo riscritto in un formato
 3390 compatibile con QAOA.

In this chapter we briefly review the results achieved in this work, and propose some directions for future work and investigations.

8.1 RESULTS ACHIEVED

8.1.1 Updating QA-Prolog

The QA-Prolog software stack had remained without maintenance while the frameworks on which it relied were updated. As a result, the pipeline initially did not work and did not correctly interface with the D-Wave Ocean framework.

Part of the work of this thesis consisted in restoring compatibility between the software written *ad hoc* for the QA-Prolog pipeline and the other software used to complete the entire translation process.

At present, the updated version of QMASM is available¹ and allows the use of the entire QA-Prolog pipeline, that is, starting from a Prolog program, obtaining an equivalent problem in QUBO form, and retrieving the results in a readable format after solving the QUBO problem.

8.1.2 Ontology in Prolog

The work of this thesis continued with the translation of an ontology expressed in OWL (Section 3.1.3) into an equivalent knowledge base expressed in Prolog. We started by following the literature and implementing the main concepts offered by OWL in Prolog. We focused on the concepts of class and subclass, relations between individuals, and the identification of inconsistencies in the KB. We also mentioned that in the literature there are works showing how to translate more advanced concepts such as the open world assumption and enumerable classes.

QA-Prolog does not implement all the functionalities of Prolog; the main limitation is that it does not allow the definition of recursive rules. The contribution of this thesis was, therefore, to propose solutions to overcome the restrictions imposed by QA-Prolog, managing to obtain a KB equivalent to the original ontology and compatible with the QA-Prolog pipeline.

The proposed solution to address the problem of recursive calls was to perform recursion unfolding. We then focused on the drawbacks of this technique. We observed that developing many levels of recursion through unfolding leads to a significant increase in the size of the problem, and therefore in the number of qubits required to represent it. We estimated

¹ a <https://github.com/DavideCamino/qmasm>.

and discussed the quadratic increase in the size of the problem as a function of the number of unfolded recursion levels and observed that this can be problematic considering the current state of quantum machines, both gate-based and quantum annealers.

8.1.3 QAOA Algorithm

The last chapter of the thesis presents an alternative algorithm to annealing for solving an optimization problem in QUBO form such as the one obtained at the end of the QA-Prolog pipeline.

We presented the QAOA algorithm, briefly discussing its characteristics from a theoretical point of view. We then focused on automating the rewriting from the QUBO weight matrix (Section 2.3.3) to a format compatible with IBM's Qiskit framework.

After obtaining the translation, we performed some tests starting from the output of QA-Prolog and attempting to find the solution through QAOA. We carried out various tests and commented on the obtained results, observing that QAOA is not always able to return the correct solution.

8.1.4 A rewriting process

This work can be summarized as a long chain of rewritings of problems into equivalent problems. The goal is to move from one format to another, from one language to another, while preserving certain properties of interest, the most important one, of course, is that the solution of the problem must never change.

This rewriting process makes it possible to have different versions of the same problem, to observe it from different perspectives, and to choose the most convenient format to manipulate the problem depending on the objective. For example, we would like to describe the problem in the most convenient way possible, using a high-level language; on the other hand, we would like to solve it as quickly as possible, and if this means using a quantum machine, we would like to be able to move from one representation to another easily. All intermediate steps are also important because they offer different views of the problem and give us the possibility to extract properties that are not obvious at other levels of abstraction and, consequently, to optimize the problem or identify the best solving strategy.

8.2 FUTURE WORK

We present some directions for future research that can benefit from the tools developed in this work and from the results achieved.

8.2.1 Testing on quantum hardware

In this thesis we limited ourselves to testing the proposed algorithms and solutions on the simulators provided by D-Wave and IBM. The reasons for

this choice are mainly related to licensing issues. D-Wave does not allow the use of its platforms without a subscription; IBM instead offers a certain amount of computational time on its quantum machines, but without a paid plan the priority assigned to tasks is minimal, and due to time constraints it was not possible to perform satisfactory tests on quantum computers.

A possible future direction would be to verify the experimental results achieved in this thesis also on quantum hardware. On one hand, we can expect a speedup in performance due to the exploitation of quantum properties rather than their simulation on classical hardware. On the other hand, an improvement in the precision of the obtained solutions is not sure, due to noise issues that are still significant in current machines.

8.2.2 Increasing QMASM compatibility

QMASM (Section 5.2.1) is the last stage of the QA-Prolog pipeline and was developed to interface with the D-Wave Ocean framework. Thanks to QMASM it is therefore possible to submit a problem in QUBO form directly to D-Wave quantum annealers or to solve it using one of the simulators provided within the Ocean framework. These simulators are designed to test code on small instances in order to be more confident in correct execution when moving to quantum hardware. In our experiments we encountered the limitations of these simulators in Section 5.2.3.

Making QMASM compatible with other frameworks and libraries for solving QUBO problems would allow for greater flexibility in choosing the solving tool, potentially enabling both classical and quantum approaches. This would open the possibility of using simulated annealing, simulated quantum annealing, or the QAOA algorithm, in addition to quantum annealing. It would also remove the dependency on the D-Wave framework.

Interfacing QMASM with other tools is an approach fundamentally different from the one adopted in Chapter 7. There, we started from the matrix representation of the QUBO problem and translated it into a format compatible with Qiskit, thus ending all interactions with QMASM. If QMASM itself were able to interface with other solvers, we could also benefit from traversing the pipeline in reverse, that is, once the results are obtained, translating them back into a high-level language to facilitate interpretation.

In Chapter 7 we chose to implement the QAOA algorithm in Qiskit to solve the satisfiability problem, since we could directly read the solution from the final configuration of the qubits. Encoding a KB and a query would have been possible, but interpreting the results in binary format would have been extremely complex. If QMASM interfaced with Qiskit in the same way it does with Ocean, it would be possible to obtain results in a human-readable format.


```

1 // Verilog version of Prolog program friends.pl
2 // Conversion by QA-Prolog, written by Scott Pakin <pakin@lanl.gov>
3 //
4 // This program is intended to be passed to edif2qasm, then to qasm,
5 // ↪ and
6 // finally run on a quantum annealer.
7 //
8 // Note: This program uses 3 bit(s) for atoms and 1 bit(s) for
9 // ↪ (unsigned)
10 // integers.
11
12 // Define all of the symbols used in this program.
13 `define alice 3'd0
14 `define bob 3'd1
15 `define charlie 3'd2
16 `define enemies 3'd3
17 `define friends 3'd4
18 `define hates 3'd5
19
20 // Define hates(atom, atom).
21 module \hates/2 (A, B, Valid);
22 input [2:0] A;
23 input [2:0] B;
24 output Valid;
25 wire [1:0] $v1;
26 assign $v1[0] = A == `alice;
27 assign $v1[1] = B == `bob;
28 wire [1:0] $v2;
29 assign $v2[0] = A == `bob;
30 assign $v2[1] = B == `charlie;
31 assign Valid = &$v1 | &$v2;
32 endmodule
33
34 // Define enemies(atom, atom).
35 module \enemies/2 (A, B, Valid);
36 input [2:0] A;
37 input [2:0] B;
38 output Valid;
39 wire $v1;
40 \hates/2 \hates_pujkx/2 (A, B, $v1);
41 wire $v2;
42 \hates/2 \hates_DoUbH/2 (B, A, $v2);
43 assign Valid = &$v1 | &$v2;
44 endmodule
45
46 // Define friends(atom, atom).
47 module \friends/2 (A, B, Valid);

```

```

46  input [2:0] A;
47  input [2:0] B;
48  output Valid;
49  (* keep *) wire [2:0] C;
50  wire [2:0] $v1;
51  \enemies/2 \enemies_GcbiL/2 (A, C, $v1[0]);
52  \enemies/2 \enemies_GNLnA/2 (C, B, $v1[1]);
53  assign $v1[2] = A != B;
54  assign Valid = &$v1;
55  endmodule
56
57  // Define Query(atom, atom).
58  module Query (P1, P2, Valid);
59      input [2:0] P1;
60      input [2:0] P2;
61      output Valid;
62      wire $v1;
63      \friends/2 \friends_KMGjP/2 (P1, P2, $v1);
64      assign Valid = &$v1;
65  endmodule

```

Listing A.1: QA-Prolog compilation result

```

1  ancestor1(I1, I2) :-
2      hasproperty(I2, sonOf, I1).
3  ancestor1(I1, I2) :-
4      hasproperty(I2, sonOf, I3).
5      hasproperty(I1, marry, I3).
6  ancestor2(I1, I2) :-
7      hasproperty(I1, grandParentOf, I2).
8  ancestor2(I1, I2) :-
9      ancestor1(I1, I3),
10     hasproperty(I2, sonOf, I3).
11 ancestor2(I1, I2) :-
12     ancestor1(I3, I2),
13     hasproperty(I3, sonOf, I4).
14     hasproperty(I1, marry, I4).
15 ancestor3(I1, I2) :-
16     ancestor1(I3, I2)
17     hasproperty(I1, grandParentOf, I2).
18 ancestor3(I1, I2) :-
19     ancestor2(I3, I2)
20     hasproperty(I3, sonOf, I1).
21 ancestor3(I1, I2) :-
22     ancestor2(I3, I2)
23     hasproperty(I3, sonOf, I4).
24     hasproperty(I1, marry, I4).
25 ...

```

Listing A.2: Ancestor definition extended to family relation

LIST OF FIGURES

3510	Figura 1.1	Schema dell'esperimento	3
3511	Figura 1.2	Albero della disgiunzione	6
3512	Figura 2.1	Porte logiche	26
3513	Figura 2.3	Tavola di verità della porta <code>TOFFOLI</code>	33
3514	Figura 2.2	Esempio di circuito	34
3515	Figura 2.4	Sfera di Bloch [6]	35
3516	Figura 2.5	Porte come rotazioni [6]	37
3517	Figura 2.6	Porta <code>CNOT</code>	38
3518	Figura 2.7	Porta oracolo quantistico	40
3519	Figura 2.8	Oracolo di fase	40
3520	Figura 2.9	Algoritmo di Deutsch	41
3521	Figura 2.10	Simulated annealing e quantum annealing [5]	47
3522	Figura 3.1	Graph for T-Box	54
3523	Figura 3.2	Simpson family tree	54
3524	Figura 3.3	First layer of DOLCE taxonomy [18]	55
3525	Figura 3.4	Complexity classes	57
3526	Figura 4.1	Stack software Qiskit	62
3527	Figura 4.2	Esempio di transpilazione	63
3528	Figura 4.3	Stack software Ocean	64
3529	Figura 4.4	Formulazione Ising	65
3530	Figura 5.1	Pipeline di QA-Prolog	73
3531	Figura 5.2	Elaborazione con Yosys	78
3532	Figura 6.1	T-Box dell'ontologia	88
3533	Figura 6.2	Finestra di dialogo degli errori in Protégé	89
3534	Figura 6.3	Dimensione del problema	99
3535	Figura 6.4	Impatto dell'unfolding sulla dimensione	101
3536	Figura 6.5	Dati adattati	103
3537	Figura 7.1	Schema QAOA [52]	106
3538	Figura 7.2	Circuito QAOA	110
3539	Figura 7.3	Hamiltoniano del problema	113
3540	Figura 7.4	Minimizzazione del costo	113
3541	Figura 7.5	Campionamento del circuito	114
3542	Figura 7.6	Risultato di esperimenti multipli	115

3544 LIST OF LISTINGS

3545	Listing 3.1	Definition of parents	54
3546	Listing 4.1	Costruzione dello stato di Bell	62
3547	Listing 4.2	Transpilazione	63
3548	Listing 4.3	Esecuzione simulata	63
3549	Listing 4.4	Esempio Ising	65
3550	Listing 4.5	Riscrittura del MVC con pyQUBO	66
3551	Listing 4.6	Riscrittura del MVC con qubover	67
3552	Listing 5.1	KB di base	71
3553	Listing 5.2	Definizione della T-Box	75
3554	Listing 5.3	Soddisfacibilità del circuito	76
3555	Listing 5.4	Risultati della soddisfacibilità del circuito	77
3556	Listing 5.5	Hamiltoniano simbolico	78
3557	Listing 5.6	KB il nemico del mio nemico è mio amico	79
3558	Listing 5.7	Soddisfacibilità in Prolog	81
3559	Listing 5.8	Errore di macro non definite	82
3560	Listing 5.9	Nucleo del preprocessore	83
3561	Listing 6.1	Definizione della T-Box	90
3562	Listing 6.2	Definizione ricorsiva del predicato sottoclasse	91
3563	Listing 6.3	Definizione della A-Box	91
3564	Listing 6.4	Predicato isA	92
3565	Listing 6.5	Regole di attribuzione delle classi	92
3566	Listing 6.6	Definizione di antenato	93
3567	Listing 6.7	Classi disgiunte	93
3568	Listing 6.8	Regola di errore	94
3569	Listing 6.9	Rimozione di liste e termini complessi	95
3570	Listing 6.10	Unfolding delle sottoclassi	96
3571	Listing 6.11	Unfolding degli antenati	97
3572	Listing 6.12	Risultati delle query	98
3573	Listing 6.13	Input unfolding	101
3574	Listing 6.14	Nucleo dello script di unfolding	101
3575	Listing 7.1	Dalla forma QUBO alla forma Ising	108
3576	Listing 7.2	Generazione dell'operatore di Pauli	109
3577	Listing 7.3	Definizione della funzione di costo	109
3578	Listing 7.4	Costruzione del circuito	110
3579	Listing 7.5	Stima di $F(\vec{\gamma}, \vec{\beta})$	111
3580	Listing 7.6	Ottimizzazione di $\vec{\gamma}$ e $\vec{\beta}$	111
3581	Listing 7.7	Campionamento del circuito	112
3582	Listing A.1	QA-Prolog compilation result	124
3583	Listing A.2	Ancestor definition extended to family relation	124

3584 BIBLIOGRAPHY

- 3586 [1] Scott Pakin. "Performing fully parallel constraint logic programming
3587 on a quantum annealer". In: *Theory and Practice of Logic Programming*
3588 18.5-6 (2018), pp. 928–949.
- 3589 [2] Leonard Susskind e Art Friedman. *Quantum mechanics: the theoretical*
3590 *minimum*. Basic Books, 2014.
- 3591 [3] Michael A Nielsen e Isaac L Chuang. *Quantum computation and*
3592 *quantum information*. Cambridge university press, 2010.
- 3593 [4] Catherine C McGeoch. *Adiabatic quantum computation and quantum*
3594 *annealing: Theory and practice*. Springer Nature, 2022.
- 3595 [5] Bikas K. Chakrabarti e Arnab Das. "Transverse Ising model, glass and
3596 quantum annealing". In: *Quantum Annealing and Other Optimization*
3597 *Methods*. Springer, 2005, pp. 1–36.
- 3598 [6] Thomas G Wong. *Introduction to classical and quantum computing*.
3599 Rooted Grove Omaha, Nebraska, 2022.
- 3600 [7] Jong-Jean Kim. "Ergodicity, replica symmetry, spin glass and quan-
3601 tum phase transition". In: *Quantum Annealing and Other Optimization*
3602 *Methods*. Springer, 2005, pp. 101–129.
- 3603 [8] Sei Suzuki e Masato Okada. "Simulated quantum annealing by the
3604 real-time evolution". In: *Quantum Annealing and Other Optimization*
3605 *Methods*. Springer, 2005, pp. 207–238.
- 3606 [9] Zhengbing Bian et al. "The Ising model: teaching an old problem new
3607 tricks". In: *D-wave systems 2* (2010), pp. 1–32.
- 3608 [10] Barry Smith. "Ontology". In: (2012).
- 3609 [11] Gian Piero Zarri. "Ontologies and Their Practical Implementation".
3610 In: *Encyclopedia of Database Technologies and Applications*. IGI Global
3611 Scientific Publishing, 2005, pp. 438–449.
- 3612 [12] Thomas R Gruber. "A translation approach to portable ontology
3613 specifications". In: *Knowledge acquisition 5.2* (1993), pp. 199–220.
- 3614 [13] Marco Fossati, Emilio Dorigatti e Claudio Giuliano. "N-ary rela-
3615 tion extraction for simultaneous T-Box and A-Box knowledge base
3616 augmentation". In: *Semantic Web 9.4* (2018), pp. 413–439.
- 3617 [14] Giuseppe De Giacomo, Maurizio Lenzerini et al. "TBox and ABox
3618 reasoning in expressive description logics." In: *KR 96*.316–327 (1996),
3619 p. 10.
- 3620 [15] Markus Krötzsch, Frantisek Simancik e Ian Horrocks. "A description
3621 logic primer". In: *arXiv preprint arXiv:1201.4089* (2012).
- 3622 [16] Pascal Hitzler et al. *OWL 2 Web Ontology Language Primer (Second Edi-*
3623 *tion)*. W3C Recommendation REC-owl2-primer-20121211. World Wide
3624 Web Consortium (W3C), dic. 2012. URL: [https://www.w3.org/TR/](https://www.w3.org/TR/2012/REC-owl2-primer-20121211/)
3625 [2012/REC-owl2-primer-20121211/](https://www.w3.org/TR/2012/REC-owl2-primer-20121211/).

- 3626 [17] Pascal Hitzler. “A review of the semantic web field”. In:
3627 *Communications of the ACM* 64.2 (2021), pp. 76–83.
- 3628 [18] Stefano Borgo et al. “DOLCE: A descriptive ontology for linguistic and
3629 cognitive engineering”. In: *Applied ontology* 17.1 (2022), pp. 45–69.
- 3630 [19] Boris Motik, Peter F. Patel-Schneider e Bernardo Cuenca Grau. *OWL 2*
3631 *Web Ontology Language: Direct Semantics (Second Edition)*. W3C Recom-
3632 mendation REC-owl2-direct-semantics-20121211. World Wide Web
3633 Consortium (W3C), dic. 2012. URL: [https://www.w3.org/TR/2012/REC-](https://www.w3.org/TR/2012/REC-owl2-direct-semantics-20121211/)
3634 [owl2-direct-semantics-20121211/](https://www.w3.org/TR/2012/REC-owl2-direct-semantics-20121211/).
- 3635 [20] Ilianna Kollia, Birte Glimm e Ian Horrocks. “Query answering over
3636 SROIQ knowledge bases with SPARQL”. In: *Proceedings of the 24th In-*
3637 *ternational Workshop on Description Logics, Barcelona, Spain*. 2011, pp. 13–
3638 16.
- 3639 [21] Franz Baader. *The description logic handbook: Theory, implementation and*
3640 *applications*. Cambridge university press, 2003.
- 3641 [22] Birte Glimm et al. “HermiT: an OWL 2 reasoner”. In: *Journal of auto-*
3642 *mated reasoning* 53.3 (2014), pp. 245–269. URL: [http://www.hermit-](http://www.hermit-reasoner.com/)
3643 [reasoner.com/](http://www.hermit-reasoner.com/).
- 3644 [23] William M Kaminsky, Seth Lloyd e Terry P Orlando. “Scalable super-
3645 conducting architecture for adiabatic quantum computation”. In: *arXiv*
3646 *preprint quant-ph/0403090* (2004).
- 3647 [24] Los Alamos National Laboratory / Scott Pakin. *QA-Prolog: Quantum*
3648 *Annealing Prolog*. Accessed: 2026-02-13, GitHub repository. URL: <https://github.com/lanl/QA-Prolog>.
- 3650 [25] Umer Farooq, Zied Marrakchi e Habib Mehrez. “FPGA architectu-
3651 res: An overview”. In: *Tree-Based Heterogeneous FPGA Architectures:*
3652 *Application Specific Exploration and Optimization* (2012), pp. 7–48.
- 3653 [26] William F Clocksin e Christopher S Mellish. *Programming in PROLOG*.
3654 Springer Science & Business Media, 2003.
- 3655 [27] Patrick Blackburn, Johan Bos e Kristina Striegnitz. *Learn Prolog Now! –*
3656 *Free Online Version*. Accessed: 2026-02-13. 2012. URL: [https://lpn.swi-](https://lpn.swi-prolog.org/lpnpage.php?pageid=online)
3657 [prolog.org/lpnpage.php?pageid=online](https://lpn.swi-prolog.org/lpnpage.php?pageid=online).
- 3658 [28] Robert Kowalski e Steve Smoliar. “Logic for problem solving”. In:
3659 *ACM SIGSOFT Software Engineering Notes* 7.2 (1982), pp. 61–62.
- 3660 [29] Christopher John Hogger. *Introduction to logic programming*. Academic
3661 Press Professional, Inc., 1984.
- 3662 [30] SWI-Prolog Developers. *SWI-Prolog Reference Manual*. Accessed: 2026-
3663 02-20. SWI-Prolog. URL: [https://www.swi-prolog.org/pldoc/doc_](https://www.swi-prolog.org/pldoc/doc_for?object=manual)
3664 [for?object=manual](https://www.swi-prolog.org/pldoc/doc_for?object=manual).
- 3665 [31] Jan Wielemaker et al. “SWI-Prolog”. In: *Theory and Practice of Logic*
3666 *Programming* 12.1-2 (2012), pp. 67–96. ISSN: 1471-0684.
- 3667 [32] Markus Triska. “The finite domain constraint solver of SWI-Prolog”. In:
3668 *International Symposium on Functional and Logic Programming*. Springer.
3669 2012, pp. 307–316.

- 3670 [33] Markus Triska. *clpfd: Constraint Logic Programming over Finite Domains*.
 3671 <https://github.com/triska/clpfd>. GitHub repository, accessed 2026-
 3672 02-20.
- 3673 [34] Markus Triska. *clpz: Constraint Logic Programming over Integers*. <https://github.com/triska/clpz>. GitHub repository, accessed 2026-02-20.
 3674
- 3675 [35] Roshan P James, Gerardo Ortiz e Amr Sabry. "Quantum computing
 3676 over finite fields". In: *arXiv preprint arXiv:1101.3764* (2011).
- 3677 [36] Mohamed W Hassan, Scott Pakin e Wu-chun Feng. "C to D-wave: a
 3678 high-level C compilation framework for quantum Annealers". In: *2019*
 3679 *IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE.
 3680 2019, pp. 1–8.
- 3681 [37] Scott Pakin. "A quantum macro assembler". In: *2016 IEEE High Per-*
 3682 *formance Extreme Computing Conference (HPEC)*. IEEE. 2016, pp. 1–
 3683 8.
- 3684 [38] Michael R Garey e David S Johnson. *Computers and intractability*. Vol. 29.
 3685 wh freeman New York, 2002.
- 3686 [39] Mike Gordon. "The semantic challenge of Verilog HDL". In: *Procee-*
 3687 *dings of tenth annual IEEE symposium on logic in computer science*. IEEE.
 3688 1995, pp. 136–145.
- 3689 [40] Scott Pakin. "Targeting classical code to a quantum annealer". In:
 3690 *Proceedings of the Twenty-Fourth International Conference on Architectural*
 3691 *Support for Programming Languages and Operating Systems*. 2019, pp. 529–
 3692 543.
- 3693 [41] Clifford Wolf, Johann Glaser e Johannes Kepler. "Yosys-a free Veri-
 3694 log synthesis suite". In: *Proceedings of the 21st Austrian Workshop on*
 3695 *Microelectronics (Austrochip)*. Vol. 97. 2013.
- 3696 [42] Robert Brayton e Alan Mishchenko. "ABC: An academic industrial-
 3697 strength verification tool". In: *International Conference on Computer*
 3698 *Aided Verification*. Springer. 2010, pp. 24–40.
- 3699 [43] Hongyu Zhang, Yuan-Fang Li e Hee Beng Kuan Tan. "Measuring de-
 3700 sign complexity of semantic web ontologies". In: *Journal of Systems and*
 3701 *Software* 83.5 (2010), pp. 803–814.
- 3702 [44] Markus Triska. "Boolean constraints in SWI-Prolog: A comprehensive
 3703 system description". In: *Science of Computer Programming* 164 (2018),
 3704 pp. 98–115.
- 3705 [45] Mark A Musen. "The protégé project: a look back and a look forward".
 3706 In: *AI matters* 1.4 (2015), pp. 4–12.
- 3707 [46] Ken Samuel et al. "Translating owl and semantic web rules into prolog:
 3708 Moving toward description logic programs". In: *Theory and Practice of*
 3709 *Logic Programming* 8.3 (2008), pp. 301–322.
- 3710 [47] G Filatrella, P Romano et al. "ELABORAZIONE STATISTICA DEI DA-
 3711 TI SPERIMENTALI con elementi di laboratorio". In: Edises, 2009,
 3712 pp. 135–136. ISBN: 978-88-7959-513-1.

- 3713 [48] Edward Farhi, Jeffrey Goldstone e Sam Gutmann. "A quantum ap-
3714 proximate optimization algorithm". In: *arXiv preprint arXiv:1411.4028*
3715 (2014).
- 3716 [49] Ramin Fakhimi e Hamidreza Validi. "Quantum approximate optimi-
3717 zation algorithm (QAOA)". In: *Encyclopedia of Optimization*. Springer,
3718 2023, pp. 1–7.
- 3719 [50] Zhihui Wang et al. "XY mixers: Analytical and numerical results for
3720 the quantum alternating operator ansatz". In: *Physical Review A* 101.1
3721 (2020), p. 012320.
- 3722 [51] Stuart Hadfield et al. "From the quantum approximate optimization
3723 algorithm to a quantum alternating operator ansatz". In: *Algorithms*
3724 12.2 (2019), p. 34.
- 3725 [52] Kostas Blekos et al. "A review on quantum approximate optimization
3726 algorithm and its variants". In: *Physics Reports* 1068 (2024), pp. 1–66.
- 3727 [53] Andrew Arrasmith et al. "Effect of barren plateaus on gradient-free
3728 optimization". In: *Quantum* 5 (2021), p. 558.
- 3729 [54] Jarrod R McClean et al. "Barren plateaus in quantum neural network
3730 training landscapes". In: *Nature communications* 9.1 (2018), p. 4812.
- 3731 [55] IBM Quantum. *Quantum Approximate Optimization Algorithm*. IBM
3732 Quantum documentation tutorial, accessed 2026-03-09. URL: [https://](https://quantum.cloud.ibm.com/docs/en/tutorials/quantum-approximate-optimization-algorithm)
3733 [quantum.cloud.ibm.com/docs/en/tutorials/quantum-approximate-](https://quantum.cloud.ibm.com/docs/en/tutorials/quantum-approximate-optimization-algorithm)
3734 [optimization-algorithm](https://quantum.cloud.ibm.com/docs/en/tutorials/quantum-approximate-optimization-algorithm).